Management and Analysis of Physics Datasets — Module A

Master's notes

2026-10-16

PhyAMR

PhyAMR

Table of contents

1 Welcome 1
1.1 Foreword . . . 1

2 Arithmetic in computers 3
2.1 Binary numbers . . . 3
2.1.1 Multiples of bits . . . 3
2.1.2 Conversion to and from decimal . . . 4
2.2 Hexadecimal numbers . . . 5
2.3 Modular arithmetic . . . 6
2.4 Signed binary numbers . . . 7
2.5 Product of binary numbers . . . 8
2.6 Bitwise operations . . . 9
2.6.1 Bit-masking . . . 9
2.7 Real numbers . . . 10
2.7.1 Fixed point . . . 10
2.7.2 Floating point . . . 11

3 Fundamentals of Boolean Algebra 15
3.1 Postulates . . . 15
3.1.1 Proof of non-contradiction . . . 15
3.2 Theorems . . . 16
3.2.1 De Morgan's Laws . . . 16
3.3 Logic functions . . . 18
3.3.1 Canonical forms . . . 18
3.3.2 Functions with two variables . . . 19

4 From algebra to circuits 21
4.1 Logic gates . . . 21
4.1.1 Functional completeness NOR and NAND . . . 22
4.2 XOR operator . . . 23
4.3 Graphical simplification . . . 25

5 Combinatorial circuits 33
5.1 Adder . . . 33
5.1.1 Half-adder . . . 33
5.1.2 Full-adder . . . 34
5.2 Subtractor . . . 34
5.2.1 Two's complement . . . 35
5.2.2 Parallel carry . . . 36
5.3 Parity generator . . . 37
5.4 Comparison . . . 38
5.5 Multiplexer . . . 39
5.5.1 Demultiplexer . . . 40
5.6 Tristate . . . 40
5.7 Programmable devices . . . 41

PhyAMR

6 Sequential circuits 43
6.1 Flip-flop . . . 43
6.1.1 Latch . . . 43
6.1.2 S-R Flip-flop . . . 44
6.1.3 J-K Flip-flop . . . 45
6.1.4 D Flip-flop . . . 47
6.2 Counters . . . 49
6.2.1 Decimal counter . . . 49
6.2.2 Johnson counter . . . 50
6.3 Registers . . . 51
6.3.1 Shift registers . . . 51
6.3.2 Memory . . . 52
6.4 State machines . . . 53

7 SAP-1 57
7.1 Architecture . . . 57
7.2 Instruction set . . . 58
7.3 Assembly language . . . 59
7.4 Timing state . . . 60
7.5 Instruction cycle . . . 61
7.5.1 Fetch cycle . . . 61
7.5.2 Execution cycles . . . 61

8 SAP-2 65
8.1 Architecture . . . 65
8.2 Instructions . . . 67
8.2.1 Memory-reference instructions . . . 67
8.2.2 Register instructions . . . 68
8.2.3 Jump instructions . . . 69
8.2.4 Logical instructions . . . 70
8.2.5 Other . . . 71
8.2.6 Overall table . . . 71

9 SAP-3 73
9.1 Instructions . . . 73
9.1.1 Data transfer instructions . . . 73
9.1.2 Arithmetic instructions . . . 73
9.1.3 Rotation instructions . . . 74
9.1.4 Logical instructions . . . 75
9.1.5 Jump instructions . . . 76
9.1.6 Extended-register instructions . . . 76
9.1.7 Indirect instructions . . . 77
9.1.8 Stack instructions . . . 77
9.1.9 Overall table . . . 79

10 Exercises 81
10.1 Exam — 22 November 2022 . . . 81
10.1.1 From the exam paper . . . 84
10.2 Exam — 3 February 2023 . . . 87
10.2.1 From the exam paper . . . 88
10.3 Exam — 24 February 2023 . . . 91
10.3.1 From the exam paper . . . 94

PhyAMR

10.4 Exam — 23 March 2023 96
10.4.1 From the exam paper 98
10.5 Exam — 23 June 2023 101
10.6 Exam — 21 July 2023 103
10.7 Exam — 1 September 2023 105
10.8 Exam — 13 November 2023 107
10.9 Exam — 25 January 2024 110
10.10 Exam — 22 February 2024 112
10.11 Exam — 20 June 2024 114
10.12 Exam — 11 July 2024 117
10.13 Exam — 29 August 2024 119

11 ASCII Table 123

12 Times Table of 16 125

13 Throughput calculation in RAID 0 127

14 Cheatsheet 129

PhyAMR

PhyAMR

Welcome

1.1 Foreword

Disclaimer

I am **NOT** the original author of the content in this document. My contribution is limited solely to the translation and re-organization of the original material. All credit, recognition, and compliments for the ideas, structure, and substance of the work should be directed to the original creator **Dario Liotta**.

Dario Liotta is **NOT** responsible for the management of this version or any errors that it might contain.

No copyright

All the source code of this document is private and therefore is not public for any uses. Any feedback and/or questions related to the document should be directed to the original author of this document **Dario Liotta**.

Contributing

If you want to contribute to this version of the notes open an issue at **my repository of the master**.

This is the notebook for the *Management and Analysis of Physics Datasets — Module A* course. It collects the lectures on number systems, Boolean algebra, combinatorial and sequential circuits, the SAP-1/2/3 educational architectures, and a separate chapter of exam-style exercises.

Use the sidebar to navigate the chapters, or browse straight from the table of contents below.

PhyAMR

PhyAMR

Arithmetic in computers

2.1 Binary numbers

In light of what we have seen so far, it becomes clear why computers work with the **binary number system** and not with the decimal one with which we humans are used to reasoning: the electronic components operate by alternating only two states following Boolean algebra, just as there are two possible digits in binary numbers. It is no coincidence that the minimum unit of information is called a *bit*, which is a portmanteau of the terms ***bi**nary digi**t***, whose formal definition is:

the amount of information necessary and sufficient to remove the uncertainty related to the occurrence of one of two equiprobable and mutually exclusive events

In simple terms: bits allow to discriminate between two states, and this is sufficient to guarantee the functioning of the various circuits that make up a computer; this duality, moreover, fits perfectly with Boolean logic, which operates on logical propositions that contemplate only the values *true* and *false* (as demonstrated by Shannon in 1938).

All this has been taken for granted so far: when we introduced the first combinatorial circuits (the adders and subtractors) the various numbers considered in the operations were evidently in binary form, just as the decimal counter, introduced among the sequential circuits, was nothing more than a converter of decimal digits into binary numbers. Now we will go deeper into this numerical system to better understand how information is stored, processed and manipulated within digital processors.

PIAMR

2.1.1 Multiples of bits

The bit turns out to be too small a unit of scale to describe the amount of information associated with the amount of data to be processed today. For this reason, derived units are used as multiples of the bit, indicated in the following table.

The *byte* is certainly the best-known unit deriving from the bit: it is, in the modern standard, the number of bits useful for encoding a single character. The best definition of a byte is that given by David Brailsford:

«Any agglomeration of bits that your machine deemed to be worthy of holding a character.»

Over time, different standards have existed and coexisted to define a byte: today's is the result on the one hand of reaching a sufficiently high number of values for character encoding ($2^8 = 256$) and on the other of the convenience, for reasons of calculation and optimization, of having a power of two as a fundamental unit (think, for example, of the selectors in a multiplexer).

A *nibble*, on the other hand, is equivalent to half a byte[1] and we will see its usefulness in writing bytes in hexadecimal.

[1] In the mnemonics to refer to the extended register, only the first letter of the register is used: we will therefore have *B* for *BC*, *D* for *DE* and *H* for *HL*.

Finally, a *word* is composed of the number of bits that a processor can process at a time, that is, in a single *clock* cycle; in reality, the term can be used to refer to a bit string of a specific length, where by *string* we mean a group of characters (letters or numbers, so in general bytes): for example, three characters form a *24-bit word*.

Due to the base-two numbering system, the multiples indicated by prefixes such as *kilo*, *mega*, *giga* and so on will not go as 10^3, 10^6, 10^9, ... but as 2^{10}, 2^{20}, 2^{30}, ...

Let's see, for example, how many bits are contained in a megabyte:

$$1\ MB = 2^{20}\ B = 2^{20} \cdot 8\ b = 8\,388\,608\ b$$

being B the symbol for the byte and b that for the bit.

2.1.2 Conversion to and from decimal

To understand how to switch from the binary system to decimal system (and vice versa), it is necessary to remember how a numbering system works in general.

In any system, a certain number of symbols is available, which represents the *base* of the system itself: there will be ten symbols for the decimal system and two for the binary system. The count begins by considering a digit that starts by assuming the value 1 and continues until the symbols are exhausted; at that point a *reset* occurs: this digit is set to 0 and a new digit is placed in front of it, which will also start from 1 and will be updated every 10 units.

The system just described is a *positional* type system, in the sense that each digit assumes a different weight based on its position in the number itself. Any decimal number, according to what has been explained, can be broken down as follows:

$$\begin{aligned} 36541 &= 3 \cdot 10\,000 + 6 \cdot 1000 + 5 \cdot 100 + 4 \cdot 10 + 1 \cdot 1 = \\ &= 3 \cdot 10^4 + 6 \cdot 10^3 + 5 \cdot 10^2 + 4 \cdot 10^1 + 1 \cdot 10^0 \end{aligned}$$

The general formula, called c_i the generic digit of a number N with n digits (scrolling from right to left), will be the following:

$$N = \sum_{i=1}^{n} c_i \cdot 10^{i-1}$$

The binary numbering system is analogous to what has been described: the only difference, as we know, is that the symbols are only two, 0 and 1, and therefore in the count the reset will occur much earlier; this implies that a number expressed in binary will always be less compact than the same number expressed in decimal.

The useful formula to convert numbers from binary to decimal, therefore, will be

$$\boxed{N = \sum_{i=1}^{n} c_i \cdot 2^{i-1}}$$

It now remains to understand how to convert a decimal number into a binary number. Let's stay for now in the context of the decimal system and think about the inverse operation with respect to the multiplication that we used in the decomposition, that is the division; what happens if we divide any decimal number by 10?

Let's think, for example, of the number 357: the integer division will give us 35 with a remainder of 7; repeating the process on 35 we will get 3 with a remainder of 5, and applying it one last time to 3 we will have 0 with a remainder

PhyAMR

of 3. In practice, the remainders, placed from right to left, will give us the same number again.

For the conversion to binary we will not have to do anything different, but considering the base 2: the remainders can only be 0 (in the case of an even number) or 1 (in the case of an odd number), and put in order they will form the number in the desired binary form. This division algorithm is called the **double-dabble algorithm**.

EXAMPLE 2.1 | § 2.1

Convert 82_{10} *(82 in decimal) to binary.*

Let's use the double-dabble algorithm:

$$\begin{aligned} 82//2 &= 41 \text{ with remainder } \boxed{0} \\ 41//2 &= 20 \text{ with remainder } \boxed{1} \\ 20//2 &= 10 \text{ with remainder } \boxed{0} \\ 10//2 &= 5 \text{ with remainder } \boxed{0} \\ 5//2 &= 2 \text{ with remainder } \boxed{1} \\ 2//2 &= 1 \text{ with remainder } \boxed{0} \\ 1//2 &= 0 \text{ with remainder } \boxed{1} \end{aligned}$$

So we find:

$$82_{10} = 1010010_2$$

Let's check the correctness of the result through the formula [eq:arithmetic-conversion-binary-to-decimal]:

$$\begin{aligned} N &= 1 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = \\ &= 64 + 16 + 2 = 82 \end{aligned}$$

2.2 Hexadecimal numbers

If we want to switch to a number system with a base greater than 10, the number of symbols conventionally used for digits (from 0 to 9) is no longer enough, so it is necessary to introduce new symbols. By convention, it was decided to take the letters of the Roman alphabet in uppercase form, so A, B, and so on; in the case of hexadecimal numbers, letters up to F can be found, which will represent the value 15 in decimal.

The hexadecimal number system is widely used in the field of electronics and programming, due to its base which is a direct power of two: since $16 = 2^4$, in fact, each digit in hexadecimal can always be expressed uniquely through four bits, as we can see in the following table.

We understand, therefore, that each hexadecimal number can be easily converted into binary by directly replacing the digit with the respective nibble. For the same reason, a byte (which, we remember, is the minimum unit of data that can be accessed in a computer) can be expressed by means of two hexadecimal digits, making its encoding simpler and more immediate.

The conversion to and from decimal follows the same logic as those seen for the binary system, that is, the decomposition into factors according to the base and the division algorithm, called in this case *hex-dabble algorithm*.

EXAMPLE 2.2 | § 2.2

Convert $10A4_{16}$ *to binary.*

Let's convert each single digit using the table [tab:aritmetica-esadecimale-quattro-bit]:

$$\begin{cases} 1 = 0001 \\ 0 = 0000 \\ A = 1010 \\ 4 = 0100 \end{cases} \implies 10A4_{16} = 1\ 0000\ 1010\ 0100_2$$

EXAMPLE 2.3 | § 2.2

Convert 650_{10} *to hexadecimal.*

Let's apply the hex-dabble algorithm:

$$650//16 = 40 \text{ with remainder } \boxed{10}$$
$$40//16 = 2 \text{ with remainder } \boxed{8}$$
$$2//16 = 0 \text{ with remainder } \boxed{2}$$

So we find:

$$650_{10} = 28A_{16}$$

Let's check the correctness of the result by decomposing it into factors:

$$N = 2 \cdot 16^2 + 8 \cdot 16^1 + 10 \cdot 16^0 = 512 + 128 + 10 = 650$$

2.3 Modular arithmetic

Working with bytes means working with a *finite* number of bits. Since bytes can represent numbers (within the context of arithmetic operations performed by the computer) they cannot be infinite in number, but will fall within a precise range: an 8-bit byte, for example, corresponds to 256 different states, so associated a state with a number (expressed in binary form) we will have to deal with, at most, 256 different numbers.

In this context it is useful to study an arithmetic based on integers and in finite quantity, called . It takes into account a **modulus** which, in our case, corresponds precisely to the maximum value expressible through binary digits.

Let's try, for example, to perform the sum between the binary numbers 10011111 (159) and 01100100 (100):

The most significant bit, that is the one on the far left, is the ninth, so it cannot be included in the final byte which will simply contain 00000011, that is 3: this bit, therefore, we will have to subtract $2^9 = 256$ from the result.

We will therefore write:

$$259 \equiv 3 \ (mod\ 2^9) \quad \text{or} \quad 259\ mod\ 2^9 = 3$$

In modulo 2^9, therefore, it is correct to state that $159 + 100 = 3$.

When the sum or difference between two numbers produces a number outside the range given by the number of bits available, it is said that the operation has gone into arithmetic **overflow**.

2.4 Signed binary numbers

To pass from the domain of natural numbers $\mathbb{N}$ to that of relative integers $\mathbb{Z}$ it is necessary to adopt a convention to distinguish negative numbers from positive ones, also trying to understand how the operations of addition and subtraction can return the correct values.

We will list below three different representations of relative numbers in binary form:

1. : it is the most intuitive method to understand but also the least practical from a computational point of view.
   According to this convention, the most significant bit assumes the role of sign: 0 for positive numbers and 1 for negative ones; this means that with n bits we will have one dedicated to the sign and $n - 1$ dedicated to the magnitude: in general, therefore, we will be able to represent the numbers from $-2^{n-1} - 1$ to $2^{n-1} - 1$.
   The first problem is given by the fact that the number 0 has two representations: the positive one $(00 \dots 0)$ and the negative one $(00 \dots 0)$. This complicates the machine's calculations not a little, since having two representations for what is a single value can cause undesirable behavior: just think that adding -0 to any positive number the result will return the same number but with a changed sign, and the same goes for $+0$ added to a negative number.
   The sum between two numbers, moreover, requires more steps to be performed: first it is necessary to compare the magnitude of the two numbers, in order to set as the sign bit of the result the one associated with the larger number; then, it is necessary to perform the sum between the magnitudes if they are numbers with the same sign, or perform the difference between the larger and the smaller if they are numbers with opposite signs.
2. : the convention is the same as the sign-magnitude representation, with the difference that the opposite of a number is obtained by inverting all its bits.
   The dual representation of zero is also present in this case: we will have that $00 \dots 0 \rightarrow +0$ and $11 \dots 1 \rightarrow -0$.
   The sum between two numbers, however, is simplified: by adding two numbers, in fact, the result will always be exact; the only extra element to consider is a possible final carry: if this is present, in fact, you must add 1 to the result to get the correct sum.
   To understand the reason for the addition of this final carry, also called *end-around carry*, just think of the ones' complement of a number N as the subtraction of the same number from $11 \dots 1$:

$$\bar{N} = 11 \dots 1 - N = (2^n - 1) - N$$

   By performing the sum of a number M and $\bar{N}$ you would get

$$M + \bar{N} = (M - N) + 2^n - 1$$

   and ignoring the final carry in the sum (due to the limited number of bits) means the result of a contribution of 2^n:

$$M + \bar{N} \rightarrow (M - N) - 1$$

PhyAMR

In this case, therefore, it is correct to add 1 to the final result:

$$M + \bar{N} = (M - N) - 1 + 1 = M - N$$

3. : the same convention as the ones' complement, with the difference that after inverting all the bits it is necessary to add 1 to get the number with the opposite sign.
   We had already introduced this representation when talking about the subtractor (paragraph [sec:complemento-a-due]), so we already know that it corrects the two defects of the previous conventions: zero is encoded in a single sequence, that is 00 ... 0, and for this reason the smallest representable number goes from $-2^{n-1} - 1$ to -2^{n-1}; moreover, the sum returns the desired result without problems, since the end-around carry, in a certain sense, is already in the complementation of the number itself.

EXAMPLE 2.4 | § 2.4

Calculate

$$10011 - 11100$$

using the two's complement method.

Let's write the two's complement of 11100:

$$11100 \longrightarrow 00011 + 1 = 00100$$

Let's perform the sum:

To check the correctness of the result, let's convert everything to decimal:

$$10011 \longrightarrow 16 + 2 + 1 = 19$$
$$11100 \longrightarrow 16 + 8 + 4 = 28$$

We know that $19 - 28 = -9$, so the two's complement of the result obtained must give us 9:

$$10111 \longrightarrow 01000 + 1 = 01001 \longrightarrow 8 + 1 = 9$$

2.5 Product of binary numbers

Performing the product of binary numbers, as is also true for the sum and for the difference, is equivalent to what we already know from the decimal system. The product in column will be more extensive, since binary numbers are generally less compact, but at the same time much simpler, since the product will only involve the digits 0 (which cancels the entire number by which it is multiplied) and 1 (which returns the number by which it is multiplied).

Let's see, for example, the product of 1101 and 1011:

In general, if the first number is composed of n bits and the second number is composed of m bits, the final product will be composed of $n + m$ bits.

Figure 1.1 shows the electronic implementation of the column product just seen. It considers two numbers: M, composed of the bits (m_3, m_2, m_1, m_0), and Q, composed of the bits (q_3, q_2, q_1, q_0), which will give as product P,

expressed by $(p_7, p_6, p_5, p_4, p_3, p_2, p_1, p_0)$. The product between the individual bits is simply given by the AND logic gate, and the result is added to the upper digit in the same column by means of an adder, at the same time carrying the carry to the left so that the sum is coherent along all the columns.

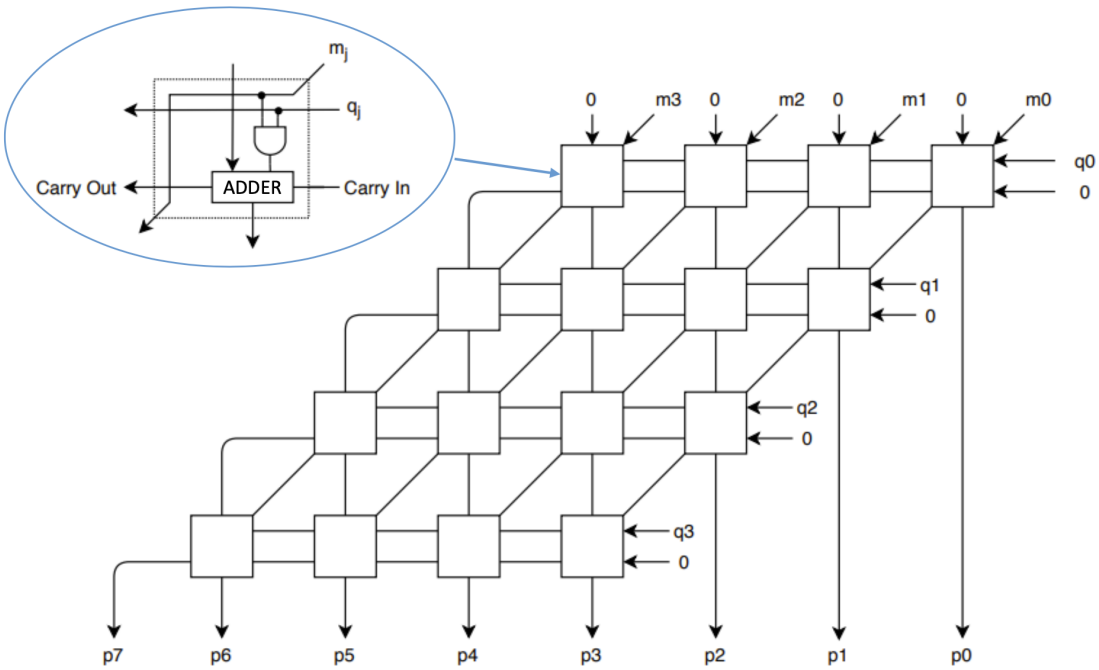


FIG. 2.1 Circuit of a multiplier between binary numbers

2.6 Bitwise operations

So far we have considered operations that involved entire bytes, but sometimes it is useful to act on single bits of information: in some memory registers, for example, each bit that composes them has a different function, so it is important to be able to access and modify the information contained in each of them. Operations that act at the bit level are called .

2.6.1 Bit-masking

This is a standard technique that makes use of *masks*, that is, sequences of bits, which are to the registers to isolate, modify or extract specific bits. They are usually indicated by hexadecimal numbers.

Let's see the four main operations applied to a register R:

1. **Set a bit to 1**: an *OR* is performed with a mask composed of 0s except for a 1 in the position of the bit concerned.
2. **Set a bit to 0**: an *AND* is performed with a mask composed of 1s except for a 0 in the position of the bit concerned.
3. **Invert a bit**: a *XOR* is performed with a mask composed of 0s except for a 1 in the position of the bit concerned.
4. **Test a bit**: an *AND* is performed with a mask composed of 0s except for a 1 in the position of the bit concerned.

EXAMPLE 2.5 | § 2.6

Check if the MSB (Most Significant Bit) of a 32-bit register is set.

To do a check on a bit of a register we must build a mask composed entirely of 0s except for a 1 in the position of the bit of our interest. Since the register is 32 bits, the mask will express the binary number $100\ldots0$, where the 0s are 31, which in hexadecimal form becomes:

$$100\ldots0_2 = 2^{31} = 2^3 \cdot 2^{28} = 2^3 \cdot \left(2^4\right)^7 = 8 \cdot 16^7 = 80000000_{16}$$

Let R be the register, therefore, the check will be given by

$$R \; AND \; 0x80000000$$

If the bit in question is equal to 0 the result will return 0x00, otherwise it will return the mask itself, that is 0x80000000.

EXAMPLE 2.6 | § 2.6

Get the third bit (starting from the least significant bit) of an 8-bit register.

We will have to repeat what we saw in the previous exercise, that is, build a mask of all 0s except for the third to last place. The conversion to hexadecimal will trivially give us 4:

$$00000100_2 = 2^2 = 4 \cdot 16^0 = 4_{16}$$

We will therefore perform:

$$R \; AND \; 0x04$$

If this operation returns 0x00 this means that the bit was 0, while if we get 0x04 then the bit was 1.

2.7 Real numbers

So far we have dealt with the set of natural numbers and relative integers, studying three different representations of signed binary numbers. With a finite number of bits we can also represent numbers belonging to $\mathbb{R}$, with the obvious consequence that they will be approximated: the greater the number of bits available for a number, the better this approximation will be.

We will see below two different representations of real numbers through bits: *fixed point* and *floating point*.

2.7.1 Fixed point

In arithmetic, a part of the bits used to encode a number are dedicated to its decimal part.

Let k be the total number of bits and r the number of bits dedicated to the decimal part, the formula that allows the conversion from fixed point to real number is

$$\boxed{N = \sum_{i=-r}^{k-r-1} c_i \cdot 2^i}$$

Let's take, for example, the number 00110110 and imagine we have $r = 4$; according to this convention the considered bit string will express:

$$N = \underbrace{0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0}_{\text{Integer part}} \underbrace{+ 0 \cdot 2^{-1} + 1 \cdot 2^{-2} + 1 \cdot 2^{-3} + 0 \cdot 2^{-4}}_{\text{Decimal part}} =$$

$$= 2 + 1 + 0.25 + 0.125 = 3 + 0.375 = 3.375$$

To convert a real number to fixed point it is necessary to reason separately for the integer part and for the decimal part: for the integer part the double-dabble algorithm that we introduced in paragraph 1.1.2 is sufficient; for the decimal part, instead, we will have to apply, in a certain sense, an inverse algorithm: we double this quantity and extract the integer part of the number obtained until we reach zero. Let's take an example with the 0.375 from the previous example:

$$0.375 \cdot 2 = 0.750 \text{ which has integer part } \boxed{0}$$
$$0.750 \cdot 2 = 1.5 \text{ which has integer part } \boxed{1}$$
$$0.5 \cdot 2 = 1 \text{ which has integer part } \boxed{1}$$
$$0 \cdot 2 = 0 \text{ which has integer part } \boxed{0}$$

which gives us exactly 0110.[2]

2.7.2 Floating point

Fixed points can represent a limited subset of $\mathbb{R}$; in order for it to be extended it is useful to switch to arithmetic.

The standard that we will see below is called **IEEE 754**, and it is the same used in the so-called *float* variables declared in the *C* programming language.

Each number is determined by 32 bits (in order):

- 1 bit for the *sign* S
- 8 bits for the *exponent* E
- 23 bits for the *mantissa* M

The conversion formula is as follows:

$$\boxed{N = (-1)^S \cdot 1.M \cdot 2^{E-127}}$$

Let's specify three important aspects:

1. The bit associated with the sign is the most significant (being the first, therefore the one on the far left): floating point arithmetic, therefore, follows the **sign-magnitude representation**
2. The exponent, being encoded by eight bits, can go from 0 to 255, so the exponential will assume as minimum value 2^{-127} and as maximum value 2^{128}

[2] Not necessarily located in the last memory addresses.

PhyAMR

3. The mantissa is calculated exactly like the decimal part in fixed point arithmetic

Let's consider a very simple example:

$$10111111101000000000000000000000$$

The mantissa consists solely of 0s except for the second bit, which being associated with 2^{-2} will give the result 0.25. Putting it all together we get:

$$N = (-1)^1 \cdot 1.25 \cdot 2^{127-127} = -1.25$$

Let's now see how the two main operations in floating point arithmetic take place:

1. **Sum between floating points**
   As in the sign-magnitude representation, the sign of the result is established based on a comparison between the magnitudes of the numbers.
   That said, the sum we will have to perform will be

   $$M_1 \cdot 2^{E_1} + M_2 \cdot 2^{E_2}$$

   The number with the lowest exponent is identified: the mantissa relative to it will be redefined in such a way that the two exponents are equal to each other. Assuming that E_1 is the lowest exponent we will pass from the mantissa M_1 to the mantissa M_1' so that:

   $$M_1 \cdot 2^{E_1} = M_1' \cdot 2^{E_2}$$

   Substituting in [eq:aritmetica-floating-point-somma] we get:

   $$M_1' \cdot 2^{E_2} + M_2 \cdot 2^{E_2} = \left(M_1' + M_2\right) 2^{E_2}$$

   which can finally be normalized to $1.M \cdot 2^E$.
   Let's see the following example:

   $$1.0001 \cdot 2^2 + 1.1101 \cdot 2^4$$

   The smallest exponent is that of the first number, so let's rewrite this number so that the exponents are equal:

   $$0.010001 \cdot 2^4 + 1.1101 \cdot 2^4 = (0.010001 + 1.1101)2^4$$

   By renormalizing we finally have:

   $$10.000101 \cdot 2^4 = 1.0000101 \cdot 2^5$$

2. **Product between floating points**
   It is sufficient to apply the algebraic rules we know:

   $$M_1 \cdot 2^{E_1} \cdot M_2 \cdot 2^{E_2} = (M_1 + M_2)\, 2^{E_1+E_2}$$

   Remember that the result must be normalized to $1.M \cdot 2^E$.

PhyAMR

The same, of course, also applies to the ratio between floating points:

$$\frac{M_1 \cdot 2^{E_1}}{M_2 \cdot 2^{E_2}} = \frac{M_1}{M_2} \cdot 2^{E_1 - E_2}$$

EXAMPLE 2.7 | § 2.7

Calculate

$$23.75_{10} + 4.5_{10}$$

using floating point.

To convert the numbers to binary you would have to use the double-dabble algorithm for the integer part and the corresponding algorithm for the decimal part; however, being particularly simple in this case, I will trivially write the numbers in the form of contributions in powers of two:

$$23.75_{10} = 2^4 + 2^2 + 2^1 + 2^0 + 2^{-1} + 2^{-2} = 10111.11_2$$

$$4.5_{10} = 2^2 + 2^{-1} = 100.1_2$$

In floating point form the sum we have to perform is

$$1.011111 \cdot 2^4 + 1.001 \cdot 2^2$$

Let's make sure to equalize the powers of two:

$$1.011111 \cdot 2^4 + 0.01001 \cdot 2^4 = (1.011111 + 0.01001)2^4$$

Let's perform the sum:

So we get:

$$1.110001 \cdot 2^4$$

To check the correctness of the result, let's convert this number to decimal:

$$11100.01_2 = 2^4 + 2^3 + 2^2 + 2^{-2} = 16 + 8 + 4 + 0.25 = 28.25_{10}$$

which is exactly $23.75_{10} + 4.5_{10}$.

PhyAMR

Fundamentals of Boolean Algebra

In Boolean algebra, variables can only take on two values, generally indicated as 0 and 1 or as *false* and *true*. In the context of circuitry (logic or digital), it is of fundamental importance as it makes use of *quantized* signals, that is, a series of signals that alternate between two states: absence and presence (also called ***LOW*** and ***HIGH***).

3.1 Postulates

Formally, the algebra is formed by a M and by the two operators of (+, called ***OR***) and (·, called ***AND***).

We will therefore be dealing with a triplet $(M, +, cdot)$ that must satisfy the following properties:

The bar above a variable ($\bar{A}$) indicates the operation of *negation*, also called ***NOT***, and expresses the complementary value of the variable, so if $A = 0$ then $\bar{A} = 1$ and vice versa.

We note that all the properties listed are valid for both the sum operator and the product operator; this is actually a valid characteristic for all the theorems and statements that are given in Boolean algebra: in particular, starting from a certain expression, by exchanging the operators and replacing the complements of the variables, the expression remains valid. This is known as the , and the expression obtained through the substitutions is called the of the initial expression.

3.1.1 Proof of non-contradiction

In order for the postulates to express a coherent algebra, it is necessary that they do not contradict each other; let's verify, therefore, that the results obtained from all the various combinations are compatible with the properties introduced.

We will use the so-called *truth tables*, which we will study in more detail later when talking about Boolean functions: these are tables in which the variables considered are reported along the columns, while the values that these variables can assume are indicated along the rows, ordered in such a way as to obtain all possible combinations.

1. Commutativity is evident from the central rows, in which the result is the same even if the values are inverted with respect to the variables.
2. The neutral element is expressed in the first two rows of the *OR*, in which the sum of 0 does not modify the variable A whether it is $A = 0$ or $A = 1$; the same can also be said of the last two rows of the *AND*.
3. The complement is given in the central rows of both tables, in which the result is 1 in the case of the *OR* and 0 in the case of the *AND*.

For the last property it is necessary to write two new truth tables built from the two in table [tab:fondamenti-di-algebra-booleana-tabelle-and-or].

The columns in red represent the two members of the distributive property, and since they are equal, the two expressions will also be equal.

PhyAMR

We have thus shown that all the postulated properties are useful for defining a coherent algebra.

3.2 Theorems

Let's now prove three other properties starting from the postulates.

$T1$:

$$\begin{cases} A + A \overset{P2}{=} (A + A) \cdot 1 \overset{P3}{=} (A + A)\left(A + \bar{A}\right) \overset{P4}{=} A + A\bar{A} \overset{P3}{=} A + 0 \overset{P2}{=} A \\ AA \overset{P2}{=} AA + 0 \overset{P3}{=} AA + A\bar{A} \overset{P4}{=} A\left(A + \bar{A}\right) \overset{P3}{=} A \cdot 1 \overset{P2}{=} A \end{cases}$$

$T2$:

$$\begin{cases} A + 1 \overset{P2}{=} (A + 1) \cdot 1 \overset{P3}{=} (A + 1)\left(A + \bar{A}\right) \overset{P4}{=} A + \bar{A} \cdot 1 \overset{P2}{=} A + \bar{A} \overset{P3}{=} 1 \\ A \cdot 0 \overset{P2}{=} A \cdot 0 + 0 \overset{P3}{=} A \cdot 0 + A\bar{A} \overset{P4}{=} A\left(\bar{A} + 0\right) \overset{P2}{=} A\bar{A} \overset{P3}{=} 0 \end{cases}$$

$T3$:

$$\begin{cases} A + AB \overset{P2}{=} A \cdot 1 + AB \overset{P4}{=} A(B + 1) \overset{T2}{=} A \cdot 1 \overset{P2}{=} A \\ A(A + B) \overset{P2}{=} (A + 0)(A + B) \overset{P4}{=} A + B \cdot 0 \overset{T2}{=} A + 0 \overset{P2}{=} A \end{cases}$$

3.2.1 De Morgan's Laws

De Morgan's laws allow us to express the negation of a sum and a product of variables.

They state that:

$$\boxed{\overline{A + B} = \bar{A} \cdot \bar{B}} \qquad \overline{A \cdot B} = \bar{A} + \bar{B}$$

Let's proceed first with the proof of the negation of the sum, remembering that by the principle of duality the proof of the negation of the product will then also be immediate.

The idea is to verify what both the sum and the product of the second member with the first non-negated member give rise to:

$$A + B + \bar{A} \cdot \bar{B} \overset{P4}{=} \left(A + B + \bar{A}\right)\left(A + B + \bar{B}\right) \overset{P3}{=} (B + 1)(A + 1) \overset{T2}{=} 1 \cdot 1 = 1$$
$$(A + B) \cdot \bar{A} \cdot \bar{B} \overset{P4}{=} A \cdot \bar{A} \cdot \bar{B} + B \cdot \bar{A} \cdot \bar{B} \overset{P3}{=} \bar{B} \cdot 0 + \bar{A} \cdot 0 \overset{T2}{=} 0 + 0 = 0$$

According to property it follows that $A + B$ is the complement of $\bar{A} \cdot \bar{B}$, hence the thesis.

As already mentioned, by the principle of duality the proof of the second equality follows from the first in an immediate way; only for completeness we also report it here below:

$$A \cdot B + \bar{A} + \bar{B} \overset{P4}{=} \left(\bar{A} + \bar{B} + A\right)\left(\bar{A} + \bar{B} + B\right) \overset{P3}{=} (\bar{B} + 1)(\bar{A} + 1) \overset{T2}{=} 1 \cdot 1 = 1$$
$$A \cdot B \cdot (\bar{A} + \bar{B}) \overset{P4}{=} A \cdot B \cdot \bar{A} + A \cdot B \cdot \bar{B} \overset{P3}{=} B \cdot 0 + A \cdot 0 \overset{T2}{=} 0 + 0 = 0$$

EXAMPLE 3.1 | § 3.2

Prove that

PhAMR

$$(A + B)\left(\bar{A} + C\right) = \bar{A}B + AC$$

Let's apply the distributive property:

$$(A + B)\left(\bar{A} + C\right) = \cancel{A\bar{A}} + AC + \bar{A}B + BC$$

Let's multiply the last term on the right side by 1, which, by property , will be equal to $A + \bar{A}$:

$$(A + B)\left(\bar{A} + C\right) = AC + \bar{A}B + BC \cdot 1 = AC + \bar{A}B + BC \cdot \left(A + \bar{A}\right)$$

Let's apply the distributive property again:

$$(A + B)\left(\bar{A} + C\right) = AC + \bar{A}B + ABC + \bar{A}BC$$

Let's collect the terms in $\bar{A}B$ and in AC:

$$(A + B)\left(\bar{A} + C\right) = \bar{A}B(1 + C) + AC(1 + B) = \bar{A}B + AC$$

EXAMPLE 3.2 | § 3.2

Prove that

$$f(A, B) = \bar{A}\bar{B} + AB = \bar{f}\left(\bar{A}B\right)$$

Let's make $\bar{f}\left(\bar{A}B\right)$ explicit:

$$\bar{f}\left(\bar{A}B\right) = A\bar{B} + \bar{A}B$$

Let's apply De Morgan's law to the sum:

$$\bar{f}\left(\bar{A}B\right) = \bar{A\bar{B}} \cdot \bar{\bar{A}B}$$

Let's apply De Morgan's law again, but this time to the two products:

$$\bar{f}\left(\bar{A}B\right) = \left(\bar{A} + B\right) \cdot \left(A + \bar{B}\right)$$

Finally, let's use the distributive property:

$$\bar{f}\left(\bar{A}B\right) = \cancel{A\bar{A}} + \bar{A}\bar{B} + AB + \cancel{B\bar{B}} \equiv f(A, B)$$

3.3 Logic functions

A , also called a *Boolean function*, [1], is a function that takes n Boolean variables as input and returns a Boolean value, therefore 0 or 1.

It can be expressed in **algebraic form**, reporting the Boolean variables and the logical operators that link them, such as for example

$$f(A, B) = \bar{A} + \bar{B}$$

Another very useful expression is that by means of a **truth table**, which we introduced in paragraph 1.1.1, whose number of rows is equal to 2^n, this being the number of possible combinations considering the n variables and the two values that each can assume.

Taking up the example just seen, here is how to express this function by means of a truth table:

3.3.1 Canonical forms

Starting from the same truth table it is possible to derive different algebraic expressions. The following functions, for example, all have the same truth table written above:

$$\bar{A \cdot B} \qquad \bar{A}\bar{B} + A\bar{B} + \bar{A}B \qquad \bar{A}B + \bar{B}$$

and are therefore perfectly equivalent alternative expressions to $\bar{A} + \bar{B}$.

One of these, however, is considered a **canonical form** through which to express Boolean functions starting from the truth table. In addition to this, by the principle of duality, there is another one; we will see below that of these forms there is the one that makes use of *minterms* and the one that makes use of *maxterms*.

The (m_i) are given by the product of all the Boolean variables of a function, both in negated and non-negated form. For this reason the input of the function will determine *only one* minterm equal to 1 when all the others will instead be 0; it follows that to each combination of values of the variables corresponds a single non-zero minterm: in the case in which the input is 000, for example, the associated minterm is $\bar{A}\bar{B}\bar{C}$, while if the input is 0101 the minterm will be $\bar{A}B\bar{C}D$, and so on; we immediately understand, therefore, that the rule is to write in order the variable associated with the input 0 as *negated* and the one associated with the input 1 as *non-negated*.

For clarity, we report below the table with three variables and the minterms associated with each input; in the last column is reported the notation associated with the minterms.

The (M_i), on the other hand, are given by the sum of all the Boolean variables, also both in negated and non-negated form. The input of the function, this time, will determine a single maxterm equal to 0 when all the others will be equal to 1. It follows that to each combination of values of the Boolean variables corresponds a single *null* maxterm, in which - it is easy to guess - each variable will be the complement with respect to the minterm associated with the same input.

Starting from minterms and maxterms it is possible to express a Boolean function in the following ways:

- **Logical sum of minterms**

[1] In the mnemonics to refer to the extended register, only the first letter of the register is used: we will therefore have *B* for *BC*, *D* for *DE* and *H* for *HL*.

PhyAMR

- **Logical product of maxterms**

In particular, the procedure is as follows: one observes in the truth table which inputs give rise to an output 1 and the minterms associated with these inputs are added together. To a given input, in fact, one and only one minterm will be equal to 1, and if this is present among the minterms of the sum, an output 1 will also result; vice versa, if this minterm is not present in the sum, then this will give as a result 0, which is what we expect from the truth table.

Let's consider, for example, the function $f(A,B) = \bar{A} + \bar{B}$ seen at the beginning of the section; from the truth table we understand that the output equal to 1 is associated with three different inputs, so it will be sufficient to put their three minterms in sum to obtain the canonical form of the function:

$$f(A,B) = \bar{A}\bar{B} + A\bar{B} + \bar{A}B$$

The canonical form with the maxterms is, in a certain sense, the of the one just seen: the inputs that produce 0 as output are considered and the relative maxterms are multiplied through logical product; since, given an input, there will be only one maxterm equal to 0, if this is present in the product then it will cancel the entire function, as provided by the truth table, while if it is not present the product will necessarily be 1. Consequently, the function $f(A,B)$ has already been written in canonical form through the product of maxterms, since the only maxterm is precisely $\bar{A} + \bar{B}$.

3.3.2 Functions with two variables

From the reasoning just made we understand that the number of Boolean functions is more limited than one might initially think, since the real of a function is its truth table and these are limited by a certain number of combinations.

Let's study, for example, the functions that contemplate only two Boolean variables, A and B. We know that for them the number of possible inputs, that is of rows in the truth table, is $2^2 = 4$; the question now is: how many combinations in input can we get?

To answer, let's imagine placing the outputs f_i (that is, the rows of the truth table) in order as columns of a new table; it is clear that the possible combinations will be a total of $2^4 = 16$, and that therefore, more generally, the number of possible functions given n variables is equal to 2^{2^n}.

We report below this table with some names associated with the most important functions, of which some new ones in bold and in red that we will study in more detail later.

PhyAMR

PhyAMR

From algebra to circuits

When Boolean algebra was initially introduced - around the mid-nineteenth century - it was seen as a simple connection between mathematics and logic, but almost a century later - in the 1930s - it was realized that this algebra could be applied to the design and implementation of electrical circuits: in particular, it was Claude Shannon (the father of information theory) who understood that the *open* and *closed* states of a switch could be made to correspond to the *true* and *false* of Boolean variables, and in this way an electrical signal passing through these switches would follow exactly what was predicted by Boolean algebra.

4.1 Logic gates

If Boolean variables are represented by electrical signals, the operators through which to compose these variables are represented by circuits, also called **logic gates**. Let's see below how the three main operations are converted into circuits:

- **NOT** (*negation*)
  The input is connected through a resistor to a *bipolar junction transistor*. When there is no current (input 0) the transistor is off and no current flows from the collector, so at the output (V_{out}) there is a voltage V_{CC} due to which current flows; when there is current (input 1) the transistor is on and this time the current flows through the collector, bringing the voltage to zero and consequently interrupting the electrical signal from V_{out}.
  By placing two *NOT* gates in sequence, a *buffer* is obtained, which returns the same input at the input; it is used to delay an electrical signal.

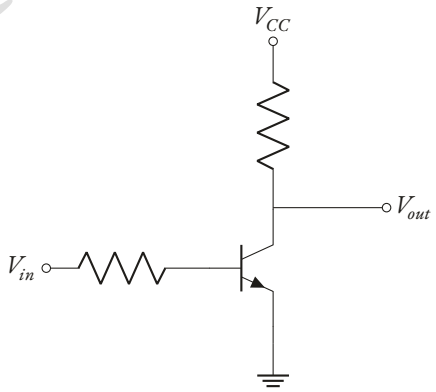


FIG. 4.1 Porta logica *NOT*

- **OR** (*logical sum*)
  If at least one of the two inputs is *HIGH* the relative diode will be in conduction allowing the voltage signal to reach the output; if both inputs are *LOW*, instead, the two diodes are interrupted and no current will reach the output, since the circuit is connected to ground.

PhyAMR

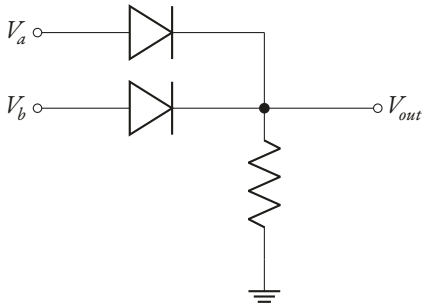


FIG. 4.2 Porta logica *OR*

- **AND** (*logical product*)
  If at least one of the two inputs is *LOW* the relative diode will be in conduction preventing the voltage signal from reaching the output; if both inputs are *HIGH*, instead, the two diodes are interrupted, diverting the current from the V_{CC} to the output.

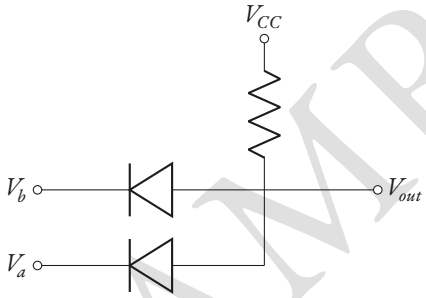


FIG. 4.3 Porta logica *AND*

Below are the symbols of the logic gates used in the circuits:

```
```{=latex}
\input{Grafiche/Figure/dellalgebra-ai-circuiti-simboli-porte-logiche.tex}
```
```

4.1.1 Functional completeness NOR and NAND

In circuits, usually, the ***NOR*** and ***NAND*** gates are preferred to the logic gates just introduced:

```
```{=latex}
\input{Grafiche/Figure/dellalgebra-ai-circuiti-simboli-porte-logiche-nand-nor.tex}
```
```

The reason is that the *OR* and *AND* gates are made up of passive elements that, in the long run, degrade the signal; the inversion of the signal, instead, requires the implementation of amplifiers (like the transistor seen in the *NOT* gate) that preserve its integrity.

It is also possible to demonstrate that the *NOR* and *NAND* functions, individually, can completely replace the use of the *OR* and *AND* operators: this property is called **functional completeness** and is shown in figure [fig:dallalgebra-ai-circuiti-costruzioni-nand-nor].

PhyAMR

The *NAND* gates are used to write sums of products, while the *NOR* gates to write products of sums, and from the principle of duality we know that, as long as the inputs are complementary to each other, the two quantities will be the dual of each other:

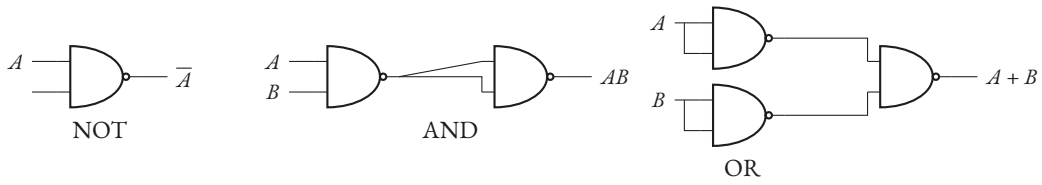


FIG. 4.4 NAND functional completeness: NOT, AND and OR built from NAND gates only

4.2 XOR operator

The last operator that we have left to study is the **XOR**, also called *Exclusive OR* and represented by the symbol $\oplus$, whose truth table has already been shown in table [tab:fondamentali-di-algebra-booleana-funzioni-due-variabili]:

The name comes from the fact that the input is 1 only if the two inputs are different from each other, and in this sense it is an . In other words we could say that this operator returns 1 only if one of the two inputs is 1, or if only one of the two outputs is 0; since these two definitions are equivalent, we immediately guess the following property:

$$A \oplus B = \bar{A} \oplus \bar{B}$$

In canonical form (using minterms) the result is the following:

$$A \oplus B = A\bar{B} + \bar{A}B$$

from which the *commutativity* of the XOR is evident.

The negated form will therefore be:

$$A \bar{\oplus} B = A\bar{B} \bar{+} \bar{A}B = \bar{A\bar{B}} \cdot \bar{\bar{A}B} = (\bar{A} + B)(A + \bar{B}) = \bar{A}\bar{B} + AB$$

being $A\bar{A} = B\bar{B} = 0$.

Connecting back to [eq:dallalgebra-ai-circuiti-xor-operatore] we can write:

$$A \bar{\oplus} B = \bar{A} \oplus B = A \oplus \bar{B}$$

Let's also verify that the XOR can also be used to replace the NOT, OR and AND operations:

- From [eq:dallalgebra-ai-circuiti-xor-operatore] it is evident that if we want to go from A to $\bar{A}$ it is sufficient to set $B = 1$, in fact:

$$A \oplus 1 = A \cdot 0 + \bar{A} \cdot 1 = \bar{A}$$

- The [eq:dallalgebra-ai-circuiti-xor-negazione] is also useful for the OR because of De Morgan's law:

PhyAMR

$$A + B = \bar{\bar{A}}\bar{\bar{B}} = \bar{A}\bar{B} \oplus 1 = [(A \oplus 1)\,(B \oplus 1)] \oplus 1$$

- Similarly, always for De Morgan's law, we can also derive the expression for the AND:

$$AB = \bar{\bar{A}} + \bar{\bar{B}} = (\bar{A} + \bar{B}) \oplus 1 = [(A \oplus 1) + (B \oplus 1)] \oplus 1$$

Finally, the symbol of the logic gate associated with the XOR is the following:



FIG. 4.5 Symbol of the XOR gate

EXAMPLE 4.1 | § 4.2

Prove that

$$(A \oplus B) \oplus C = A \oplus (B \oplus C) = B \oplus (A \oplus C)$$

Let's start from the first member and make the second XOR explicit:

$$(A \oplus B) \oplus C = (A \oplus B)\bar{C} + (A \,\bar{\oplus}\, B)C$$

At this point let's make the XOR and the negated XOR explicit:

$$(A \oplus B) \oplus C = (A\bar{B} + \bar{A}B)\bar{C} + (\bar{A}\bar{B} + AB)C$$

Let's now multiply the various terms:

$$(A \oplus B) \oplus C = A\bar{B}\bar{C} + \bar{A}B\bar{C} + \bar{A}\bar{B}C + ABC$$

Let's collect the terms in A and in $\bar{A}$ and proceed backwards in the calculations:

$$\begin{aligned}(A \oplus B) \oplus C &= A(\bar{B}\bar{C} + BC) + \bar{A}(B\bar{C} + \bar{B}C) = \\ &= A(B \,\bar{\oplus}\, C) + \bar{A}(B \oplus C) = A \oplus (B \oplus C)\end{aligned}$$

We have thus proved the first equality; to prove also the second one let's go back to equation [eq:dallalgebra-ai-circuiti-esercizio-xor] and perform the same calculations but collecting B and $\bar{B}$:

$$\begin{aligned}(A \oplus B) \oplus C &= B(\bar{A}\bar{C} + AC) + \bar{B}(A\bar{C} + \bar{A}C) = \\ &= B(A \,\bar{\oplus}\, C) + \bar{B}(A \oplus C) = B \oplus (A \oplus C)\end{aligned}$$

4.3 Graphical simplification

To build efficient circuits it is necessary to simplify as much as possible the algebraic expressions of the Boolean functions that these circuits express.

The most efficient way to simplify functions is given by the use of **Karnaugh maps**, which express a function in its canonical form through a grid. Let's start from functions with two variables, whose Karnaugh map is as follows:

	A	$\bar{A}$
B	AB	$\bar{A}B$
$\bar{B}$	$A\bar{B}$	$\bar{A}\bar{B}$

FIG. 4.6 Karnaugh map with two variables

The various cells represent the minterms: the first column, in fact, is associated with the variable A, while the second with its complement $\bar{A}$, and the same will apply to the rows with B and $\bar{B}$.

Adding a variable means doubling the number of cells, as shown by the three-variable and four-variable maps:

3-variable K-map

$AB\backslash C$	0	1
00		
01		
11		
10		

4-variable K-map

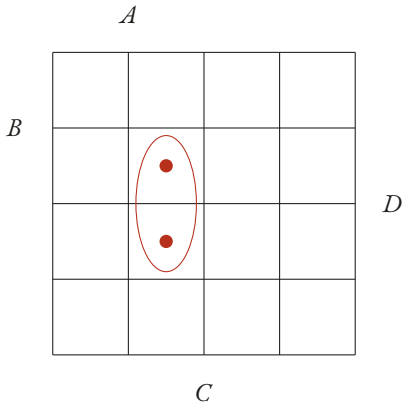
$AB\backslash CD$	00	01	11	10
00				
01				
11				
10				

FIG. 4.7 Karnaugh maps for three and four variables (rows / columns in Gray-code order)

In each map the minterms that make up the canonical expression of the function are marked. The simplification will take place by grouping some marked cells according to certain rules:

- **Pairs**
  Two adjacent cells, horizontally or vertically, imply the presence of two terms that differ only by a negated variable: this variable can be simplified.
  Let's look at the following example:

PiyAMR



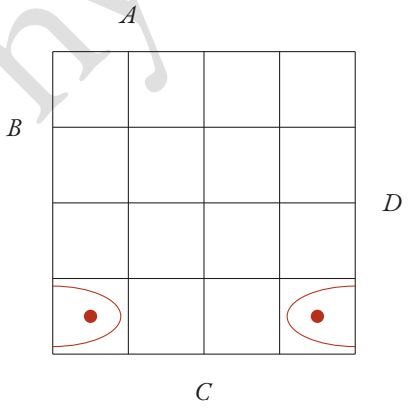
The function will be

$$f(A,B,C,D) = ABCD + A\bar{B}CD$$

The only variable that changes is B; in fact, by factoring out the rest, we get:

$$f(A,B,C,D) = ACD(B + \bar{B}) = ACD$$

An important thing to underline is that the map is connected at the edges with itself, in the sense that the left column is continuous with the right column and the top row with the bottom one.
We can therefore also simplify a configuration like the following:

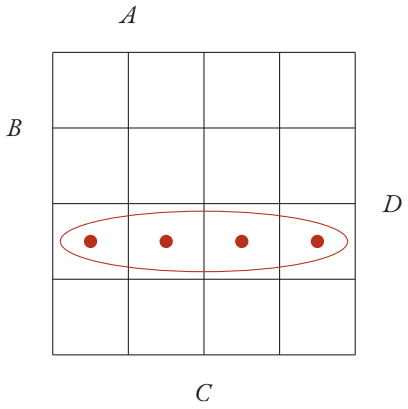


In fact:

$$f(A,B,C,D) = A\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}\bar{C}\bar{D} = \bar{B}\bar{C}\bar{D}(A + \bar{A}) = \bar{B}\bar{C}\bar{D}$$

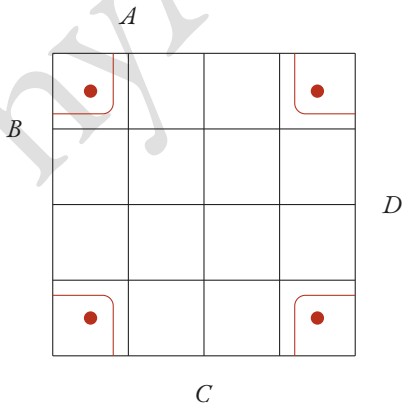
- **Quads**
  Four consecutive cells filling a row or a column, or placed at the vertices of a square, imply the variation of two variables that can be simplified.
  First example:

PhyAMR



$$f(A,B,C,D) = A\bar{B}\bar{C}D + A\bar{B}CD + \bar{A}\bar{B}CD + \bar{A}\bar{B}\bar{C}D =$$
$$= A\bar{B}D(C+\bar{C}) + \bar{A}\bar{B}D(C+\bar{C}) =$$
$$= A\bar{B}D + \bar{A}\bar{B}D = \bar{B}D(A+\bar{A}) = \bar{B}D$$

Second example (remembering the connection between the opposite sides of the map):

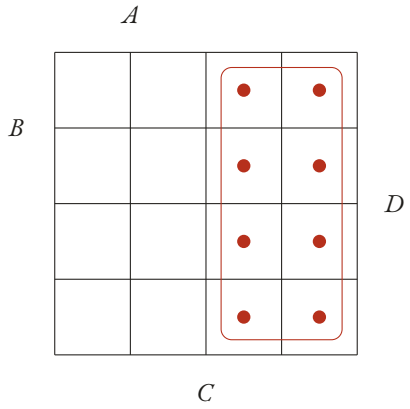


$$f(A,B,C,D) = AB\bar{C}\bar{D} + \bar{A}B\bar{C}\bar{D} + \bar{A}\bar{B}\bar{C}\bar{D} + A\bar{B}\bar{C}\bar{D} =$$
$$= B\bar{C}\bar{D}(A+\bar{A}) + \bar{B}\bar{C}\bar{D}(A+\bar{A}) =$$
$$= B\bar{C}\bar{D} + \bar{B}\bar{C}\bar{D} = \bar{C}\bar{D}(B+\bar{B}) = \bar{C}\bar{D}$$

- **Octets**

  Eight consecutive cells filling two adjacent rows or two adjacent columns imply the variation of three variables that can be simplified, leaving a single variable to describe these terms.

PhyAMR



$$
\begin{aligned}
F(A,B,C,D) = &\bar{A}BC\bar{D} + \bar{A}B\bar{C}\bar{D} + \bar{A}BCD + \bar{A}B\bar{C}D \\
&+ \bar{A}\bar{B}CD + \bar{A}\bar{B}\bar{C}D + \bar{A}\bar{B}C\bar{D} + \bar{A}\bar{B}\bar{C}\bar{D} \\
= &\bar{A}B\bar{D} + \bar{A}BD + \bar{A}\bar{B}D + \bar{A}\bar{B}\bar{D} = \\
= &\bar{A}B + \bar{A}\bar{B} = \bar{A}
\end{aligned}
$$

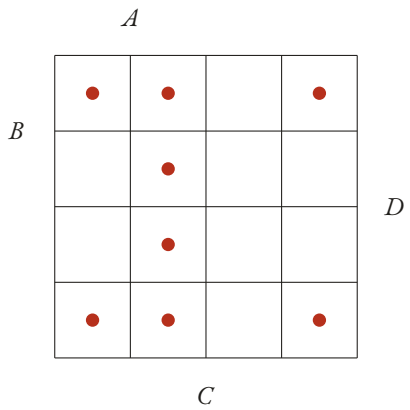
EXAMPLE 4.2 | § 4.3

Simplify, through the Karnaugh map, the following function written in canonical form:

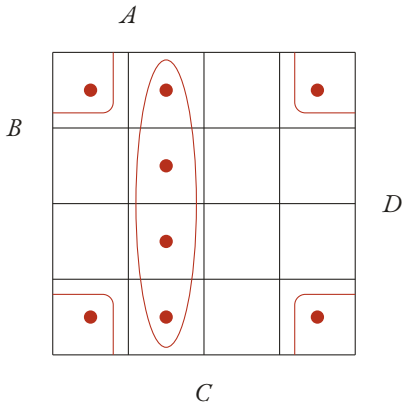
$$
\begin{aligned}
f(A,B,C,D) = &\bar{A}\bar{B}\bar{C}\bar{D} + A\bar{B}\bar{C}\bar{D} + \bar{A}B\bar{C}\bar{D} + AB\bar{C}\bar{D} \\
&+ A\bar{B}C\bar{D} + ABC\bar{D} + A\bar{B}CD + ABCD
\end{aligned}
$$

Also, report the solution in both NAND and NOR form.

Let's report the Karnaugh map:



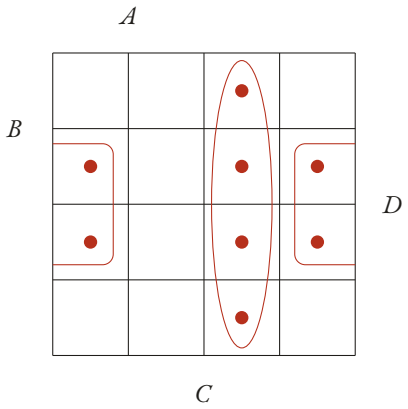
Let's identify two quads:



In the quad along the column the only variables that do not change are A and C, while as for the one at the corners the only ones are $\bar{C}$ and $\bar{D}$. Simplifying according to the rules, therefore, it will result:

$$f(A,B,C,D) = AC + \bar{C}\bar{D}$$

This is also the *NAND* form, being written as a sum of products of variables. The quickest way to get the *NOR* form is to calculate the negated form of the function from the Karnaugh map, which will be the of the one previously reported:



From the simplification rules we get

$$\bar{f}(A,B,C,D) = \bar{A}C + \bar{C}D$$

from which, according to Morgan's law:

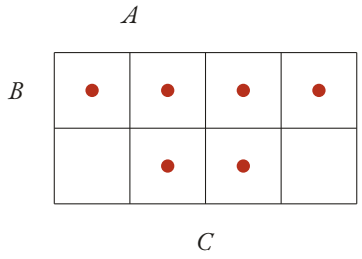
$$f(A,B,C,D) = \bar{A}C + \bar{C}D = (A + \bar{C})(C + \bar{D})$$

EXAMPLE 4.3 | § 4.3

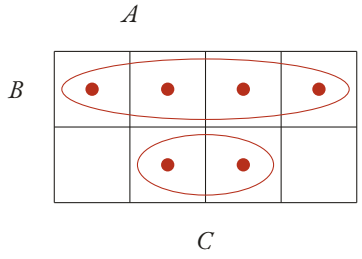
Simplify the following function:

$$f(A,B,C) = AB\bar{C} + ABC + \bar{A}BC + \bar{A}B\bar{C} + A\bar{B}C + \bar{A}\bar{B}C$$

Let's draw the Karnaugh map:



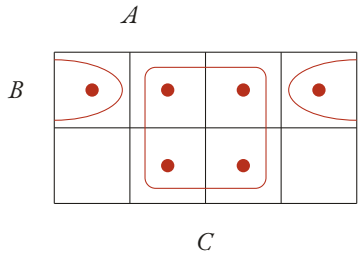
Let's identify a quad and a pair:



Along the first row the only variable that does not change is B, while in the pair $\bar{B}$ and C do not vary, therefore:

$$f(A,B,C) = B + \bar{B}C$$

We can also note that we could have performed the pairings differently:



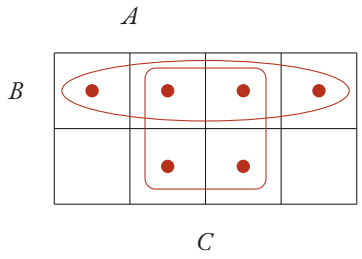
From this we get:

$$f(A,B,C) = C + B\bar{C}$$

The two quantities are equal, in fact:

$$B + \bar{B}C = B(C + \bar{C}) + \bar{B}C = BC + B\bar{C} + \bar{B}C =$$
$$= C(B + \bar{B}) + B\bar{C} = C + B\bar{C}$$

A further possibility was to enclose everything in two quads:



Considering the same points in different groupings is allowed, since algebraically it means repeating the term to be added several times and for the property this does not change the Boolean function.

So we find:

$$f(A,B,C) = B + C$$

which is analogous to the two previous expressions, in fact:

$$B + \bar{B}C = (B + \bar{B})(B + C) = 1 \cdot (B + C) = B + C$$

PhyAM

PhyAMR

Combinatorial circuits

We will see below the main circuits called *combinatorial*, in the sense that their operation concerns only the relationship between the input and output of the signal from the circuits themselves described by logic functions.

5.1 Adder

5.1.1 Half-adder

Our goal is to create a circuit capable of adding two numbers together. Since logic circuits work through *HIGH* and *LOW* signals, the first thing we can do is try to add two bits together, or - in other words - two single-digit binary numbers.

If both bits are 0 ($A = B = 0$) the result will obviously be zero, while if one is 0 and the other is 1 ($A \oplus B = 1$) the result will be 1; the last case is $A = B = 1$, where the result is 10: it therefore requires two bits in output, for which we will have to use in general two bits (that is, two variables containing the information) to perform this simple sum considering all cases.

The truth table will be as follows:

The variable S contains what we indicate as the *sum*; the variable C, instead, is called , and indicates what is the *carry* of an addition in column: in the first three cases the variable S coincides with the actual sum of A and B, while in the last case the sum gives rise to a carry that is stored in the variable C.

From table [tab:circuiti-combinatori-half-adder] we immediately understand which operations are needed to obtain the two output variables:

$$S = A \oplus B$$

$$C = AB$$

Converting this into a circuit we get:

This circuit alone does not allow us to add numbers with more than one bit; let's try, for example, to perform the addition between 111 and 101:

The sum - as we know[1] - proceeds from right to left. We will first add 1 with 1 which we know gives 10, that is a 0 with a carry of 1; now the problem is: how do we proceed with the addition? The next sum, in fact, would be 1 + 0, but we cannot ignore the carry, since otherwise we would erroneously write 1 as the result according to the circuit just described. The solution, therefore, is to introduce a new circuit that takes into account not two bits, but *three* bits, that is the two bits to be added and the carry. For this reason we call the circuit just described , not being yet the version of what interests us.

[1] In the mnemonics to refer to the extended register, only the first letter of the register is used: we will therefore have *B* for *BC*, *D* for *DE* and *H* for *HL*.

PhyAMR

5.1.2 Full-adder

Let's go back to the example just made: in that case the solution is obviously 1 which, with the carry equal to 1 to be added, becomes 10; since the sum continues further, we take 0 as the result and the new carry will have a value of 1.

With the same logic we can build a new truth table, where we indicate with C_i the carry contemplated in the sum between A and B and with C_{i+1} the carry of the next sum:

Let's calculate S using minterms:

$$\begin{aligned} S &= A\bar{B}\bar{C_i} + \bar{A}B\bar{C_i} + \bar{A}\bar{B}C_i + ABC_i = \\ &= \left(A\bar{B} + \bar{A}B\right)\bar{C_i} + \left(\bar{A}\bar{B} + AB\right)C_i = (A \oplus B)\bar{C_i} + (A \bar{\oplus} B)C_i = \\ &= (A \oplus B) \oplus C \end{aligned}$$

Similarly for C_{i+1}:

$$C_{i+1} = AB\bar{C_i} + A\bar{B}C_i + \bar{A}BC_i + ABC_i$$

For the property we can add ABC_i twice:

$$\begin{aligned} C_{i+1} &= AB\bar{C_i} + A\bar{B}C_i + \bar{A}BC_i + ABC_i + ABC_i + ABC_i = \\ &= AB\left(\bar{C_i} + C_i\right) + AC_i\left(\bar{B} + B\right) + BC_i\left(\bar{A} + A\right) = \\ &= AB + AC_i + BC_i \end{aligned}$$

Let's convert everything into a circuit:

This is the useful circuit for our purposes, that is for the sum of numbers with more than one bit: for this reason it is called .

The sum between the numbers 111 and 101 will be performed as follows:

and in fact 111 + 101 = 1100.

The general scheme of the adder, therefore, is the following:[2]

5.2 Subtractor

To obtain the difference between two binary numbers we follow the same reasoning as for the adder. Let's start by building the truth table for a subtraction between two bits: when both are equal to 0 or 1 the result will be 0, while when the minuend is 1 and the subtrahend is 0 the result will be 1; the remaining case is the minuend equal to 0 and the subtrahend equal to 1, or in other words the subtraction 0 − 1: in the context of subtraction with more bits we need the result to be 1 while at the same time recording a *borrow* (also equal to 1) that is subtracted from the next minuend along with its subtrahend.

Indicating the borrow with R and the difference with D we will have:

From which it follows:

[2] Not necessarily located in the last memory addresses.

PhyAMR

$$D = A \oplus B$$

$$R = \bar{A}B$$

Let's now try to build the full-subtractor table, remembering that R_i is simply another bit to be subtracted from A in addition to B:

We note that the variable D assumes the same values as S in the full-adder table (table [tab:circuiti-combinatori-full-adder-tabella]), so its algebraic expression will also be the same:

$$D = (A \oplus B) \oplus R_i$$

Finally, let's calculate R_{i+1}:

$$R_{i+1} = \bar{A}B\bar{R_i} + \bar{A}\bar{B}R_i + \bar{A}BR_i + ABR_i$$

Let's add $\bar{A}BR_i$ twice:

$$\begin{aligned} R_{i+1} &= \bar{A}B\bar{R_i} + \bar{A}\bar{B}R_i + \bar{A}BR_i + ABR_i + \bar{A}BR_i + \bar{A}BR_i = \\ &= \bar{A}B\left(R_i + \bar{R_i}\right) + \bar{A}R_i\left(B + \bar{B}\right) + BR_i\left(A + \bar{A}\right) = \\ &= \bar{A}B + \bar{A}R_i + BR_i \end{aligned}$$

By putting a half-subtractor in a row and then the various full-subtractors it is possible to obtain a real multi-bit subtractor, exactly as in the case of the adder.

5.2.1 Two's complement

Building two different circuits for sum and difference can be expensive and inefficient: is there a way to build a circuit capable of performing both operations?

A first idea is to take the full-subtractor circuit seen in figure [fig:circuiti-combinatori-full-subtractor-circuito] and replace the NAND that inverts the signal of A with a XOR, which takes as input the signal of A and another signal that we indicate with S/D: if we want to perform the sum we set S/D *LOW*, so that

$$A \oplus 0 = A$$

and therefore we get back figure [fig:circuiti-combinatori-full-adder-circuito]; if our goal is the difference then we set S/D *HIGH* obtaining

$$A \oplus 1 = \bar{A}$$

and therefore the full-subtractor circuit.

Another idea is to use only full-adders using the , which is a representation of binary numbers useful for defining positive and negative numbers. The two's complement of an n-bit number is obtained by inverting all its bits (so the 0s into 1s and the 1s into 0s) and adding 1 to the total; for example, the 2's complement of $A = 1011$ will be $A' = 0100 + 1 = 0101$. The reason why this representation is useful is that by performing the two's complement of the same number twice we get the starting number, in fact:

PhyAMR

$$A'' = 1010 + 1 = 1011 = A$$

If performing an operation on a number twice returns the starting number then we can understand this operation as a change of sign, since by inverting the sign of a number twice you get the starting one.

Let's now imagine working with 4-bit numbers; starting from the number 0 (0000) we add 1 obtaining 0001 whose two's complement is 1111, which will correspond to -1, and we continue like this until we get to 0111, which corresponds to 7, and to its two's complement which is 1001: it is evident, therefore, that the first bit will represent the sign of the number, which is 0 for positive numbers and 1 for negative numbers; according to this reasoning, the last missing digit, that is 1000, is negative and represents, therefore, -8.

In general, an n-digit binary number can represent the numbers between -2^{n-1} and $2^{n-1} - 1$.

According to this representation we can think of the difference of two numbers as the sum of a positive number and a negative number. This means that we can use only adders in the following way:

When you want to use the adder you set $SUB = 0$ so that the bits of B are not inverted and the carry of the first full-adder is set to 0, making it in all respects a half-adder. When you want to switch to a subtractor you must set $SUB = 1$ so that each bit of B is complemented and the carry on the first full-adder is set to 1 by default: in this way we will go from B to $\bar{B} + 1 = B'$ and we will get the difference between the numbers in binary.

Keep in mind that for a circuit with four full-adders, which therefore considers four-bit numbers, the digits that represent the number in modulus are only the last three, since the first is relative only to the sign: this means that it is possible to operate only with numbers ranging from −8 to 7.

Furthermore, depending on the sum (or difference) performed, the last carry can:[3]

- Have no meaning if the result x is a number expressible within three bits ($|x| \leqslant 7$)
- Indicate the sign of the resulting number x if it is a number expressible with four bits ($7 \leqslant |x| \leqslant 15$)
- Indicate the first digit of the binary number in modulus if the numbers added or subtracted are both 1000, that is −8 (in both cases, in fact, the result will give 10000, that is 16 in binary)

Let's see an example of the difference between 3 and 7: we know that the number 3 in binary with four bits is 0011, while 7 is 0111, so we will have

The number 1100, in two's complement notation, represents exactly −4.

5.2.2 Parallel carry

One last consideration to make concerns the efficiency of the circuit: in an adder circuit, in fact, each full-adder simultaneously receives as input the two bits of the two numbers to be added, and all those after the first (we remember that according to the various schemes shown the signal proceeds from right to left) must wait for the generation of the carry from the previous full-adders. This means that the time taken by the circuit to perform the sum depends on the number of full-adders: the greater this is, the greater the time, also called **propagation delay**.

In particular, the relationship between the propagation delay T_D and the number of full-adders n is shown to be *directly proportional*, from which it follows that[4]

[3] We remember that a memory address is encoded by 2 bytes, and in it only one byte can be contained; in the SAP-2, in fact, two addresses were used to save the current point.

[4] The structure of this register depends on the specific model of SAP-3. In the 8085 microprocessor - one of the most widespread - it is constituted as follows: *SZ*00 0*P*0*CY*. Beyond the specific position of the flags, the other four bits are always set to zero by default.

PhyAMR

$$T_D = 2T_P \cdot n = 2nT_P$$

A certainly more efficient and useful way when performing a sum with many bits is to generate and propagate the carry *in parallel*.

Let's refer to table [tab:circuiti-combinatori-full-adder-tabella] and write the algebraic expression for which we have in output $C_{i+1} = 1$:

$$\begin{aligned} C_{i+1} &= AB\bar{C}_i + A\bar{B}C_i + \bar{A}BC_i + ABC_i = \\ &= AB\left(C_i + \bar{C}_i\right) + \left(A\bar{B} + \bar{A}B\right)C_i = AB + (A \oplus B)C_i \end{aligned}$$

Following this scheme, each full-adder will not produce a carry C_{i+1} but will output the signals AB and $A \oplus B$, which will arrive on a circuit associated with the algebraic expression written above; we remember that the initial carry C_0 is equal to 0 if you intend to perform a sum and 1 if you intend to perform a difference.

In this case the propagation delay is shown to be[5]

$$T_D = 3T_P$$

and therefore independent of n.

5.3 Parity generator

The is a circuit used to verify the correct information of the transmitted data. It adds a final bit P to the bit string that represents the transferred information based on the number of 1s present in that string, and is divided into two types:

- **Even parity generator**: finds $P = 0$ if the number of 1s is already even and $P = 1$ if the number is odd and must be made even
- **Odd parity generator**: finds $P = 0$ if the number of 1s is already odd and $P = 1$ if the number is even and must be made odd

In both cases, the construction proceeds by analyzing all the bits of the considered string individually; at each i-th iteration, we will reason as follows:

- If $P_i = 0$ and the input bit is 0 then the parity is not changed and the added bit remains 0
- If $P_i = 0$ and the input bit is 1 then the parity is changed, therefore - to - the added bit becomes 1
- If $P_i = 1$ and the input bit is 0 then the parity is not changed and the added bit remains 1
- If $P_i = 1$ and the input bit is 1 then the parity is changed, so the added bit must change accordingly becoming 0

Indicating the input bit with D_i we can put everything in the truth table:

[5] I believe that the presence of the factor 2 in [eq:circuiti-combinatori-propagation-delay-carry-non-parallelo] is due to that of the factor 3 of [eq:circuiti-combinatori-propagation-delay-carry-parallelo]: if there had not been the 2, in fact, we would have had to put a non-integer number (that is 1.5) in the latter. All this is valid to the extent that I have actually understood this explanation, a fact on which I do not assure anything since I have not found any source that quantitatively treated the propagation delay of adders with parallel carry and not.

PhyAMR

It is evident that the algebraic form will be:

$$P_{i+1} = D_i \oplus P_i$$

The question at this point is: how do we distinguish an even parity generator from an odd parity generator? To understand this, let's consider a single-bit information: if $D_0 = 0$ we must have $P_1 = 0$ in the case of an even parity generator and $P_1 = 1$ in the case of an odd parity generator, and from table [tab:circuiti-combinatori-parity-generator] it is evident that these two conditions can be obtained, in order, with $P_0 = 0$ and $P_0 = 1$; the same occurs assuming $D_0 = 1$, so in general we can say that for the even parity generator we have $P_0 = 0$ and for the odd parity generator $P_0 = 1$.

Ultimately, the parity generator consists of a series of XORs that take as input a bit of the transmitted information and the output of the previous XOR:

Another possibility is to consider all the input bits in pairs. Let's look at table [tab:circuiti-combinatori-parity-generator] and imagine that there are two information bits in input: the output will be 1 only when one of the two is 0 and the other is 1, or - in other words - when the input 1 is present an odd number of times; the output will instead be 0 when both inputs are 0 or are 1, that is when 1 is present an even number of times.

Therefore, by placing all the input bits in the various XORs we will obtain an even parity generator, since we will always get as output 0 when the number of 1s is even and 1 when it is odd.

In the case where you want to obtain an odd parity generator, it is sufficient to place an inverter at the end of the circuit that returns the opposite output with respect to the even parity generator.

5.4 Comparison

The goal is to understand if a binary number is equal, greater or lesser than another.

Since there are three cases, it is necessary to consider two bits in output, since with a single bit we could distinguish at most two cases. Let's indicate these two bits with M and N and encode the information in the following way:

Considering two n-bit numbers, the comparison will be made starting from the digits with the highest weight, therefore from to : for this reason we will say that the iteration will proceed from the $i + 1$-th to the i-th and so on, since we are gradually moving towards bits with a lower weight.

Before starting the comparison, $M = N = 0$ are initially set. From the first comparison, therefore, the truth table will be the following:

where a_i and b_i represent the i-th bits of the numbers a and b.

When the central cases occur, that is $a_i > b_i$ or $a_i < b_i$, regardless of the subsequent digits it will result respectively $a > b$ or $a < b$, so the values of M and N will simply be whatever the subsequent combinations of the bits of a and b.[6]

The complete truth table, therefore, will be

The last four cases analyzed are those in which $M_{i+1} = N_{i+1} = 1$, which in reality never occur since this combination of bits does not encode any information according to what was established at the beginning. For this reason,

[6] At this point I wonder: but isn't there a system that stops the bit-by-bit comparison when $M_i \oplus N_i = 1$ occurs, since from that point on it is already clear which number is greater than the other?

PlyAVR

the combinations of bits in the last four rows of the table give rise to the so-called ,[7] useful only in the eventual simplification of the algebraic expression of the boolean functions.

Let's draw the Karnaugh maps for M_i and N_i:

Redundant terms can be used to complete groupings or be ignored, since their contribution is irrelevant; in this case they are useful for completing an octet.

The algebraic functions will therefore be:

$$M_i = M_{i+1} + a_i \bar{b}_i \bar{N}_{i+1}$$
$$N_i = N_{i+1} + \bar{a}_i b_i \bar{M}_{i+1}$$

EXAMPLE 5.1 | § 5.4

Create a circuit capable of comparing two numbers two bits at a time.

Indicating the number a with $a_1 a_0$ and b with $b_1 b_0$ we will have the following truth table:

To derive the expressions for M and N we use Karnaugh maps:

From which we find:

$$M = a_1 \bar{b}_1 + \bar{b}_1 a_0 \bar{b}_0 + a_1 a_0 \bar{b}_0$$
$$N = \bar{a}_1 b_1 + \bar{a}_1 \bar{a}_0 b_0 + b_1 \bar{a}_0 b_0$$

5.5 Multiplexer

The (also called ***MUX***) is a device capable of selecting a single signal from several inputs.

It consists of three parts:

1. n selectors (also called *addresses*) S_i which have the task of identifying which of the signals to propagate
2. 2^n input signals D_j
3. A single output Y

It is no coincidence that n selectors are needed for 2^n signals, since n bits are sufficient to encode each of the 2^n inputs through a specific unique sequence. Let's clarify with examples from $n = 1$ to $n = 3$:

$$n = 1 \longrightarrow 2 \text{ inputs: } \{0, 1\}$$
$$n = 2 \longrightarrow 4 \text{ inputs: } \{00, 10, 01, 11\}$$
$$n = 3 \longrightarrow 8 \text{ inputs: } \{000, 100, 010, 110, 001, 101, 011, 111\}$$

As usual, let's start from the simplest case with $n = 1$ and then extend to a circuit with a generic n.

The device is schematized as follows:

Imagining to associate $S = 0$ with the input D_0 and $S = 1$ with the input D_1 we will have the following truth table:

It is evident that the algebraic expression of Y will be

[7] In English *redundant terms* or - if I understood correctly - also *don't care conditions*.

$$Y = D_0\bar{S} + D_1S$$

which is easily achievable in a circuit using three NANDs and an inverter:

At this point it is easy to extend the reasoning for larger n. For $n = 3$, for example, the MUX is schematized as follows:

The truth table useful to describe it is the following:

5.5.1 Demultiplexer

The (also called ***DEMUX***) is in a certain sense the of the multiplexer, having a single input and 2^n outputs. The objective of this device is to a single signal into one of the several available channels, defined through the selectors.

What we must keep in mind is that all outputs will be 0 except for the one selected through the addresses, which will propagate the input value: it is clear, therefore, that when the input is 0 all outputs will be 0 regardless of the value of the selectors.

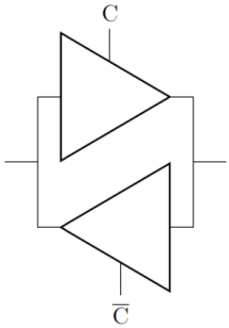
This time too it is useful to study the circuit element with $n = 1$ starting from the truth table, so as to be able to easily guess what the general mechanism is.

The demultiplexer can be considered a form of **decoder**, since the input provided through selectors can be interpreted as a that allows access to a single channel that is activated while all the others are left off.

5.6 Tristate

The - as the name itself says - is a device that has three possible output states: in addition to the usual 0 and 1, in fact, there is also the possibility of **high impedance**, indicated with Z, which causes the connection line to be isolated and does not allow the exchange of any information.

The circuit is in all respects a buffer, with the only difference that there is another input - called **control line** or **enable** - which, when not active, makes the circuit in all respects an *open switch*.



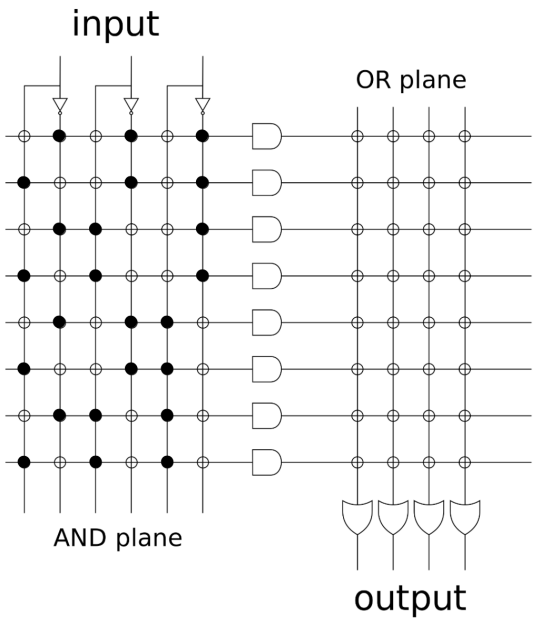
R0.2

Tristates are used in almost all bidirectional connections (called in electronics), present when devices connected to each other can both send and receive information, so to avoid electrical conflicts it is essential to interrupt the connection from one direction when the one from the other is active. Usually two tristates are placed in two different branches for the same connection in the two opposite directions, sending the signal C in one and the negated $\bar{C}$ in the other, so that when one activates the high impedance the other allows conduction.

PhyAMR

5.7 Programmable devices

From boolean algebra we know that every boolean function can be expressed as a sum of products of variables. On this principle towards the end of the 70s the first programmable devices were commercialized, where by we mean that they can be configured in order to associate the specific combination of operations necessary to give rise to the desired function.



L0.45

The first device made for the general public was the , consisting of an input matrix of AND logic gates, called **AND array**, in which each combination of inputs is brought to another region, that of the ORs: in this sense we will say that the AND array is *fixed* while the OR array is *programmable*, since the desired combination will emerge from the latter in output.

In figure [fig:circuiti-combinatori-programmable-logic-array] we find an example with three inputs and, consequently, eight possible combinations of products. There are therefore eight AND gates connected to the OR array from which four different functions can emerge simultaneously based on the connections chosen by the manufacturer for the device.

The main advantage of this device is that it has a *constant* propagation delay, while the disadvantage is that the two dimensions grow excessively as the number of inputs increases. To overcome this problem, Programmable Logic Arrays evolved into , consisting of fundamental blocks called **Configurable Logic Blocks** that perform boolean functions through **Look-Up Tables**, which are structures that associate a specific output to each combination of inputs: these are very useful since - often - performing operations through dedicated circuits can be slower and more laborious than inserting all the possible combinations associated with the desired result. The individual blocks are put in communication with each other through connections that are in turn programmable, called *programmable interconnector*, and the elaborate array that is formed is by input/output blocks that connect the device with the outside, managing the direction of the signal through tristates. The most important novelty for Field-Programmable Gate Arrays, moreover, is that they are programmable by the user and not by the manufacturing company: the term *field*, in fact, indicates that the programming takes place , that is, outside the factory.

PRV-MR

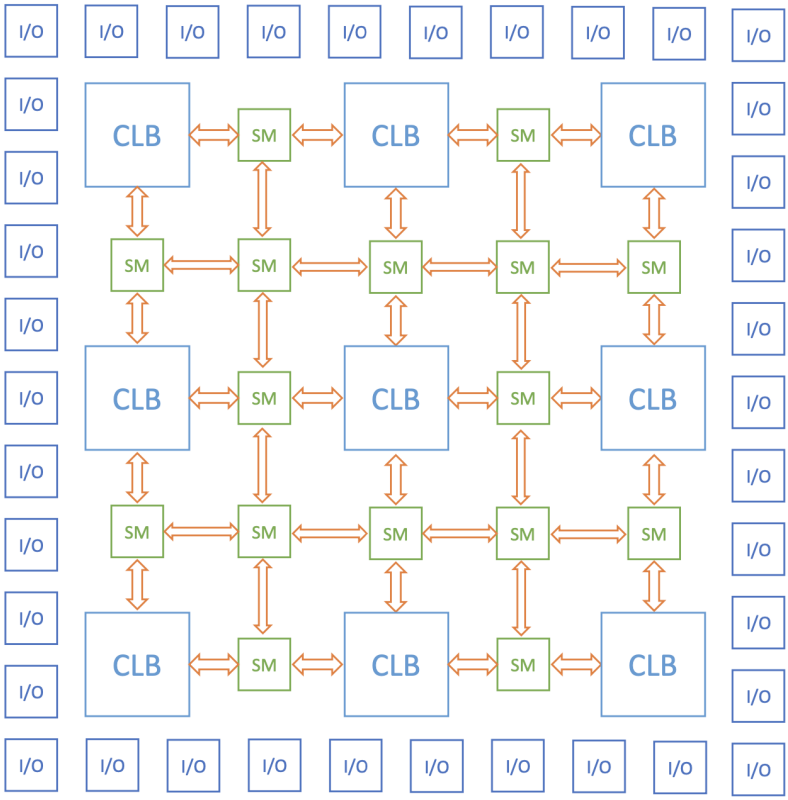


FIG. 5.1 Field-Programmable Gate Array

PhyAM

Sequential circuits

A digital device - such as, in general, a computer - cannot be constituted solely by combinatorial circuits: it needs, in fact, to be constituted also by *memory circuits*, that is, circuits that the previous state so that this influences the subsequent outputs. For this reason, these circuits are called , having to depend on the sequence of states assumed by the various components over time.

6.1 Flip-flop

The elementary circuit for storing a simple bit of information is the **flip-flop**, so called because of the noise caused by the first prototypes. We will start from their fundamental structure, the *latch*, from which all the various types of flip-flops are then built.

PhyAMR

6.1.1 Latch

The is a circuit in which two of the four total inputs derive directly from the outputs in the form of **feedback**. The latches that we will study are constituted by two NAND gates, but different versions exist.

To each pair of inputs (with the exception of one) corresponds a single pair of *stable* outputs. To understand which ones, let's remember the fundamental products of the NAND:

$$NAND(X, 0) = 1 \qquad NAND(X, 1) = \bar{X}$$

Let's look at the various combinations now:

- $A = 0$ and $B = 0$: the outputs must necessarily be equal to 1, and these will be retroactively placed as input in the two $NAND$s, coherently producing the same $HIGH$ signal in both lines.
- $A = 0$ and $B = 1$: whatever Q_2 comes out, the output Q_1 will necessarily be 1; the latter, multiplied by B and negated, will give 0 as a result, so ultimately we have $Q_1 = 1$ and $Q_2 = 0$. Obviously the situation will be symmetrical in the case $A = 1$ and $B = 0$: in general the output will be complementary with respect to the fixed input of the same logic gate.
- $A = 1$ and $B = 1$: the output will always be complementary with respect to the retroactive input that produced it, so as long as Q_1 and Q_2 are complementary to each other, the state of the circuit will remain stable.

All the reasoning is summarized in the following truth table:

A	B	Q_1	Q_2
0	0	1	1
1	0	0	1
0	1	1	0
1	1	Y	$\bar{Y}$

TABLE 6.1 Tabella della verità del latch

From the perspective of information storage, we already notice aspects that can be useful to us: when the inputs are complementary to each other (central rows of the table), the outputs are fixed to values that are in turn complementary to each other; what is more important is that once the inputs are set to $1-1$, these outputs remain constant, as if they were - precisely - in memory.

Input $0-0$ appears to be the most , in the sense that it produces outputs that are equal to each other and are not compatible with the storage discussion just made, since to be compatible with input $1-1$ they must be two complementary values. To better understand this discussion, let's imagine initially setting $A = B = 0$ and simultaneously modifying these inputs to $A = B = 1$: what would happen?

In an *ideal* circuit, the two signals travel at the same speed and the outputs are complemented at the same instant, producing $Q_1 = Q_2 = 0$; these will be retroactively propagated to the two unfixed inputs, causing a new complementation that will make them return to $Q_1 = Q_2 = 1$, repeating the cycle indefinitely and thus obtaining an oscillation of the signal along the lines. In a *real* circuit, at some point one of the two signals will be propagated faster, thus being complemented before the other undergoes the same process, and in this way the system finds itself in a stable state - but which is, however, **unpredictable** - between $1-0$ and $0-1$. The phenomenon just described is called *race competition* and is something that one always wants to avoid precisely because of the randomness of the final result that nullifies the logic of sequentiality.

PhyAMR

6.1.2 S-R Flip-flop

For the realization of a real memory device, therefore, we postulate that the two outputs must necessarily be complementary to each other: consequently, we will indicate them with Q and $\bar{Q}$.

Another element to add is a *synchronizer*, that is, a signal that the device when it is active and leaves it when it is not active. This is useful when it is necessary to make a state change through new inputs, avoiding a possible race competition: having a signal that prevents a new state change until the previous one has been completely memorized makes the use of the device safer.

The addition of the synchronizer t can be obtained by adding two more NAND gates, in each of which one input will be given precisely by t and the other will be the fixed input considered in the latch; the output of these gates will then reconnect to the input of the NAND gates seen previously:

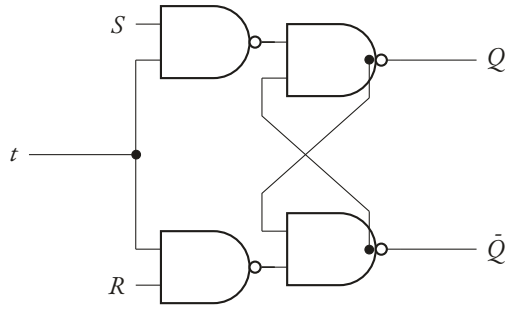


FIG. 6.1 Circuito del flip-flop S-R

We note that when the synchronizer is active ($t = 1$), the input signals are inverted, so the truth table of the device will be equal to table [tab:circuiti-sequenziali-latch] with the difference that the outputs will be inverted between the first and fourth rows and between the second and third.

We have renamed the generic inputs $A - B$ to $S - R$, where these variables stand for **set-reset**: with $S = 1$ and $R = 0$, in fact, $Q = 1$ is , while with $S = 0$ and $R = 1$, $Q = 0$ is . For this reason, this flip-flop is called a .

Looking at the circuit in figure [fig:circuiti-sequenziali-flip-flop-sr], let's calculate what the variable Q^{n+1} corresponds to starting from the previous state; assuming that the device is on ($t = 1$), we will have

$$Q^{n+1} = \left(\bar{S} \cdot \bar{\bar{Q}}\bar{R}\right)^n = \left(S + Q\bar{R}\right)^n$$

The algebraic form also confirms, therefore, that the input $0 - 0$ is the one that preserves the state of Q.

6.1.3 J-K Flip-flop

Having a state is always a risk to keep in mind. To try to eliminate it, it is possible to try to associate a stable output ($1 - 0$ or $0 - 1$) with this state; let's now build a circuit in which the input $1 - 1$ is associated with the inversion of Q with $\bar{Q}$ (we will see later why precisely this output is the most convenient to realize). This circuit is called a in honor of Jack Kilby, the inventor of integrated circuits.

To build the new circuit, it is necessary to write S and R as a function of the two new variables J and K. First, let's see which combinations of $S - R$ correspond to each possible transition from Q^n to Q^{n+1} (which we will indicate with $Q^n \to Q^{n+1}$):

- $0 \to 0$: the memory state is preserved, so one possibility is that $S = R = 0$; another possibility is that the reset $R = 1$ is fixed and, consequently, $S = 0$
- $1 \to 0$: the change of output necessarily implies the reset $R = 1$ and therefore $S = 0$
- $0 \to 1$: the change of output necessarily implies the set $S = 1$ and therefore $R = 0$
- $1 \to 1$: the memory state is preserved, so one possibility is that $S = R = 0$; another possibility is that the set $S = 1$ is fixed and, consequently, $R = 0$

Summarizing:

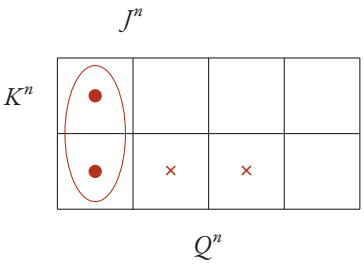
PhyAMR

t	S	R	Q	$\bar{Q}$
0	X	X	Y	$\bar{Y}$
1	0	0	Y	$\bar{Y}$
1	1	0	1	0
1	0	1	0	1
1	1	1		/

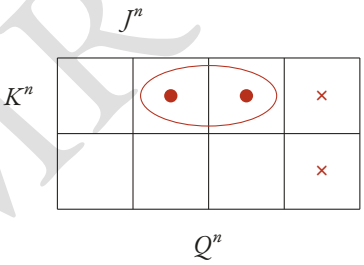
TABLE 6.2 Tabella della verità del flip-flop S-R

Now we can build the table also including the variables $J - K$: we will consider, therefore, the various combinations of J^n, K^n and Q^n; based on the combinations of J^n and K^n we will write the consequent Q^{n+1} (we remember that the behavior must be the same as the set-reset with the only difference that the input $1 - 1$ must invert the state of Q^n); finally, we will report for each combination of Q^n and Q^{n+1} the inputs $S - R$ as reported in the table just built.

The Karnaugh maps give us the following:



Mappa di Karnaugh per S^n



Mappa di Karnaugh per R^n

The algebraic solutions will be

$$S = \bar{Q}J \qquad R = QK$$

from which the circuit is obtained:

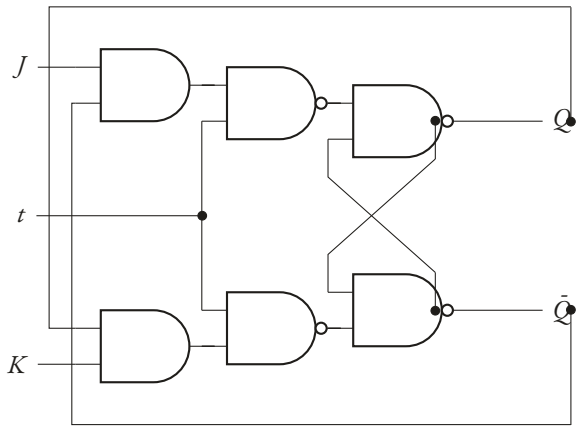


FIG. 6.3 Circuito del flip-flop J-K

Any other choice for the input $J = K = 1$ would have led to a more complex and artificial circuit.

PhyAMR

Returning to equation [eq:circuiti-sequenziali-flip-flop-sr-espressione-algebrica] and substituting the values of S and R as a function of J and K, we also obtain the algebraic expression for the J-K flip-flop:

$$Q^{n+1} = (S + Q\bar{R})^n = \left(\bar{Q}J + Q\bar{Q}\bar{K}\right)^n = \left(\bar{Q}J + Q\left(\bar{Q} + \bar{K}\right)\right)^n$$

From which we will have

$$\boxed{Q^{n+1} = \left(\bar{Q}J + Q\bar{K}\right)^n}$$

6.1.4 D Flip-flop

It is possible to build a flip-flop using only one input D: this device is called a .

To realize it, let's retrace what we did to design the J-K flip-flop: let's consider all possible combinations of Q^n and Q^{n+1} and write the relative conditions of the input variables so that this state transition occurs.

We postulate that $D = Q^{n+1}$ regardless of the value assumed by Q^n. The combinations of D and Q^n will give us:

D^n	Q^n	Q^{n+1}	J^n	K^n
0	0	0	0	X
1	0	1	1	X
0	1	0	X	1
1	1	1	X	0

TABLE 6.3 Tabella della verità del flip-flop D

From which we trivially find:

$$J = D \qquad K = \bar{D}$$

In circuit form:

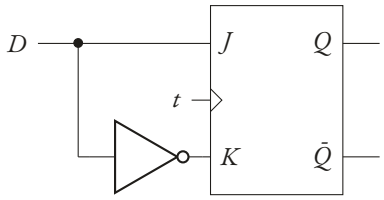


FIG. 6.4 Circuito del flip-flop D attraverso lo schema di un flip-flop J-K

In schematic form, the flip-flop is represented by a rectangle, with inputs on the left side and outputs on the right side. We can therefore schematize the D flip-flop as follows:

The variable D is therefore useful for establishing the output $1-0$ or $0-1$; from the moment one simply wishes to preserve the state as it is, it is sufficient to the device by setting $t = 0$.

PhyAMR

EXAMPLE 6.1 | § 6.1

Realize a J-K flip-flop starting from a D flip-flop.

What we need to do is in a certain sense the of what was seen in table [tab:circuiti-sequenziali-flip-flop-d]: we will have to write the various combinations of J^n, K^n and Q^n, the next state Q^{n+1} associated with each combination and finally the variable D^n; at that point we will be able to derive D^n as a function of J^n and K^n, which is precisely our objective.

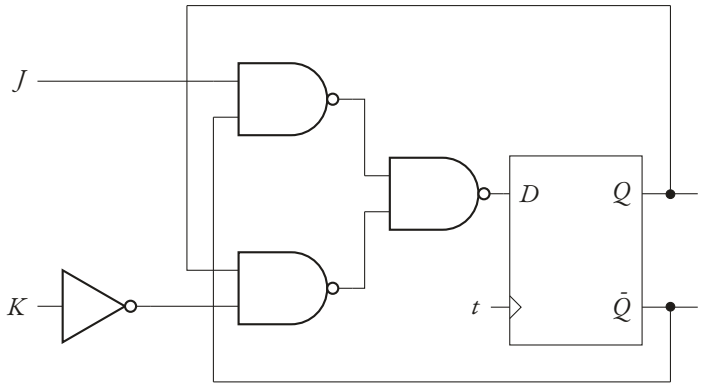
The Karnaugh map gives us:

which translates into

$$D = \bar{Q}J + Q\bar{K}$$

The reasoning followed is the most general one applicable to any problem of this type. For this specific case, however, it would have been enough to consider equation [eq:circuiti-sequenziali-flip-flop-jk] and remember that $D^n = Q^{n+1}$.

In any case, the circuit will be the following:



EXAMPLE 6.2 | § 6.1

Find the truth table of the flip-flop described by the following equation:

$$Q^{n+1} = [Q \oplus (M \oplus N)]^n$$

Also, realize this flip-flop starting from a J-K flip-flop.

Truth table:

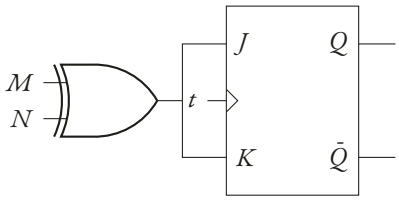
To build the flip-flop from the J-K, it is useful to look at equation [eq:circuiti-sequenziali-flip-flop-jk] comparing it with the one given by the text, which is explicit as

$$Q^{n+1} = \left[Q\left(M \bar{\oplus} N\right) + \bar{Q}(M \oplus N)\right]^n$$

from which we find:

$$\begin{cases} J = M \oplus N \\ \bar{K} = M \bar{\oplus} N \end{cases} \Longrightarrow \begin{cases} J = M \oplus N \\ K = M \oplus N \end{cases}$$

The circuit, therefore, will be



6.2 Counters

Flip-flops are the unit for realizing sequential circuits. The simplest to imagine are **counters**, that is, circuits with the task of reporting numbers in sequence; to do this, it is evident that each state of the system must be influenced by the previous state, so it is certainly a sequential type circuit.

6.2.1 Decimal counter

are counters of the sequence of numbers from 0 to 9. Since there are ten numbers in total, we will need four bits to encode them all.

As mentioned, each state that encodes a number will correspond to the next state that will encode the next number, finally reaching the number 9 beyond which it will start again from 0. The truth table, therefore, will be the following:

The remaining six combinations are all redundant terms.

We obtain the following expressions:

$$\begin{aligned} A^{n+1} &= \bar{A}^n \\ B^{n+1} &= \left[B\bar{A} + \bar{B}(A\bar{D})\right]^n \\ C^{n+1} &= \left[C\left(\bar{A} + \bar{B}\right) + \bar{C}(AB)\right]^n \\ D^{n+1} &= \left[D\bar{A} + \bar{D}(ABC)\right]^n \end{aligned}$$

Note that in the case of D^{n+1} we could have paired the minterm $ABC\bar{D}$ with the redundant term $ABCD$, but in this way we would have $\bar{D}$ which instead is useful for writing D^{n+1} in flip-flop form: if we note, in fact, all the expressions obtained are in the same form as equation [eq:circuiti-sequenziali-flip-flop-sr-espressione-algebrica], therefore they can all be built using J-K flip-flops.[1]

EXAMPLE 6.3 | § 6.2

Realize a four-bit down counter.

Let's start from the truth table.

[1] In the mnemonics to refer to the extended register, only the first letter of the register is used: we will therefore have *B* for *BC*, *D* for *DE* and *H* for *HL*.

PhyAMR

Now let's use Karnaugh maps to obtain the algebraic forms:

We obtain:

$$A^{n+1} = \bar{A}^n$$

$$B^{n+1} = \left[AB + \bar{A}\bar{B}\right]^n = A \bar{\oplus} B^n$$

$$\begin{aligned} C^{n+1} &= \left[AC + BC + \bar{A}\bar{B}\bar{C}\right]^n = \left[C(A+B) + \bar{C}\left(\bar{A}\bar{B}\right)\right]^n = \\ &= \left[C(A+B) + \bar{C}\left(A \bar{+} B\right)\right]^n = \left[C \oplus \left(A \bar{+} B\right)\right]^n \end{aligned}$$

$$\begin{aligned} D^{n+1} &= \left[AD + BD + CD + \bar{A}\bar{B}\bar{C}\bar{D}\right]^n = \\ &= \left[D(A+B+C) + \bar{D}\left(\bar{A}\bar{B}\bar{C}\right)\right]^n = \\ &= \left[D(A+B+C) + \bar{D}\left(A + \bar{B} + C\right)\right]^n = \left[D \oplus \left(A + \bar{B} + C\right)\right]^n \end{aligned}$$

6.2.2 Johnson counter

In the , counting is performed by changing only one bit at a time. Here is an example with three bits:

Before observing the algebraic expressions, let's understand how many states are possible for an m-bit Johnson counter. Let's divide the calculation into three logical steps:

1. The first state is certainly the one that defines zero, trivially given by $000\ldots0$.
2. At this point we start replacing the 0s from the left with 1s until we reach a state with only 1s:

   $$100\ldots0 \rightarrow 110\ldots0 \rightarrow \ldots \rightarrow 111\ldots1$$

   Since there are m bits, we will have a total of m states.
3. Finally, we will replace the 1s from left to right with 0s, excluding the last bit, since otherwise we would get $000\ldots0$, which we have already considered in the first point:

   $$011\ldots1 \rightarrow 001\ldots1 \rightarrow \ldots \rightarrow 000\ldots1$$

   Since we replace all bits except one, we get a total of $m-1$ states.

Summing the three contributions, we get:

$$N_{states} = 1 + m + (m-1) = 2m$$

Now let's look for the algebraic formulas, returning to the table proposed as an example:

$$A^{n+1} = \bar{C}^n$$
$$B^{n+1} = A^n$$
$$C^{n+1} = B^n$$

The solution is very elegant, providing us with a simple chain of D flip-flops:

PhyAMR

This circuit extends to a generic m-bit Johnson counter: it will be enough to put m D flip-flops in series and connect the output of one to the input of the other, with the exception of the last one with the first, since it will be necessary to connect the *negated* output to the input of the first flip-flop in the chain.

There is, however, a drawback: if - for any reason - the initial state of the system coincided with one of the redundant terms, the circuit would oscillate between two redundant terms, preventing the correct functioning of the counter.

Starting from $A = 0$, $B = 1$ and $C = 0$, in fact, we get:

$$\begin{cases} A^n = 0 \\ B^n = 1 \\ C^n = 0 \end{cases} \Longrightarrow \begin{cases} A^{n+1} = 1 \\ B^{n+1} = 0 \\ C^{n+1} = 1 \end{cases} \Longrightarrow \begin{cases} A^{n+2} = 0 \\ B^{n+2} = 1 \\ C^{n+2} = 0 \end{cases}$$

To solve this drawback, it is possible to rewrite the expression of B^{n+1} in a slightly more articulated way:

$$\begin{aligned} A^{n+1} &= \bar{C}^n \\ B^{n+1} &= \left[A\left(B + \bar{C}\right)\right]^n \\ C^{n+1} &= B^n \end{aligned}$$

Let's verify what happens starting from a redundant term:

$$\begin{cases} A^n = 0 \\ B^n = 1 \\ C^n = 0 \end{cases} \Longrightarrow \begin{cases} A^{n+1} = 1 \\ B^{n+1} = 0 \\ C^{n+1} = 1 \end{cases} \Longrightarrow \begin{cases} A^{n+2} = 0 \\ B^{n+2} = 0 \\ C^{n+2} = 0 \end{cases}$$

from which the system returns to normal counting.

PhyAMR

6.3 Registers

So far we have seen the storage of data exclusively at one bit within flip-flops. To obtain multi-bit memory devices, it is necessary to combine these flip-flops, obtaining the so-called .

6.3.1 Shift registers

A first idea is to place D flip-flops in cascade, also connecting the last one with the first, so that the bits are propagated from one flip-flop to the next, passing from state n to state $n + 1$ according to the synchronizer t. These registers are called .

Due to the ring closure of the device, a cyclicity occurs in the order of the bits: every n *clocks*, where n is the number of flip-flops, the same data sequence will be repeated.

To modify the bit sequence, it is possible to place multiplexers between one flip-flop and another, so that one input is connected to the output of the previous flip-flop and the other input to an external circuit that takes care of loading a new bit.

Without the closure of the device (i.e., the connection of the last flip-flop in the chain to the first), a shift register can be used for the conversion of a *serial* signal into a *parallel* signal and vice versa:

With a serial signal, data are transferred one bit at a time - in serial form, precisely - on the same line, so for parallel-to-serial conversion, the introduced vertical lines load the data onto the flip-flops of the register, which will have the task of transferring them to the right sequentially. In sequential-to-parallel conversion, instead, the reverse happens: following the loading of data onto the register, the vertical lines are activated, collecting all the bits at once.

EXAMPLE 6.4 | § 6.3

Realize an n-bit shift register that starts from a state where all bits are equal to 0*:*

$$00\ldots0 \rightarrow 10\ldots0 \rightarrow 01\ldots0 \rightarrow \ldots$$

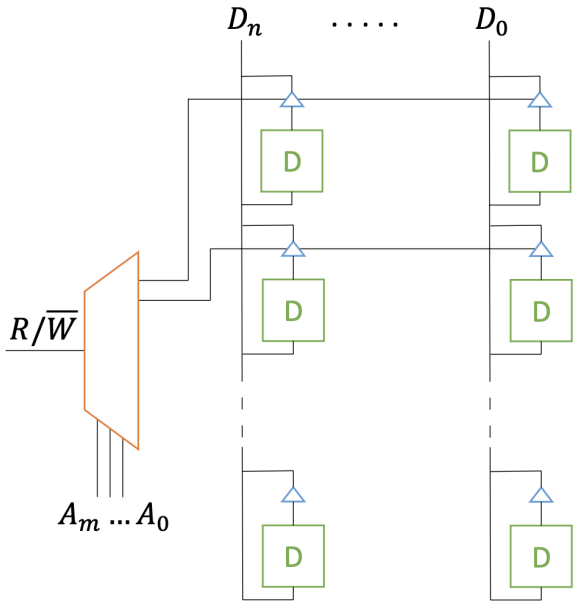
Let's reason by considering three bits, keeping in mind that the discussion can extend to generic n bits.

$$\begin{cases} A^{n+1} = \left[\bar{A}\bar{B}\bar{C}\right]^n = A + \bar{B} + C^n \\ B^{n+1} = A^n \\ C^{n+1} = B^n \end{cases}$$

This is therefore a normal shift register, with the only exception of the connection between the last flip-flop and the first, which includes the presence of a *NOR* gate whose inputs are all the bits of the device.

6.3.2 Memory

Placing flip-flops exclusively in sequence to create registers that store information becomes inconvenient beyond a certain number of bits. To organize such a device more efficiently, it is useful to create a *matrix* of flip-flops, whose rows can be accessed through a demultiplexer.



R0.5

Let's look at figure [fig:rom] a memory called (**Read Only Memory**),[2] capable only of reading the data contained within the flip-flops previously loaded with the data during the device's realization phase. Once the desired line

[2] Not necessarily located in the last memory addresses.

is selected by means of a demultiplexer, the tristates positioned at the output of the flip-flops of that row will be activated, closing the flip-flop circuit and allowing the data to be read from above.

If the circuit also allows writing data to memory, we will speak of (**Random Access Memory**), where indicates the fact that the access time to each memory location is the same, since it is possible to access the memory unit directly, i.e., without sequentially traversing others. For efficiency reasons, RAM has the characteristic of *volatility*, i.e., the loss of information when the electrical power is interrupted.

6.4 State machines

We conclude the chapter on sequential logic by talking about , reasoning in particular about *finite* state machines (**FSM**, which stands for ***Finite State Machines***). These are in all respects system models that move between different states Q_i (in *discrete* and *finite* number) considering one or more inputs C_i at each new state and producing outputs P_i, also called **control signals**.

Let's summarize the operation in the following steps:[3]

1. The sequential part will define a new state Q^n starting from the received input, also considering the previous state Q^{n-1} that the sequential circuit itself
2. This state (together, eventually, with the input C^n, as we will see better later) will produce the control signal P^n
3. The new input signal C^n will encounter a combinatorial logic circuit together with the state Q^n, producing the input that will define Q^{n+1}, from which the cycle will restart

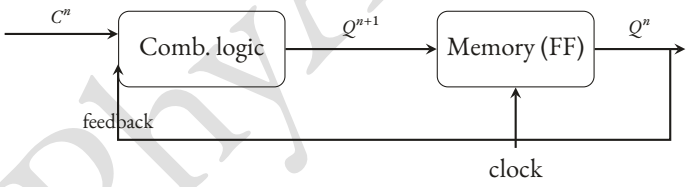


FIG. 6.5 General diagram of a state machine

We distinguish two types of state machines:

- : the output P is associated directly with the state Q, therefore it will depend only on it
- : the output P is not associated exclusively with the state Q but will also depend on the input C

To understand the difference between the two behaviors, let's start with an example: let's imagine having two inputs, M and N (which we will indicate in the form (M, N)) and wanting to identify the transition $(1, 0) \rightarrow (0, 1)$; to do this, we will build our machine in such a way that the control signal is equal to 0 for any other transition and is instead equal to 1 when this transition occurs.

Let's start by building a Moore machine and let's define the number of states:

1. The system will first be in an initial state S_0: what happens at that point? There are two possibilities: if the input is $(0, 0)$, $(0, 1)$ or $(1, 1)$, it will remain in state S_0, while if the input is $(1, 0)$, the system will into state S_1.

[3] We remember that a memory address is encoded by 2 bytes, and in it only one byte can be contained; in the SAP-2, in fact, two addresses were used to save the current point.

PIYAMR

2. Once in state S_1, there are three possibilities this time: if the input is $(0,0)$ or $(1,1)$, the system will return to state S_0, having to start over; if the input is $(1,0)$, it will remain in S_1, the condition for it to be effectively in S_1 having been repeated; finally, if the input is $(0,1)$, then the desired transition has actually occurred, so the system can move to S_2.
3. If the system is in S_2, as mentioned, the objective has been reached, so the output will signal $P = 1$. At that point there are two possibilities: if the new input is $(0,0)$, $(0,1)$ or $(1,1)$, the system will transition to S_0, having to repeat the cycle, while if the input is $(1,0)$, the system will go directly to S_1, the desired transition having partially occurred again.

To summarize what is written in a single scheme, it is possible to create a , in which the states are indicated by circles and the transitions by arrows in which the associated input is specified.

As usual, to build the circuit we must start from the truth table, so we need to define all possible inputs and outputs of the system.

Inputs will certainly be M and N along with the variables that define the state of the system; since there are three states in total, we will need two variables that will uniquely identify the state itself, which we will call A and B. To the input (M, N, A^n, B^n) will correspond the output (A^{n+1}, B^{n+1}, Z), where Z is the control signal.

We also note from the truth table that the control signal is uniquely determined by the state: it is sufficient to set $Z = B$, in fact, since it is sufficient for B to be active for the presence of the system in state S_2 to be signaled, which indicates the correct transition.

For the rest, we obtain:

$$\begin{cases} A^{n+1} = \left[M\bar{N}\right]^n \\ B^{n+1} = \left[\bar{M}NA\right]^n \end{cases}$$

Translated into circuit form:

Let's now see how to realize the same system using a Mealy machine:

- This time too the system will be in an initial state S_0, to which it will be brought back with all inputs except $(1,0)$, which will take it to state S_1.
- From state S_1 there are three possibilities this time: if the input is $(0,0)$ or $(1,1)$, the system will return to S_0 with a control signal $Z = 0$; if the input is $(1,0)$, the system will remain in S_1; if the input is $(0,1)$, the system will return to S_0 with a control signal $Z = 1$, indicating that the desired transition has actually occurred.

Since this time there are only two states, it will be sufficient to consider a single variable A to identify them.

In this case, the state is not sufficient to define the control signal, so we will also have to resort to the input (M, N):

$$Z = \left[\bar{M}NA\right]^n$$

The next state, instead, is given by

$$A^{n+1} = \left[M\bar{N}\bar{A} + M\bar{N}A\right]^n = \left[M\bar{N}\right]^n$$

So we find the following circuit:

PhyAMR

EXAMPLE 6.5 | § 6.4

Realize a state machine that identifies the bit sequence

$$\ldots 011010 \ldots$$

on a data line M.

Also, realize the relative circuit through the use of J-K flip-flops.

The first thing to do is to start from the Moore diagram.

From this we can build the truth table, which will be determined by four inputs, namely the bit M along with the three variables useful for defining the state of the system (we use three since the total number of states is seven).

The control signal will evidently be

$$Z = BC$$

To find the next states, we use, as usual, Karnaugh maps. Keep in mind, however, that the text requires the circuit to be obtained using J-K flip-flops, so the generic variable Q^{n+1} must be in the form

$$Q^{n+1} = \left(\bar{Q}J + Q\bar{K}\right)^n$$

The various groupings of the map, therefore, must be made in such a way as to always distinguish the terms with Q and with $\bar{Q}$, or - in other words - in such a way that the variable Q never simplifies.

$$\begin{aligned} A^{n+1} &= \left[\bar{A}\left(B\bar{C} + \bar{B}C + \bar{M}\right) + A\left(CM + \bar{B}\bar{C}\bar{M}\right)\right]^n = \\ &= \left[\bar{A}\left(B \oplus C + \bar{M}\right) + A\left(CM + \bar{B}\bar{C}\bar{M}\right)\right]^n \end{aligned}$$

$$\begin{aligned} B^{n+1} &= \left[\bar{B}(AC + AM) + B\left(\bar{A}\bar{C}M\right)\right]^n = \\ &= \left[\bar{B}(A(C + M)) + B\left(\bar{A}\bar{C}M\right)\right]^n \end{aligned}$$

$$C^{n+1} = \left[\bar{C}\left(AB\bar{M}\right) + C\left(A\bar{M} + \bar{A}\bar{B}M\right)\right]^n$$

With much effort and much patience (which I lack) it is possible to realize the circuit with three J-K flip-flops.

EXAMPLE 6.6 | § 6.4

Realize an elevator controller with the following characteristics:

- *The elevator can be on one of three floors: ground floor, first floor or second floor*
- *There is a button that expresses two possible values: and*
- *The elevator goes up one floor every time the button is on and goes down one floor every time it is on*
- *There are two lights that indicate the current floor: both lights off (00) indicate the ground floor; the first light off and the second on (01) indicate the first floor; the first light on and the second off (10) indicate the second floor.*

It is evident that the elevator floors correspond to the states in which the state machine can be, while the button is nothing more than the input described by a single bit, associating *up* with 1 and *down* with 0.

In this system there is no control signal.

The truth table is the following:

$$\begin{cases} A^{n+1} = [AM + BM]^n = [M(A + B)]^n \\ B^{n+1} = \left[A\bar{M} + \bar{A}\bar{B}M\right]^n \end{cases}$$

The circuit, therefore, is:

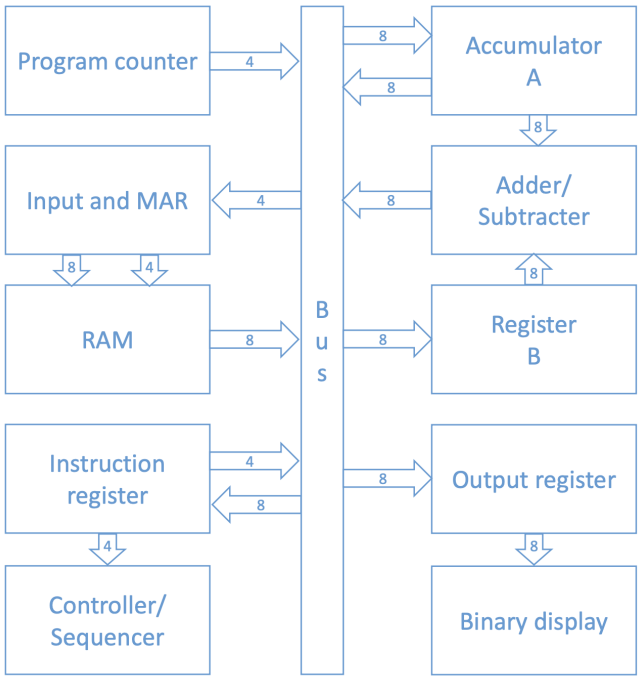
PhyAMR

SAP-1

The is a computer designed for educational purposes, in order to illustrate the ideas behind the functioning of a modern calculator without providing superfluous details: the name, not by chance, is the acronym for *Simple-As-Possible* (*computer*, implied).

We will explore three different generations of SAP, which will follow the evolution towards modern computers. Although primitive, the SAP-1 constitutes a great challenge for a beginner, so it is necessary to understand it thoroughly before moving on to the next generations.

7.1 Architecture



R0.53

The diagram we will refer to is the one shown in figure [fig:sap-1], which however does not take into account the *clock*, the *reset* and the various *control lines*.

We can divide the computer into four units:

1.
2.
3. (*Arithmetic-Logic Unit*)
4. **I/O Unit** (*Input/Output*)

The four units are not explicitly shown in the diagram, as it highlights the registers and the connections between them and to the data bus; we will see below which registers will belong to which units.

All connections of the registers with the data bus are made through tristates, in order to allow an orderly transfer of data; all other outputs of the registers are instead *two-state*, that is, unidirectional communication lines.

Let's see below the role of the various registers.

1. : this is a counter that counts from 0000 to 1111. Its role is to send the *address* of the instruction to the memory, so that it can be and executed. This register is also called *pointer* or *instruction pointer*, as it to the memory address that the computer will have to access, thus keeping track of where the program is.
2. : this is the *interface* between the SAP and the memory. Any incoming 8-bit data is loaded into the *input register*, while the 4-bit memory address (from which to read or in which to write) is loaded into the *MAR* (*Memory Address Register*). Due to its dual nature, it is an element that is part of the SAP's memory, but also of the **I/O Unit**.
3. : it is a 16 × 8 matrix that receives the address from the MAR as input and performs the reading of the data stored in that address; this data is then brought to the bus so that it can be used by other registers. Writing to the RAM, on the other hand, only occurs during the programming phase of the SAP.
4. : it stores the information read from the RAM; this is then divided in two: the first nibble, representing the instruction to be executed, is sent to the *controller/sequencer*, while the second nibble, which contains the memory address object of the instruction, is sent back to the bus.
5. : an element that manages and coordinates the different phases of the instruction execution cycle after receiving the nibble from the instruction register as input. It is also involved in the synchronization of the different units, ensuring that they work in the correct order: at the beginning of the execution of a program, for example, it sends a *reset* signal to the program counter and the instruction register.
6. : it temporarily stores the intermediate results of the operations performed; it has direct access to the adder/subtractor, where the arithmetic operations are performed.
7. : the SAP uses the two's complement convention to perform sums and differences between numbers. This element is *asynchronous*, in the sense that it is not coordinated by a *clock* but performs the operations as soon as it receives the data as input.
8. : another register in communication with the adder/subtractor, with the role of providing the number to be added (or subtracted) to the content of the accumulator.
9. **Output register**: this is the to the , where the content of the accumulator is loaded at the end of the program execution.
10. **Binary display**: a row of eight LEDs that shows the user the content of the output register in binary form.

7.2 Instruction set

A computer is a useless pile of electronic components until someone decides to program it. A *program* is nothing more than a series of instructions that must be loaded into memory before it is executed.

We recall that a data loaded into memory has a total of 8 bits, and in case of an operation this byte is divided as follows:

- the first nibble is dedicated to the instruction itself (**opcode**, also called *op code*)
- the second nibble is dedicated to the eventual memory address that is the object of the instruction (**operand**)

PIYAMR

$$\text{Instruction} = \underbrace{bbbb}_{\text{Op code}} \overbrace{bbbb}^{\text{Operand}}$$

The number of instructions and data of the program cannot exceed 16, as this is the total number of addresses in memory. For this reason, the memory address is indicated by a single hexadecimal digit.

The SAP-1 has access to a set composed of five different instructions:

1. (): loads the accumulator with the content inside the indicated memory address.
   If the number to be loaded is in address 0x8, the instruction *LDA* 0x8 will load this number into the accumulator:

$$R_8 = 0000\ 0011 \xrightarrow{LDA\ 0x8} A = 0000\ 0011$$

2. : adds to the content of the accumulator the content inside the indicated memory address.
   If the number to be added is in address 0x*D*, the instruction *ADD* 0x*D* will load this number into the B register so that it is added to the number in the accumulator; in the end, the accumulator will contain the sum of the two numbers:

$$\begin{cases} R_{13} = 0010\ 0000 \\ A = 0000\ 0011 \end{cases} \xrightarrow{ADD\ 0xD} A = 0010\ 0011$$

3. : subtracts from the content of the accumulator the content inside the indicated memory address.
   If the number to be subtracted is in address 0x*E*, the instruction *SUB* 0x*E* will load this number into the B register so that it is subtracted from the number in the accumulator; in the end, the accumulator will contain the difference between the two numbers:

$$\begin{cases} R_{14} = 0000\ 0001 \\ A = 0010\ 0011 \end{cases} \xrightarrow{SUB\ 0xE} A = 0010\ 0010$$

4. : copies the content of the accumulator to the output register; it does not require any memory address as an object.

$$A = 0010\ 0010 \xrightarrow{OUT} OUT = 0010\ 0010$$

5. (): tells the computer to stop executing the program. It is necessary to place this instruction at the end of every program in the SAP-1, as otherwise unforeseen operations would be obtained caused by uninterrupted processing. It does not require any memory address as an object.

The following table shows the opcodes associated with the five instructions.

7.3 Assembly language

The programming language that we have implicitly used in the examples seen above is the , which requires three elements:

1. The memory address where the instruction or data is located
2. The *mnemonic*, that is the three letters we used to indicate the instruction

PhyAMR

3. The eventual memory address object of the instruction

By convention, all unused memory addresses are set to 0x*FF*.

Let's try, for example, to write in assembly language the program described step by step in the description of the instructions.

This program performs the following operation:

$$3 + 32 - 1$$

and should return, precisely, 34 in binary, that is 0010 0010.

Using the opcode and the operand explicitly, instead, we will get the so-called .

where with X we indicate a nibble that can take any value.

7.4 Timing state

The SAP executes the instructions thanks to the control unit, which sends *words* - called, precisely, **control words** - that allow the fetching and execution of the instructions themselves.

As we know, everything is orchestrated by a *clock* that sends periodic pulses to the system. Each clock cycle marks the transition from one *state* of the system to the next state: due to their temporal nature, these states are called .

Each timing state is comprised between two *falling edges* of the clock, and during this period the SAP performs actions such as to modify the content of the registers following what is indicated by the control words.

The timing states are indicated by a **ring counter**, that is a shift register in which all the bits are set to 0 except for one set to 1, whose position is associated with the timing state itself. As we will see in more detail below, each instruction of the SAP-1 requires at most six timing states to be executed, so the ring counter will consist of six bits, that is, six flip-flops, whose bits scroll as

$$000001 \rightarrow 000010 \rightarrow 000100 \rightarrow 001000 \rightarrow 010000 \rightarrow 100000 \rightarrow 000001$$

Indicating with T_i the bit associated with the i-th time state we will then write

$$T = T_6 T_5 T_4 T_3 T_2 T_1$$

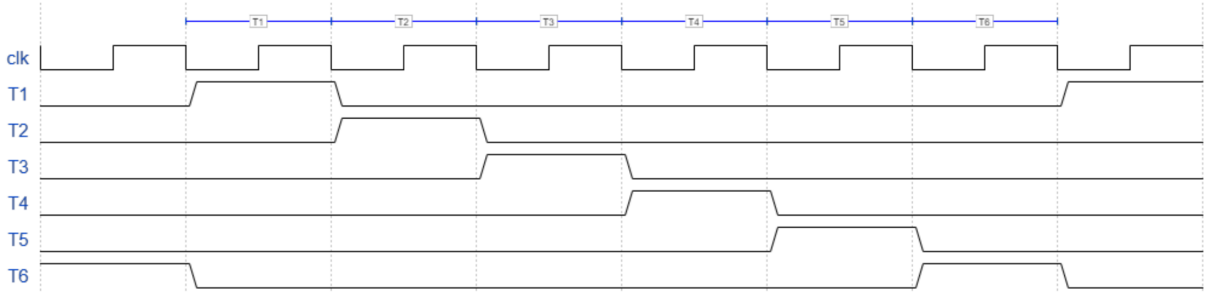


FIG. 7.1 Timing states compared with the clock

PhyAMR

7.5 Instruction cycle

We have mentioned the fact that each instruction of the SAP-1 is fetched and executed in a total of - at most - six timing states. In truth, six timing states define what is called a **machine cycle**; since in the SAP-1 this coincides with the execution period of an instruction, we can also refer to it with the name of , but in general an instruction cycle can use more than one machine cycle.

The instruction cycle is in turn divided into two cycles:

1. the , common to all instructions, which is responsible for fetching the instruction from memory and bringing it to the controller/sequencer
2. the , different for each instruction, which is responsible for executing the instruction involving the ALU and the I/O unit

In the SAP-1 both cycles are composed of three timing states. They are also called **macroinstructions**, in contrast to the control words which are instead called **microinstructions**; moreover, a microinstruction is associated with each timing state, so each macroinstruction is composed of three microinstructions, and the instruction cycle is composed of six total microinstructions.

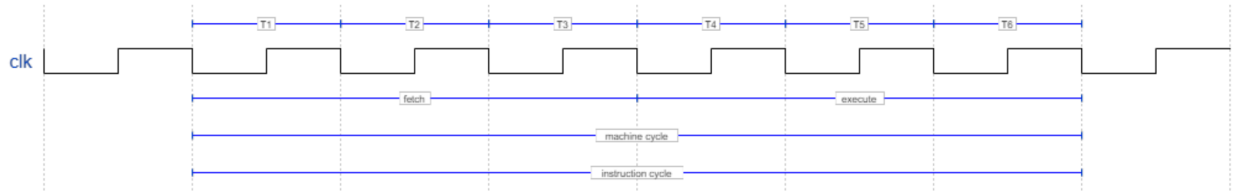


FIG. 7.2 Structure of the cycles in the SAP-1

7.5.1 Fetch cycle

1. T_1:
   Registers involved: ,
   The address indicated by the program counter is sent to the input and MAR.
2. T_2:
   Registers involved:
   The program counter is incremented.
3. T_3:
   Registers involved: , ,
   The RAM receives the address from the input and MAR, sends the information contained in that address on the bus, being finally collected and stored in the instruction register.

7.5.2 Execution cycles

As mentioned, the execution cycle changes based on the instruction used. In reference to the instructions, we will refer to the execution cycles as ***routines***.

- 1. T_4: ,

The opcode is sent to the controller/sequencer, while the operand is loaded into the input and MAR.

2. T_5: , , ,
   The object memory address is loaded into the RAM, the data is extracted and loaded onto the bus to be finally passed to the accumulator.[1]
3. T_6: *no register is active*
   State *nop* (no-operation).

- 1. T_4: ,
     The opcode is sent to the controller/sequencer, while the operand is loaded into the input and MAR.
  2. T_5: , , ,
     The object memory address is loaded into the RAM, the data is extracted and loaded onto the bus to be finally passed to the B register.[2]
  3. T_6: , , ,
     The contents in the accumulator and in the B register are passed to the adder/subtractor where the sum (in case of *ADD*) or the difference (in case of *SUB*) will be performed. The result will then be passed to the bus which will copy it back into the accumulator. Since the accumulator works both in output and in input, appropriate delay signals are sent from the control unit during the operation to avoid a possible race competition.
- 1. T_4: , , *Output register*, *Binary display*
     The opcode is sent to the controller/sequencer, while the data from the accumulator is copied to the output register, whose content is shown directly on the binary display.
  2. T_5: *no register is active*
     State *nop* (no-operation).
  3. T_6: *no register is active*
     State *nop* (no-operation).

  The *HLT* does not have a control routine as no register is involved in its execution: a final clock cycle will mark the acquisition of the instruction in the controller/sequencer, after which the clock will be interrupted as well as any operation.

PhyAMR

EXAMPLE 7.1 | § 7.5

Write a SAP-1 program that executes and displays the result of the arithmetic operation 4+5-3.

If the clock frequency is 4 MHz, how long does it take to execute this program?

We will have to, in order: load the number 4 into the accumulator, add the number 5, subtract the number 3, and finally send the result to the output on the display.

Given the clock frequency, the time associated with each timing state will be

$$T = \frac{1}{4 \cdot 10^6\ Hz} = 250\ ns$$

The program consists of five instructions; we remember, however, that with the *HLT* only the fetch cycle is executed, consisting of three timing states, and a final clock cycle after which the execution is interrupted, therefore four total.

[1] In the mnemonics to refer to the extended register, only the first letter of the register is used: we will therefore have *B* for *BC*, *D* for *DE* and *H* for *HL*.

[2] Not necessarily located in the last memory addresses.

Ultimately, the total number of timing states is given by

$$6 \cdot 4 + 4 = 28$$

From which follows the total execution time of the program:

$$T_{tot} = 28 \cdot 250\ ns = 7000\ ns = 7\ \mu s$$

PhyAMR

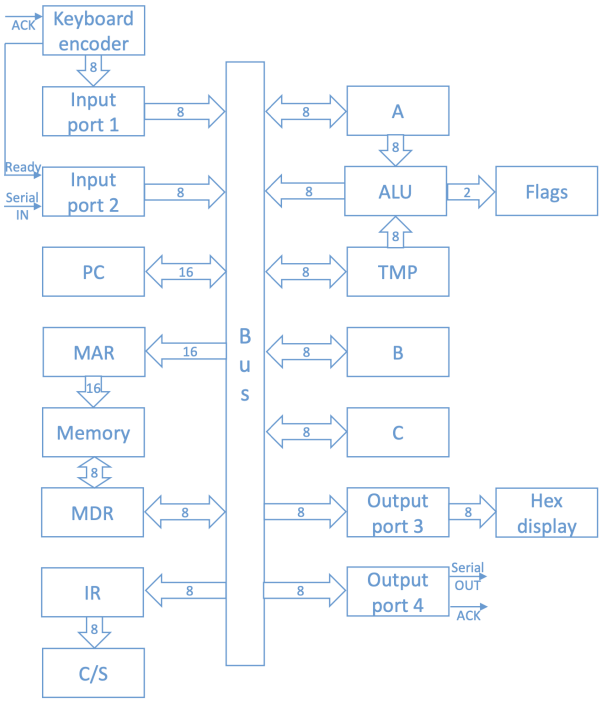
PhyAMR

SAP-2

The SAP-1 is a computer in that it is able to store a program and data before the calculations begin; subsequently it is able to execute the instructions of the program without human intervention.

The next step in the evolution towards modern computers are the *jump* instructions, which allow the computer to repeat or skip parts of a program; as we will see below, these tools will open the doors to new levels of computation unthinkable with the previous generation.

8.1 Architecture



R0.49

The diagram is shown in figure [fig:sap-2]; just like the one presented for the SAP-1, the presence of the clock, the reset and the control lines is neglected.

The main difference is the presence, in addition to the tristate connections between the registers and the bus, of the **tristate registers**: the registers of the SAP-2 have more input and output pins, and to reduce the amount of cables needed to connect the register to the computer - also called *wiring capacity* - these inputs are short-circuited; this does not cause problems, contrary to what one might think, due to the presence of appropriate control lines that take care of isolating internally input or output in an alternating way. Figure 1.1 clearly shows the saving in wiring capacity: instead of using two sets of cables (one for the input and one for the output) it will be sufficient to use a single set for the entire register; it will then be the *LOAD* and *ENABLE* inputs - which cannot be active at the same time - to take care of isolating the appropriate connections. These registers are indicated in the diagram in figure [fig:sap-2] by bidirectional arrows in the connections with other parts of the SAP.

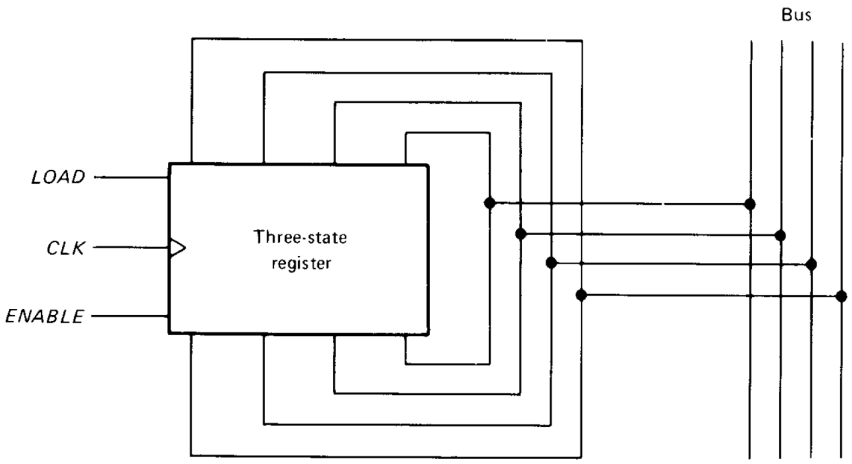


FIG. 8.1 Diagram of a tristate register

Let's explore below the characteristics of the registers and the novelties compared to the SAP-1.

1. **Input ports**: one port receives data in parallel through a keyboard that encodes hexadecimal values, while a second port is in charge of loading data sequentially; the latter, moreover, has an input called *READY* that allows the orderly passage of data from the first input port to the SAP processor: only when *READY* is high, in fact, is the transfer possible.
2. : analogous to that of the SAP-1, with the only difference that it consists of 16 bits and therefore counts from 0x0000 to 0x*FFFF*; moreover, it is always reset to 0x0000 before each execution of a program.
3. : during the *fetch cycle* the MAR receives an address from the program counter which will in turn be sent to the memory, consisting of 64 *kB*. In particular, the first 2 *kB* of memory (from 0x0000 to 0x07*FF*) are ROM, and contain the initialization data at the power-on of the SAP and the decoding of the hexadecimal inputs from the keyboard; the remaining 62 *kB* (from 0x0800 to 0x*FFFF*) constitute the computer's RAM.[1]
4. : an 8-bit register in direct connection with the RAM with the task of writing data to memory or reading data from memory and sending them to the bus; the name is an acronym for *Memory Data Register*.
5. : analogous to that of the SAP-1 with the exception of the size, since the number of instructions is greater and 8-bit opcodes are required.
6. : with the exception of the fact that the control words have a larger size, it is conceptually identical to that of the SAP-1.
7. : also identical to the SAP-1 version with the difference that it consists of eight bits and no longer four.
8. : in addition to arithmetic operations, the SAP-2 is also able to perform logical operations. The register that deals with the calculation is also connected to two *flags*, that is, two flip-flops that keep track of the change of certain conditions: in particular, a *sign flag* is set when the content inside the accumulator becomes negative and a *zero flag* when it becomes zero; in general, all instructions involving the accumulator can influence the flags.
9. : the register in communication with the ALU is called *TMP*, since it stores the values only temporarily before loading them onto the ALU so that the operation can be performed; to this are added two other registers in which to save data, giving much more freedom in the calculations.

[1] In the mnemonics to refer to the extended register, only the first letter of the register is used: we will therefore have *B* for *BC*, *D* for *DE* and *H* for *HL*.

PrYAMR

10. **Output ports**: similarly to the input ports, here too we have a port associated with a hexadecimal data (which is shown on a special display) and another port linked instead to a serial type signal. From the latter comes a further signal, called *ACKNOWLEDGE*, which connects to the first input port communicating that the transfer of data in hexadecimal to the processor has been successful, thus setting the *READY* signal to 0 which will in turn set *ACKNOWLEDGE* to 0: this communication process between input, output and processor is called **handshaking**.

8.2 Instructions

With the increase in computational capacity, the number of instructions also increases, from the five of the SAP-1 to the forty-three of the SAP-2.

Before going into the individual instructions, it is useful to understand how the fetch and execution cycles are regulated. Let's start by saying that the *fetch cycle* is identical to the one seen in the SAP-1, so it is divided into three timing states equal for all instructions. The machine cycle is also this time composed of six timing states, but the novelty is that an instruction cycle does not always coincide with a machine cycle: the computationally most complex instruction - which therefore requires the greatest number of timing states - is the *CALL*, which requires a total of three machine cycles to be executed. In figure 1.2 an example of an instruction cycle consisting of two machine cycles is shown.

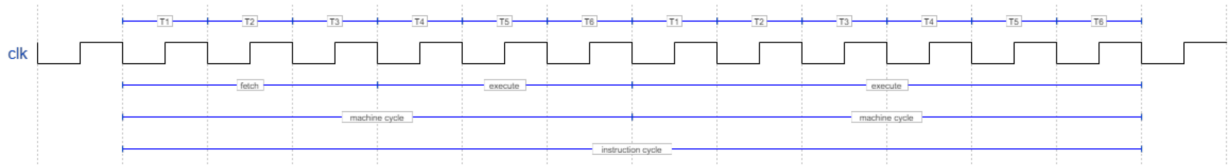


FIG. 8.2 Example of subdivision of an instruction cycle

That said, we distinguish five types of instructions:

1. : require access to memory during the execution cycle.
2. : access the registers directly without going through memory; for this reason they are on average faster than the previous ones.
3. : modify the order of the program sequence by acting on the program counter.
4. : allow logical operations.
5. **Other**: do not fall into any of the above categories in particular.

8.2.1 Memory-reference instructions

1. (): loads the accumulator with the content inside the indicated memory address.
   It requires three total bytes for execution: one byte is given by the op code and two bytes by the memory address, since this is made up of 16 bits. In total it therefore requires three memory addresses.
   If the number to be loaded is in the address 0x0071 the instruction *LDA* 0x0071 will load that number into the accumulator:

$$R_{113} = 0001\ 0011 \xrightarrow{LDA\ 0x0071} A = 0001\ 0011$$

2. (): loads the content of the accumulator into the indicated memory address.
   It requires three total bytes for execution: one byte is given by the op code and two bytes by the memory address, since this is made up of 16 bits. In total it therefore requires three memory addresses.
   If the memory address in which to load the content is 0x00$A8$ the instruction STA 0x00$A8$ will load the content of the accumulator into that address:

$$A = 0001\ 0011 \xrightarrow{STA\ 0x00A8} R_{168} = 0001\ 0011$$

3. (): loads a data into the register X.
   Since there are three registers (A, B, C) this is a total of three instructions.
   It requires two total bytes for execution: one byte is given by the op code and the other by the 8-bit operand. In total it therefore requires two memory addresses.
   If the register in which we want to load the data is B, and this data is equal to 0x35, it will be enough to use the instruction $MVI\ B$ 0x35:

$$\xrightarrow{MVI\ B\ 0x35} B = 0011\ 0101$$

8.2.2 Register instructions

1. (): copies the data from the register Y to the register X.
   Since there are three registers (A, B, C) there will be six total combinations of X and Y and therefore this is a total of six instructions.

$$\begin{cases} A = 0001\ 0011 \\ B = 0011\ 0101 \end{cases} \xrightarrow{MOV\ A\ B} \begin{cases} A = 0011\ 0101 \\ B = 0011\ 0101 \end{cases}$$

2. : adds the content of the register X to the content of the accumulator.
   Since there are two registers (B, C) this is a total of two instructions.

$$\begin{cases} A = 0011\ 0101 \\ C = 1000\ 1001 \end{cases} \xrightarrow{ADD\ C} A = 1011\ 1110$$

3. : subtracts the content of the register X from the content of the accumulator.
   Since there are two registers (B, C) this is a total of two instructions.

$$\begin{cases} A = 1011\ 1110 \\ B = 0011\ 0101 \end{cases} \xrightarrow{ADD\ C} A = 1000\ 1001$$

4. (): adds 0x01 to the content of the register X.
   Since there are three registers (A, B, C) this is a total of three instructions.

$$C = 1000\ 1001 \xrightarrow{INR\ C} C = 1000\ 1010$$

5. (): subtracts 0x01 from the content of the register X.
   Since there are three registers (A, B, C) this is a total of three instructions.

$$B = 0011\ 0101 \xrightarrow{DCR\ B} B = 0011\ 0100$$

PhyAMR

8.2.3 Jump instructions

1. (): tells the control unit to proceed from the designated memory address.
   The instruction *JMP* 0x0020 placed at the address 0x0005 sets the content of the program counter equal to the operand of the jump, that is precisely 0x0020:

$$PC = 0x0005 \xrightarrow{JMP\ 0x0020} PC = 0x0020$$

2. (): performs the jump only if the sign flag is active.
   The sign flag is defined as follows:

$$S = \begin{cases} 0 \text{ if } A \geqslant 0 \\ 1 \text{ if } A < 0 \end{cases}$$

   The instruction *JM* 0x0020, therefore, will give

$$PC = 0x0005 \xrightarrow{JM\ 0x0020} \begin{cases} PC = 0x0005 \text{ if } S = 0 \\ PC = 0x0020 \text{ if } S = 1 \end{cases}$$

3. (): performs the jump only if the zero flag is active.
   The zero flag is defined as follows:

$$Z = \begin{cases} 0 \text{ if } A \neq 0 \\ 1 \text{ if } A = 0 \end{cases}$$

   The instruction *JZ* 0x0020, therefore, will give

$$PC = 0x0005 \xrightarrow{JZ\ 0x0020} \begin{cases} PC = 0x0005 \text{ if } Z = 0 \\ PC = 0x0020 \text{ if } Z = 1 \end{cases}$$

4. (): performs the jump only if the zero flag is not active.
   The zero flag is defined as follows:

$$Z = \begin{cases} 0 \text{ if } A \neq 0 \\ 1 \text{ if } A = 0 \end{cases}$$

   The instruction *JNZ* 0x0020, therefore, will give

$$PC = 0x0005 \xrightarrow{JZ\ 0x0020} \begin{cases} PC = 0x0020 \text{ if } Z = 0 \\ PC = 0x0005 \text{ if } Z = 1 \end{cases}$$

5. and (): calls a *subroutine* finally returning to the main program.
   We call *subroutine* a program located in a specific point in memory that can be used by another program. The *CALL* instruction requires as an operand the memory address where the subroutine begins. Before the SAP proceeds with the subroutine, the two bytes of the current memory address - indicated by the program counter - are saved in the last two addresses of the RAM, that is in 0x*FFFE* and 0x*FFFF*.
   The *RET* instruction for a subroutine corresponds to the *HLT* of a program: it must always be inserted so that the computer can return to the main program; the SAP, at that point, will retrieve the data contained in

PhyAMR

the last addresses of the RAM to know where to return.

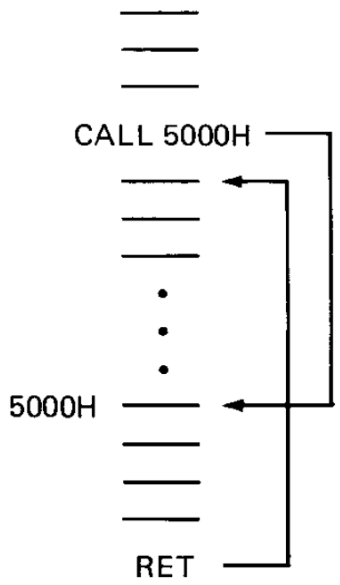


FIG. 8.3 Example of a subroutine at 0x5000

8.2.4 Logical instructions

- (): inverts all the bits contained in the accumulator.

$$A = 1000\ 1001 \xrightarrow{CMA} A = 0111\ 0110$$

- (): performs a bitwise *AND* operation between the accumulator and the register *X*. Since there are two registers (*B*, *C*) this is a total of two instructions.

$$\begin{cases} A = 0111\ 0110 \\ B = 0011\ 0100 \end{cases} \xrightarrow{ANA\ B} A = 0011\ 0100$$

- (): performs a bitwise *OR* operation between the accumulator and the register *X*. Since there are two registers (*B*, *C*) this is a total of two instructions.

$$\begin{cases} A = 0011\ 0100 \\ C = 1000\ 1010 \end{cases} \xrightarrow{ORA\ C} A = 1011\ 1110$$

- (): performs a bitwise *XOR* operation between the accumulator and the register *X*. Since there are two registers (*B*, *C*) this is a total of two instructions.

$$\begin{cases} A = 1011\ 1110 \\ B = 0011\ 0100 \end{cases} \xrightarrow{XRA\ B} A = 1000\ 1010$$

- (): performs a bitwise *AND* operation between the accumulator and an indicated byte.

$$A = 1000\ 1010 \xrightarrow{ANI\ 0xB7} A = 1000\ 0010$$

PhyAMR

- (): performs a bitwise *OR* operation between the accumulator and an indicated byte.

$$A = 1000\ 0010 \xrightarrow{ORI\ 0x3E} A = 1011\ 1110$$

- (): performs a bitwise *XOR* operation between the accumulator and an indicated byte.

$$A = 1011\ 1110 \xrightarrow{XRI\ 0xAD} A = 0001\ 0011$$

8.2.5 Other

- **NOP** (): performs no operation.
  It is used to delay data processing, occupying four timing states between fetch and execution in which no register is modified.
- **HLT** (): tells the computer to stop executing the program.
  It is necessary to place this instruction at the end of every program in the SAP-2, as otherwise unforeseen operations would be obtained caused by uninterrupted processing.
- **IN** (): transfers a data into the accumulator from the indicated input port.
  Since there are only two input ports of the SAP-2, the only allowed operands for *IN* are 0x01 and 0x02.
- **OUT** (): transfers a data from the accumulator to the indicated output port.
  Since there are only two output ports of the SAP-2, the only allowed operands for *OUT* are 0x03 and 0x04.
- **RAL** (): shifts all the bits of the accumulator one position to the left, causing the *most significant bit* to become the *least significant bit*.

$$A = 0001\ 0011 \xrightarrow{RAL} A = 0010\ 0110$$

- **RAR** (): shifts all the bits of the accumulator one position to the right, causing the *least significant bit* to become the *most significant bit*.

$$A = 0010\ 0110 \xrightarrow{RAR} A = 0001\ 0011$$

8.2.6 Overall table

EXAMPLE 8.1 | § 8.2

How many times is the DCR instruction executed in the following program?

Also, calculate how much memory space this program occupies.

Actually the code itself suggests the memory locations:

1. *MVI C* occupies one byte for the opcode and one byte for the operand, so two bytes in total
2. *DCR C* occupies only one byte for the opcode
3. *JZ* occupies one byte for the opcode and two bytes for the operand, so three bytes in total
4. *JMP* occupies one byte for the opcode and two bytes for the operand, so three bytes in total
5. *HLT* occupies only one byte for the opcode

The total is 10 bytes of memory, distributed as follows:

PhyAMR

The first instruction loads the number 3 into the register *C*, which is decremented to 2 in the next instruction. For the decrement to occur, the 3 is copied into the accumulator so that the ALU subtracts 1 and the result returns, precisely, to the register *C*: the number 2 thus remains in the accumulator, which is different from zero and therefore leaves the zero flag inactive. For this reason the *JZ* instruction is ignored and we pass directly to the *JMP*, which leads to a new decrement: this process is repeated once more and then once again; at that point the decrement will bring the number to 0, will activate the zero flag and therefore the *JZ* instruction that will stop the program. We deduce that the *DCR C* instruction is executed a total of three times.

EXAMPLE 8.2 | § 8.2

Write a program that multiplies the numbers 11_{10} *and* 5_{10}.

The SAP-2 does not have the ability to perform a multiplication between numbers, so the only possibility is to try to add the number 11 five times with itself: for this we will load the number 0 into the accumulator *A*, the number 11 into the register *B* and the number 5 into the register *C*.

From here on I rely on the solution reported in the book (similar to the one given by the professor) which however is not clear to me in one step. To perform the sum, the instruction *ADD B* is sufficient and for it to be executed five times it is necessary to use the 5 in the register *C* as a counter, a bit like what was done in the previous exercise through *DCR C*. At this point the book reports the instruction *JZ* which refers directly to the *HLT* and then a *JMP* which refers, instead, back to the sum *ADD B*.

My doubt concerns precisely the decrement of the content of *C*: for this to happen, the number in *C* must be copied into the accumulator; but if this contains the sum of the various 11s, how can we put everything in the same register without losing data? My hypothesis is that before the data transfer from *C* to *A* the content of the accumulator is copied into the register *TMP*; the counter would thus move from the accumulator to the ALU where 1 is subtracted (which would also pass from the accumulator) so that the zero flag is eventually activated; finally, the updated counter would return to *C* and the sum from *TMP* to *A*. On the matter I have only found a post on reddit that reports my same doubt (https://www.reddit.com/r/beneater/comments/jppirt/how_do_sap2s_inrdcr_work/), but I do not find any of the answers to be satisfactory.

Anyway, the final code is as follows:

An alternative is to replace the *JZ* and the *JMP* with a *JNZ*:

Finally, it is possible to consider the proposed algorithm as a subroutine to be called:

EXAMPLE 8.3 | § 8.2

Write a program that reads a data from input port 2*: if its LSB is 1 the program will load the ASCII of the character into the accumulator, otherwise it will load the ASCII of the character . This ASCII must then be sent to output port* 3.

To isolate the last bit it is possible to use the instruction *ANI* with the operand 0x01, so that all the bits of the input are set to zero except for the last one. At that point it will be sufficient to perform a conditional jump: with *JNZ* we will know that the last bit is 1, so we will move to the address that contains the instruction that loads the character *Y* (0x59); if the last bit is not 1 this instruction will be ignored, so we will load the character *N* (0x4*E*) and we will jump directly to the output.

SAP-3

The final step in the evolution towards modern computers includes three main features:

1. Four new registers (*D*, *E*, *H*, *L*) that are added to the three already existing (*A*, *B*, *C*). They, in addition to giving greater freedom in programming, make programs faster, because - as already explained in the context of the SAP-2 - the instructions that act on the registers without going through the memory occupy a smaller number of timing states.
2. The flags are indicated by a register *F*, and to the *zero* (*Z*) and *sign* (*S*) are added two more: the *carry* (*CY*), which acts as a carry or borrow (in other words, as the ninth bit of the accumulator), and the *parity* (*P*), active when the result of the ALU is composed of an even number of bits.
3. The *stack pointer*, that is a 16-bit register with the same role as the program counter dedicated to a specific area of memory called the *stack*, which we will talk about later.

9.1 Instructions

The SAP-3 inherits the entire instruction set of the SAP-2; between new registers and new instructions the total number goes from 42 to 244, although we will only see 223 of these.

They are detailed below, divided into appropriate categories, not necessarily coinciding with those of the SAP-2.

9.1.1 Data transfer instructions

All instructions are also present in the SAP-2; the only difference is that in *MOV*, for debugging reasons, the register from which the data is copied and the one in which it is passed can coincide.

9.1.2 Arithmetic instructions

The SAP-3 introduces the *carry flag*, indicated by *CY*, which allows greater freedom in arithmetic calculations by eliminating the possibility of overflow.

Two new instructions act exclusively on the carry flag:

- (): activates the carry flag.
- (): complements the carry flag.

To set the carry to 0 regardless of its current state, it is possible to first set the carry with *STC* and then complement it with *CMC*.

The sum and difference instructions (*ADD X* and *SUB X*) work in the same way as in the SAP-2, with the only difference that it is possible to include *A* as a register as well, which will thus add the content of the accumulator to itself.

PhyAMR

To these are added two other similar instructions:

- (): adds the content of the register X to the content of the accumulator, finally adding also the eventual carry.

$$\begin{cases} A = 1000\ 0011 \\ E = 0001\ 0100 \\ CY = 1 \end{cases} \xrightarrow{ADC\ E} \begin{cases} A = 1001\ 1000 \\ E = 0001\ 0100 \\ CY = 0 \end{cases}$$

- (): subtracts the content of the register X from the content of the accumulator, finally removing also the eventual carry.

$$\begin{cases} A = 1001\ 1000 \\ L = 1001\ 0110 \\ CY = 1 \end{cases} \xrightarrow{SBB\ L} \begin{cases} A = 0000\ 0001 \\ L = 1001\ 0110 \\ CY = 0 \end{cases}$$

Another novelty in the SAP-3 are the immediate sums and differences:

- (): adds the operand to the content of the accumulator.

$$A = 0000\ 0001 \xrightarrow{ADI\ 0x61} A = 0110\ 0010$$

- (): subtracts the operand from the content of the accumulator.

$$A = 0110\ 0010 \xrightarrow{SUI\ 0x21} A = 0100\ 0001$$

- (): adds the operand to the content of the accumulator, finally adding also the eventual carry.
  By activating the carry with *STC* we will have

$$A = 0100\ 0001 \xrightarrow{ACI\ 0x72} A = 1011\ 0100$$

- (): subtracts the operand from the content of the accumulator, finally removing also the eventual carry.
  By activating the carry with *STC* we will have

$$A = 1011\ 0100 \xrightarrow{SBI\ 0x11} A = 1010\ 0010$$

Finally, the increment and decrement instructions (*INR X* and *DCR X*) remain the same as in the SAP-2.

9.1.3 Rotation instructions

The SAP-3 also inherits the *RAL* and *RAR* instructions, with the difference that the rotations also include the carry. Let's see two examples of this:

$$\underset{\widetilde{CY}}{0}\ \overbrace{1000\ 0101}^{A} \xrightarrow{RAL} \underset{\widetilde{CY}}{1}\ \overbrace{0000\ 1010}^{A}$$

$$\underset{\widetilde{CY}}{1}\ \overbrace{0110\ 1000}^{A} \xrightarrow{RAR} \underset{\widetilde{CY}}{0}\ \overbrace{1011\ 0100}^{A}$$

We find the following results:

PhyAMR

$$RAL \longrightarrow \begin{cases} CY = MSB \\ LSB = CY \end{cases} \qquad RAR \longrightarrow \begin{cases} CY = LSB \\ MSB = CY \end{cases}$$

If the carry is 0 before the rotation, the latter can be seen as a *multiplication by two* (in the case of *RAL*) or as an *integer division by two* (in the case of *RAR*). The reason is quite simple: by shifting all the bits one position to the left, all the contributions of the formula [eq:arithmetic-conversion-binary-to-decimal] will shift from 2^i to 2^{i+1}, and this is equivalent to multiplying the entire summation, that is the number, by two; obviously the same will apply in the shift to the right, with the difference that the contributions will become 2^{i-1}, effectively dividing the number by two. It is important to underline that, for this to be valid, it is necessary that the carry flag is deactivated, since otherwise with the *RAL* instruction it would cause a contribution of 2^0 and with the *RAR* instruction a contribution of 2^7, both unwanted.

Two new instructions are the following:

- (): stores the *MSB* in the carry flag and rotates the bits of the accumulator to the left.

$$\underset{\widetilde{CY}}{0}\ \overbrace{1010\ 0010}^{A} \xrightarrow{RLC} \underset{\widetilde{CY}}{1}\ \overbrace{0100\ 0101}^{A}$$

$$RLC \longrightarrow \begin{cases} CY = MSB \\ LSB = MSB \end{cases}$$

- (): stores the *LSB* in the carry flag and rotates the bits of the accumulator to the right.

$$\underset{\widetilde{CY}}{1}\ \overbrace{0100\ 0101}^{A} \xrightarrow{RLC} \underset{\widetilde{CY}}{1}\ \overbrace{1010\ 0010}^{A}$$

$$RRC \longrightarrow \begin{cases} CY = LSB \\ MSB = LSB \end{cases}$$

Here are the instructions summarized below:

PhyAMR

9.1.4 Logical instructions

The only novelty is the *compare* instruction along with its immediate counterpart:

- : activates the *zero flag* if the content of the register X coincides with that of the accumulator; conversely, it deactivates the flag if the two contents are different.

$$CMP\ D \longrightarrow Z = \begin{cases} 0 & \text{if } A \neq D \\ 1 & \text{if } A = D \end{cases}$$

- : activates the *zero flag* if the indicated byte coincides with the content of the accumulator; conversely, it deactivates the flag if the two quantities are different.

$$CPI\ 0x8C \longrightarrow Z = \begin{cases} 0 & \text{if } A \neq 1000\ 1100 \\ 1 & \text{if } A = 1000\ 1100 \end{cases}$$

Considering the old instructions, the complete list will be

9.1.5 Jump instructions

- (): performs the jump only if the sign flag is not active.

$$PC = 0x0005 \xrightarrow{JP\ 0x0020} \begin{cases} PC = 0x0020 & \text{if } S = 0 \\ PC = 0x0005 & \text{if } S = 1 \end{cases}$$

- (): performs the jump only if the carry flag is active.

$$PC = 0x0005 \xrightarrow{JC\ 0x0020} \begin{cases} PC = 0x0005 & \text{if } C = 0 \\ PC = 0x0020 & \text{if } C = 1 \end{cases}$$

- (): performs the jump only if the carry flag is not active.

$$PC = 0x0005 \xrightarrow{JNC\ 0x0020} \begin{cases} PC = 0x0020 & \text{if } C = 0 \\ PC = 0x0005 & \text{if } C = 1 \end{cases}$$

- (): performs the jump only if the parity flag is active.

$$PC = 0x0005 \xrightarrow{JPE\ 0x0020} \begin{cases} PC = 0x0005 & \text{if } P = 0 \\ PC = 0x0020 & \text{if } P = 1 \end{cases}$$

- (): performs the jump only if the parity flag is not active.

$$PC = 0x0005 \xrightarrow{JPO\ 0x0020} \begin{cases} PC = 0x0020 & \text{if } P = 0 \\ PC = 0x0005 & \text{if } P = 1 \end{cases}$$

To these are added those of the SAP-2; however, we exclude the instructions *CALL* and *RET*, which here belong to another specific category (stack instructions).

9.1.6 Extended-register instructions

The SAP-3 has the ability to also process 16-bit data thanks to the **extended registers**. The six registers used to store bytes beyond the accumulator are in fact paired in the following way:

$$B + C \rightarrow \boldsymbol{BC}$$
$$D + E \rightarrow \boldsymbol{DE}$$
$$H + L \rightarrow \boldsymbol{HL}$$

In the computer there is therefore a set of instructions that treats these pairs of registers as if they were single 16-bit registers. Let's see these instructions below:[1]

- (): loads the indicated data into the extended register *X*.

[1] In the mnemonics to refer to the extended register, only the first letter of the register is used: we will therefore have *B* for *BC*, *D* for *DE* and *H* for *HL*.

PhyAMR

- (): adds the content of the extended register X to the content of HL.

$$\begin{cases} BC = 1011\ 0000\ 0100\ 1100 \\ HL = 0000\ 0100\ 0010\ 1001 \end{cases} \xrightarrow{DAD\ B} \begin{cases} BC = 1011\ 0000\ 0100\ 1100 \\ HL = 1011\ 0100\ 0111\ 1101 \end{cases}$$

- (): adds 0x0001 to the content of the extended register X.
- (): subtracts 0x0001 from the content of the extended register X.

In summary:

9.1.7 Indirect instructions

With the instruction *DAD X* we have seen that the sum is performed between the extended register X and HL, and the result is saved precisely in HL. In fact, this register has a peculiarity that distinguishes it from the other two: it does not contain a data in itself, but a *memory address* in which a data is saved. In this sense we will say that it does not contain a data but *points* to it, so just as the program counter is an *instruction pointer*, the register HL will be a .

Let's assume that the register HL contains 0x2024; if this register acts as a data pointer it means that it will point to the address R_{8228} and the read and write operations will refer to it. This address, in general, is indicated by M, and all the operations referred to it are called **indirect** since they do not concern it explicitly but refer to the pointer HL which acts as an intermediary for them.

9.1.8 Stack instructions

We had already introduced subroutines in the SAP-2 when talking about the *CALL* and *RET* instructions: they needed the last two memory addresses to save the point where the subroutine was called, so that with the return it was possible to return to the main program.

In the SAP-3 this concept is taken up and extended: an entire portion of memory[2] - whose size is at the user's discretion - is dedicated to the call and return addresses and to the *status* of the program. The pointer to the addresses of this memory is called the (indicated by SP) and is nothing more than a 16-bit register analogous to the program counter.

Let's see the first instruction useful for initializing the stack register:

- : loads the indicated memory address into the stack register.

In this context, the stack counter is treated in all respects as an extended register, so the instructions *DAD*, *INX* and *DCX* will also be possible.

As mentioned, in the stack memory it is also possible to save the *status* of the program, that is the contents inside the registers so that they can be overwritten in the execution of the subroutines without being lost. Just as the memory addresses are saved in two addresses[3] the stack instructions are made in such a way that with each of them the information is stored in two addresses: for this reason the extended registers are used, which contain precisely two bytes of information that can be stored in two addresses.

[2] Not necessarily located in the last memory addresses.

[3] We remember that a memory address is encoded by 2 bytes, and in it only one byte can be contained; in the SAP-2, in fact, two addresses were used to save the current point.

PhyAMR

We know that there are three extended registers, namely *BC*, *DE* and *HL*. In this context, however, a fourth extended register is added, called **PSW** (*Program Status Word*), which contains the accumulator *A* and the flag register *F*; we remember, in fact, that although the flags in the SAP-3 are four, the register *F* is still an 8-bit register,[4] so in total *PSW* is actually made up of two bytes.

Let's now see the two instructions that allow you to work with the program status:

- : saves the content of the extended register *X* in the stack memory.
  In order, the operations performed are the following:

  1. The stack pointer is decremented, going from *SP* to *SP-1*
  2. The most significant byte is stored in the stack memory
  3. The stack pointer is decremented, going from *SP-1* to *SP-2*
  4. The least significant byte is stored in the stack memory

  Assuming that the initialization of the stack pointer was given by *LXI SP* 0x2000, we will have

$$BC = 0xA45B \xrightarrow{PUSH\ B} \begin{cases} R_{8191} = 1010\ 0100 \\ R_{8190} = 0101\ 1011 \end{cases}$$

  being $8191_{10} = 0x1FFF$ and $8190_{10} = 0x1FFE$.
- : copies the content from the stack memory into the extended register *X*.
  In order, the operations performed are the following:

  1. the content in the address indicated by *SP* is copied as the least significant byte into the extended register *X*
  2. The stack pointer is incremented, going from *SP* to *SP+1*
  3. the content in the address indicated by *SP+1* is copied as the most significant byte into the extended register *X*
  4. The stack pointer is incremented, going from *SP+1* to *SP+2*

$$\begin{cases} R_{8190} = 0101\ 1011 \\ R_{8191} = 1010\ 0100 \end{cases} \xrightarrow{POP\ B} BC = 0xA45B$$

The instructions *PUSH* and *POP*, therefore, are the inverse of each other. Let's imagine the memory as a of addresses that are numbered from top to bottom: with the *PUSH* the stack pointer copies the data into memory (first the byte on the left, then the one on the right) going up; with the *POP* the stack pointer - going down - the data in the indicated register, copying first the byte on the right and then the one on the left.

In light of what has been said, it is easy to understand that the instructions *CALL* and *RET* in the SAP-3 have the role of *PUSH* and *POP* with respect to the program counter. The novelty is that of these instructions there are also the *conditional* versions, as for the jump instructions: we will therefore have *CM* () and *CP* () as well as *RC* () and *RP* () and so on.

[4] The structure of this register depends on the specific model of SAP-3. In the 8085 microprocessor - one of the most widespread - it is constituted as follows: *SZ*00 0*P*0*CY*. Beyond the specific position of the flags, the other four bits are always set to zero by default.

PhyAMR

9.1.9 Overall table

Two clarifications:

1. The logical instructions *ANA*, *ORA*, *XRA* and their immediate counterparts, according to Malvino, always reset the carry to zero, thus obtaining $CY \leftarrow 0$; this fact is not specified by the professor.
2. Another thing not specified by the professor concerns the flags related to the instruction *CMP* and its immediate counterpart *CPI*: according to what we have said they act only on the zero flag, but the table of instructions of the SAP-3 of the book suggests that all the flags are involved; in particular, it is made explicit that $CY \leftarrow 1$ when $A < X$ (or $A < dd$ in the case of *CMI*). Substantially, according to the table, the *compare* would be nothing more than a *subtract* with the only difference that the result of the subtraction would not be stored in the accumulator.

EXAMPLE 9.1 | § 9.1

Write a program that adds 1200_{10} *and* 3400_{10} *and saves the result in the registers* H* and *L*, both with and without the use of extended register instructions.*

First, let's convert the two numbers to hexadecimal with the hex-dabble algorithm:

$$1200//16 = 75 \textit{ extwithremainder } 0$$
$$75//16 = 4 \textit{ extwithremainder } 11$$
$$4//16 = 0 \textit{ extwithremainder } 4$$

$$1200_{10} = 0x04B0$$

$$3400//16 = 212 \textit{ extwithremainder } 8$$
$$212//16 = 13 \textit{ extwithremainder } 4$$
$$13//16 = 0 \textit{ extwithremainder } 13$$

$$3400_{10} = 0x0D48$$

Let's proceed first without the use of extended registers. In this case the registers are only able to store one byte, so we will first add the last two hexadecimal digits of the numbers in question, that is 0x*B*0 and 0x48. This sum will be stored in the register *L* which must contain the least significant byte.

Next, we will add the other two bytes, that is 0x04 and 0x0*D*, but also considering the eventual carry from the previous sum. This result will instead be saved in *H*, and at that point it will be possible to stop the program.

With the extended registers the program is greatly simplified: it will be enough to load 0x04*B*0 on an extended register and then put the other number 0x0*D*48 in *HL*; at that point, the *DAD* instruction will be sufficient to give us the desired result.

EXAMPLE 9.2 | § 9.1

Using pointers, write a program that moves all the data contained between the memory addresses 0*x*2030 *and* 0*x*212*F* *to the memory addresses between* 0*x*5030 *and* 0*x*512*F*.

We understand right away that we need a loop that takes us from the address 0x2030 to the address 0x212*F*, and in that same loop we must be able to also move between the new addresses in which to copy the data.

The steps to follow will be:

1. We increment the register *H*, which points to the memory address from which to retrieve the data
2. We copy the data into any register, for example the register *B*, and we copy the most significant byte of the address (that is 0x20) into the accumulator *A*
3. The difference between the old address and the new one is exactly 0x30 (always referring to the most significant byte), so we add this quantity to the accumulator
4. We proceed in the opposite direction by copying the content of the accumulator into *H* and that of *B* into the new address to which *H* points
5. To return to the addresses from which to retrieve the data, we subtract 0x30 from the accumulator and copy the result into *H*
6. Now it's time to figure out whether to proceed further or stop the cycle, so we copy the content of *L* into the accumulator and perform a comparison with 0x2*F*
7. If the difference is null then the address was the last one and we can stop the program, otherwise we will have to continue, so a *JNZ* associated with the address in which *H* is incremented at the first step will be enough to repeat the whole process

PhyAM

Exercises

Past-exam exercises for *Management and Analysis of Physics Datasets — Module A*, grouped chronologically by exam date.

Each item is presented as a self-contained problem (the question text is *italic*) followed by a collapsible solution block: previously worked solutions are pre-filled, the new exam papers carry empty `Solution` placeholders ready to be filled in.

10.1 Exam — 22 November 2022

EXERCISE 10.1 · *Worked exercise 1 — 22/11/22* | § 10.1

Find the truth table of the flip-flop described by the following equation:

$$Q^{n+1} = \left[\bar{Q} \oplus (M \oplus N)\right]^n$$

Also, realize this flip-flop starting from a J-K flip-flop.

SOLUTION | EX. 10.1

Truth table:

Let's make the first XOR explicit:

$$Q^{n+1} = \left[\bar{Q}\,(M \bar{\oplus} N) + Q(M \oplus N)\right]^n$$

In the form of equation [eq:circuiti-sequenziali-flip-flop-jk] we will have

$$\begin{cases} J = M \bar{\oplus} N \\ \bar{K} = M \oplus N \end{cases} \Longrightarrow \begin{cases} J = M \bar{\oplus} N \\ K = M \bar{\oplus} N \end{cases}$$

We find the following circuit:

EXERCISE 10.2 · *Worked exercise 2 — 22/11/22* | § 10.1

Realize a decimal counter that counts every two (0, 2, 4, 6, 8, 0, 2, 4, ...) in binary.

SOLUTION | EX. 10.2

The bit A^n is always equal to 0 regardless of the state assumed by the counter, so we can not consider it for the design of the circuit.

From which we obtain the following equations:

$$B^{n+1} = \left[\bar{B}\bar{D}\right]^n$$
$$C^{n+1} = \left[B\bar{C} + \bar{B}C\right]^n = [B \oplus C]^n$$
$$D^{n+1} = [BC]^n$$

EXERCISE 10.3 · *Worked exercise 3 — 22/11/22* | § 10.1

Realize a device capable of detecting the sequence

... 1100 ...

on a data line.

SOLUTION | EX. 10.3

From the diagram we build the truth table:

So we find:

$$Z = C$$

For the other variables we build the Karnaugh maps:

From which we find

$$A^{n+1} = \left[BM + \bar{A}B + \bar{A}M + A\bar{B}\bar{M}\right]^n$$
$$B^{n+1} = \left[A\bar{B}M + \bar{A}B\bar{M}\right]^n$$
$$C^{n+1} = \left[AB\bar{M}\right]^n$$

EXERCISE 10.4 · *Worked exercise 4 — 22/11/22* | § 10.1

Using two's complement notation and eleven bits, write the decimal number −765 *in binary.*

SOLUTION | EX. 10.4

Let's use the hex-dabble algorithm to write the number 765 in hexadecimal:

$$765//16 = 47 \textit{ extwithremainder } 13$$
$$47//16 = 2 \textit{ extwithremainder } 15$$
$$2//16 = 0 \textit{ extwithremainder } 2$$

We find the following expression:

$$0x2FD = 010\ 1111\ 1101$$

Let's perform the two's complement to express the same number in negative form:

$$-765_{10} = 101\ 0000\ 0010 + 1 = 101\ 0000\ 0011$$

EXERCISE 10.5 · *Worked exercise 5 — 22/11/22* | § 10.1

Using four bits for the fractional part, write the decimal number 67.625 *in fixed point representation.*

SOLUTION | EX. 10.5

The integer part is trivial to obtain, being $67 = 64 + 3$:

$$67_{10} = 100\ 0000 + 000\ 0011 = 100\ 0011$$

For the decimal part we use the algorithm seen in the theory:

$$\begin{aligned} 0.625 \cdot 2 &= 1.250 \ \textit{extwhichhasintegerpart}\ 1 \\ 0.250 \cdot 2 &= 0.5 \ \textit{extwhichhasintegerpart}\ 0 \\ 0.5 \cdot 2 &= 1 \ \textit{extwhichhasintegerpart}\ 1 \\ 0 \cdot 2 &= 0 \ \textit{extwhichhasintegerpart}\ 0 \end{aligned}$$

So we find:

$$67.625_{10} = 100\ 0011.1010$$

EXERCISE 10.6 · *Worked exercise 6 — 22/11/22* | § 10.1

Write a program for SAP-1 that executes and finally shows the result of the arithmetic operation $220 + 15 - 32$.

Let's perform the hex-dabble algorithm for the three numbers:

$$\begin{cases} 220//16 = 13 & \textit{extwithremainder}\ 12 \\ 13//16 = 0 & \textit{extwithremainder}\ 13 \end{cases} \implies 220_{10} = 0\text{x}DC$$

$$15//16 = 0 \text{ with remainder } 15 \implies 15_{10} = 0\text{x}0F$$

$$\begin{cases} 32//16 = 2 & \textit{extwithremainder}\ 0 \\ 2//16 = 0 & \textit{extwithremainder}\ 2 \end{cases} \implies 32_{10} = 0\text{x}20$$

At this point we can report the code:

SOLUTION | EX. 10.6

(to fill in)

EXERCISE 10.7 · *Worked exercise 7 — 22/11/22* | § 10.1

What does the following program for SAP-2 do?

SOLUTION | EX. 10.7

I premise that the halt address should be wrong, as *OUT* occupies two bytes and therefore the address of *HLT* should be 0x080*B*; it almost seems that the professor has confused the hexadecimal digits with decimal digits.

Apart from that, the program starts by initializing the three registers:

$$A = 0000\ 0000$$
$$B = 0001\ 0000$$
$$C = 0000\ 1110$$

At this point the subroutine is called which proceeds in the following way:

1. Adds *B* to *A*; since the latter is zero, it will in fact add *B* to itself
2. Decrements *C*
3. If the decrement has not activated the zero flag the program restarts from the first point

Since the decrement will activate the zero flag only after $0x0E = 14_{10}$ iterations this means that in the accumulator $0x10 = 16_{10}$ will have been added exactly fourteen times. This result is then shown in output from port 3.

In essence, the program performs the product $16 \cdot 14$ and sends the result to the output.

EXERCISE 10.8 · *Worked exercise 8 — 22/11/22* | § 10.1

Write a program for SAP-3 that performs the division by two of a number loaded into the accumulator using the minimum possible number of clock cycles. My idea is to reset the carry to 0 and perform a rotation of the bits to the right:

SOLUTION | EX. 10.8

(to fill in)

10.1.1 From the exam paper

EXERCISE 10.9 · *Q1 — 2 pts* | § 10.1

Without the use of graphical representation, demonstrate that □ + □□ = □ + □

SOLUTION | EX. 10.9

(to fill in)

EXERCISE 10.10 · *Q2 — 2 pts* | § 10.1

Given □(□, □) = □ □ □ *demonstrate that* □(□, □) = □(□, □) = □(□, □)

SOLUTION | EX. 10.10

(to fill in)

EXERCISE 10.11 · *Q3 — 2 pts* | § 10.1

*Simplify as much as possible the function □□□□+□□□□+□□□□+

□□□□+□□□□+□□□□+□□□□+□□□□+□□□□+□□□□ using the graphical simplification method*

SOLUTION | EX. 10.11

(to fill in)

EXERCISE 10.12 · *Q4 — 2 pts* | § 10.1

*Find the truth table of the flip-flop with the following equation

□□+1 = [□□(□□□)]

□*

SOLUTION | EX. 10.12

(to fill in)

EXERCISE 10.13 · *Q5 — 2 pts* | § 10.1

Make the above flip-flop from a J-K flip-flop

SOLUTION | EX. 10.13

(to fill in)

EXERCISE 10.14 · *Q6 — 4 pts* | § 10.1

*Design (draw the circuit of) a decimal counter that counts by two (0, 2, 4, 6, 8, 0, 2,

4...) in binary*

SOLUTION | EX. 10.14

(to fill in)

EXERCISE 10.15 · *Q7 — 4 pts* | § 10.1

*Design (draw the Moore diagram and the circuit of) a device able to find the

sequence ...1100... on a data line*

SOLUTION | EX. 10.15

(to fill in)

EXERCISE 10.16 · *Q8 — 2 pts* | § 10.1

Using 2's complement notation and 11 bits, write in binary the decimal number -765

SOLUTION | EX. 10.16

(to fill in)

EXERCISE 10.17 · *Q9 — 2 pts* | § 10.1

Using 4 bits for the fractional part, write in fixed point representation the decimal number 67.625

SOLUTION | EX. 10.17

(to fill in)

EXERCISE 10.18 · *Q10 — 3 pts* | § 10.1

Write a program for SAP-1 that executes and displays the result of the arithmetic operation 220+15-32

SOLUTION | EX. 10.18

(to fill in)

EXERCISE 10.19 · *Q11 — 3 pts* | § 10.1

*What does the following program for SAP-2 do?

Address 0x800 0x802 0x804 0x806 0x809 0x811 0x1180 0x1181 0x1182 0x1185

Mnemonics MVI A 0x00 MVI B 0x10 MVI C 0x0E CALL 0x1180 OUT 0x03 HLT ADD B DCR C JNZ 0x1180 RET*

SOLUTION | EX. 10.19

(to fill in)

EXERCISE 10.20 · *Q12 — 4 pts* | § 10.1

Write a program for SAP-3 that executes the division by 2 of a number loaded into the accumulator using as few clock cycles as possible

SOLUTION | EX. 10.20

(to fill in)

10.2 Exam — 3 February 2023

EXERCISE 10.21 · *Worked exercise 1 — 03/02/23* | § 10.2

Realize a 3-bit counter that counts towards larger numbers when a control line is set to 0 and towards smaller numbers when it is set to 1.

SOLUTION | EX. 10.21

Similarly to the decimal counter (paragraph [sec:decimal-counter]) I will consider the smallest number (encoded by 000) as immediately following the largest number (encoded by 111), and in the same way the latter as immediately preceding the first.

$$A^{n+1} = \bar{A}^n$$
$$B^{n+1} = \left[\bar{A}B\bar{S} + ABS + \bar{A}\bar{B}S + A\bar{B}\bar{S}\right]^n$$
$$C^{n+1} = \left[\bar{A}BC + ACS + \bar{B}C\bar{S} + AB\bar{C}\bar{S} + \bar{A}\bar{B}\bar{C}S\right]^n$$

EXERCISE 10.22 · *Worked exercise 2 — 03/02/23* | § 10.2

Using bit-masking operations, invert the seventh bit (starting from the most significant one) of a 32-bit register R_0.

SOLUTION | EX. 10.22

To invert a bit using bit-masking it is necessary to perform a *XOR* with a mask composed only of 0s except for a 1 in the position of the bit. Since this is in the seventh position from the left, it will be located in the second hexadecimal digit of the mask:

$$0010 = 0\text{x}2$$

Since there are 32 bits, the hexadecimal digits will be a total of 8, all equal to 0. The operation, therefore, will be the following:

$$R_0 \; XOR \; 0\text{x}02000000$$

EXERCISE 10.23 · *Worked exercise 3 — 03/02/23* | § 10.2

Write a program for SAP-1 that executes and finally shows the result of the arithmetic operation $23 - 126 + 78$.

$$\begin{cases} 23//16 = 1 & extwithremainder\ 7 \\ 1//16 = 0 & extwithremainder\ 1 \end{cases} \implies 23_{10} = 0\text{x}17$$

$$\begin{cases} 126//16 = 7 & extwithremainder\ 14 \\ 7//16 = 0 & extwithremainder\ 7 \end{cases} \implies 126_{10} = 0x7E$$

$$\begin{cases} 78//16 = 4 & extwithremainder\ 14 \\ 4//16 = 0 & extwithremainder\ 4 \end{cases} \implies 78_{10} = 0x4D$$

SOLUTION | EX. 10.23

(to fill in)

EXERCISE 10.24 · *Worked exercise 4 — 03/02/23* | § 10.2

What operation does the following program for SAP-2 perform?

Suggest an alternative program that performs the same operation.

SOLUTION | EX. 10.24

The program loads into the accumulator the content of the register R_{0x2100}, performs the two's complement of the number and loads the result into the next address R_{0x2101}.

An alternative way to perform the same operation is to load the number 0 into the accumulator and then subtract the content of the register from it, obtaining the same number with the opposite sign, which is precisely the result of the two's complement.

EXERCISE 10.25 · *Worked exercise 5 — 03/02/23* | § 10.2

Write a program for SAP-3 that checks if a signed number loaded into the accumulator is positive or negative using the minimum possible number of clock cycles. The sign of a number in the accumulator is determined by its most significant bit, so we can extract this bit through an *AND* with the byte 0x80 = 1000 0000 and send the result to the output: if the number was positive the bit was 0 and we will get 0x00, if instead the number was negative the bit was 1 and we will get 0x80.

SOLUTION | EX. 10.25

(to fill in)

10.2.1 From the exam paper

EXERCISE 10.26 · *Q1 — 2 pts* | § 10.2

*Without the use of graphical representation, demonstrate that □□ + □□ + □□ =

□□ + □□ + □□**

SOLUTION | EX. 10.26

(to fill in)

EXERCISE 10.27 · *Q2 — 2 pts* | § 10.2

How many NAND gate are needed for implementing a XNOR (□□□) gate? Draw the circuit

SOLUTION | EX. 10.27

(to fill in)

EXERCISE 10.28 · *Q3 — 2 pts* | § 10.2

Simplify as much as possible the function □□□□+□□□□+□□□□+□□□□+□□□□+□□□□+□□□□+□□□□ using the graphical simplification method

SOLUTION | EX. 10.28

(to fill in)

EXERCISE 10.29 · *Q4 — 2 pts* | § 10.2

*Find the truth table of the flip-flop with the following equation

□□+1 = [□□(□+□)]□*

SOLUTION | EX. 10.29

(to fill in)

EXERCISE 10.30 · *Q5 — 2 pts* | § 10.2

Make the above flip-flop from a J-K flip-flop

SOLUTION | EX. 10.30

(to fill in)

EXERCISE 10.31 · *Q6 — 4 pts* | § 10.2

Design (draw the circuit of) a 3-bit counter that counts upwards when a control line is set to zero and downwards when it is set to one

SOLUTION | EX. 10.31

(to fill in)

EXERCISE 10.32 · *Q7 — 4 pts* | § 10.2

Design (draw the Moore diagram and the circuit of) a controller of an elevator. The elevator can be at one of the three floors: ground, first and second. For moving the elevator there is one 2-bit selector switch that can assume the values 00,

01, 10 corresponding to the three floors and 11 that means to go down by one floor (if already at ground it remains at ground)

SOLUTION | EX. 10.32

(to fill in)

EXERCISE 10.33 · *Q8 — 2 pts* | § 10.2

Using bit-masking operations invert the seventh bit (starting from the MSB) of the 32-bit register □0

SOLUTION | EX. 10.33

(to fill in)

EXERCISE 10.34 · *Q9 — 2 pts* | § 10.2

Find the decimal value of the floating point (IEEE 754) number 0x41160000

SOLUTION | EX. 10.34

(to fill in)

EXERCISE 10.35 · *Q10 — 3 pts* | § 10.2

Write a program for SAP-1 that executes and displays the result of the arithmetic operation 23-126+78

SOLUTION | EX. 10.35

(to fill in)

EXERCISE 10.36 · *Q11 — 3 pts* | § 10.2

What operation does the following program for SAP-2 do? Suggest an alternative program for performing the same operation Address 0x800 0x803 0x804 0x805 0x808 Mnemonics LDA 0x2100 CMA INR A STA 0x2101 HLT

SOLUTION | EX. 10.36

(to fill in)

EXERCISE 10.37 · *Q12 — 4 pts* | § 10.2

Write a program for SAP-3 that checks if a signed number loaded into the accumulator is positive or negative using as few clock cycles as possible

SOLUTION | EX. 10.37

(to fill in)

10.3 Exam — 24 February 2023

EXERCISE 10.38 · *Worked exercise 1 — 24/02/23* | § 10.3

Find the truth table of the flip-flop described by the following equation:

$$D^{n+1} = [D \oplus (MN)]^n$$

Also, realize this flip-flop starting from a D flip-flop.

SOLUTION | EX. 10.38

Let's make the XOR explicit:

$$D^{n+1} = \left[D\left(\bar{M}N\right) + \bar{D}(MN)\right]^n$$

In this form it is simple to write everything in the form of a J-K flip-flop:

$$\begin{cases} J = MN \\ \bar{K} = \bar{M}N \end{cases} \implies J = K = MN$$

We have already seen in the first exercise related to the D flip-flop (paragraph [sec:flip-flop-d]) the construction of a J-K flip-flop through a D flip-flop, so we will just have to replicate the same scheme where at the inputs J and K is placed MN:

EXERCISE 10.39 · *Worked exercise 2 — 24/02/23* | § 10.3

Using bit-masking operations, check that the third bit (starting from the most significant) of a 32-bit register R_0 is set.

SOLUTION | EX. 10.39

The third bit from the left defines the first hexadecimal digit as follows:

$$0010 = 0x2$$

All the other seven nibbles to the right will be zero. For the check, an *AND* with this mask is sufficient:

$$R_0 \; AND \; 0x20000000$$

If the result is 0 the bit is 0, if instead the mask is obtained again the bit is 1.

EXERCISE 10.40 · *Worked exercise 3 — 24/02/23* | § 10.3

Find the decimal value of the following floating points (according to the IEEE 754 standard):

SOLUTION | EX. 10.40

$$\texttt{0x40600000}$$

$$\texttt{0x41}D\texttt{00000}$$

Let's start with the first one:

$$\texttt{0x40600000} = \texttt{0100 0000 0110 0000 0000 0000 0000 0000}$$

Let's calculate E and M:

$$E = 2^7 = 128 M = 2^{-1} + 2^{-2} = 0.5 + 0.25 = 0.75$$

The sign is given by 0, so the number is positive and equal to

$$1.75 \cdot 2^{128-127} = 1.75 \cdot 2 = 3.5$$

Let's do the same calculations for the second number:

$$\texttt{0x41}D\texttt{00000} = \texttt{0100 0001 1101 0000 0000 0000 0000 0000}$$

$$E = 2^7 + 2^1 + 2^0 = 128 + 2 + 1 = 131 M = 2^{-1} + 2^{-3} = 0.5 + 0.125 = 0.625$$

$$1.625 \cdot 2^{131-127} = 1.75 \cdot 2^4 = 1.75 \cdot 16 = 28$$

EXERCISE 10.41 · *Worked exercise 4 — 24/02/23* | § 10.3

If the clock frequency of a SAP-1 is 5 *MHz, how long does it take to execute a program that calculates* 23 − 126 + 78 *and shows the result?*

SOLUTION | EX. 10.41

The program is the same as the previous exercise, so it will be enough to refer to the code above.

Let's calculate the time of each timing state:

$$T = \frac{1}{5 \cdot 10^6\ Hz} = 200\ ns$$

We remember that all the instructions of the SAP-1 require 6 timing states with the exception of the HLT which requires 4, so in total we will have:

$$6 \cdot 4 + 4 = 28$$

From which we find the total time:

$$T_{tot} = 28 \cdot 200\ ns = 5600\ ns = 5.6\ \mu s$$

EXERCISE 10.42 · *Worked exercise 5 — 24/02/23* | § 10.3

What check does the following program for SAP-3 perform? The program loads 0x*FF* into the register *B*:

$$B = 1111\ 1111$$

The content of the register R_{0x2800} is then loaded into the accumulator; let's make this number explicit, indicated with the carry, with the most significant bit and with the least significant bit:

$$A = R_{0x2800} = C\ \ MXXX\ XXXL$$

A rotation of the bits to the right is followed:

$$A = L\ \ CMXX\ XXXX$$

- If $L = 0$ the content of *B* is loaded into *A* and this will in turn be saved in the register R_{0x2810}:

$$R_{0x2810} = 1111\ 1111$$

- If $L = 1$ the byte 0x00 is loaded into *B*, which then passes to the accumulator and finally to the register R_{0x2810}:

$$R_{0x2810} = 0000\ 0000$$

In summary: the program saves in R_{0x2810} a byte composed only of 1s if the least significant bit in R_{0x2800} is 0, while if the latter is 1 a byte composed only of 0s is saved in the same address.

SOLUTION | EX. 10.42

(to fill in)

EXERCISE 10.43 · *Worked exercise 6 — 24/02/23* | § 10.3

Write a program for SAP-3 that finds the absolute value of a given number. My idea is the following: if we load the number 0 into the accumulator and then add the number that interests us, the sign flag will be activated. This will allow us to perform conditional jumps: if it is positive, through *JP*, we will go directly to the halt since the number is already in absolute value; if instead it is negative we will perform the two's complement, and therefore we will complement the number and add 1. If this number is contained in a register *X* it is possible to replace *ADI* with *ADD X*; if instead the number is already contained in the accumulator to activate the flag it is possible to simply execute *ADD* 0x00.

SOLUTION | EX. 10.43

(to fill in)

10.3.1 From the exam paper

EXERCISE 10.44 · *Q1 — 2 pts* | § 10.3

Using algebraic manipulation, prove that □ + □□ + □□ = □ + □ + □

SOLUTION | EX. 10.44

(to fill in)

EXERCISE 10.45 · *Q2 — 2 pts* | § 10.3

Using logic ports, design a 5-bit even-parity generator

SOLUTION | EX. 10.45

(to fill in)

EXERCISE 10.46 · *Q3 — 2 pts* | § 10.3

Using the graphical representation method, prove that □□□ = □ + □ + □

SOLUTION | EX. 10.46

(to fill in)

EXERCISE 10.47 · *Q4 — 2 pts* | § 10.3

*Find the truth table of the flip-flop with the following equation

□□+1 = [□ □ (□ □)]□*

SOLUTION | EX. 10.47

(to fill in)

EXERCISE 10.48 · *Q5 — 2 pts* | § 10.3

Make the above flip-flop from a D flip-flop

SOLUTION | EX. 10.48

(to fill in)

EXERCISE 10.49 · *Q6 — 4 pts* | § 10.3

Design (draw the circuit of) a counter that counts from zero to eleven and then it stops (i.e., it remains at eleven)

SOLUTION | EX. 10.49

(to fill in)

EXERCISE 10.50 · *Q7 — 4 pts* | § 10.3

Design (draw the Moore diagram and the circuit of) a state machine able to detect the transition (0,1) -> (1,1) -> (1,0) of two bits (M,N)

SOLUTION | EX. 10.50

(to fill in)

EXERCISE 10.51 · *Q8 — 2 pts* | § 10.3

Using bit-masking operations check if the third bit (starting from the MSB) of the 32-bit register □0 is set

SOLUTION | EX. 10.51

(to fill in)

EXERCISE 10.52 · *Q9 — 2 pts* | § 10.3

Find the decimal value of the sum of two floating point (IEEE 754) numbers: 0x40600000 and 0x41D00000

SOLUTION | EX. 10.52

(to fill in)

EXERCISE 10.53 · *Q10 — 3 pts* | § 10.3

If the clock frequency of SAP-1 is 5 MHz, how long does it take to run a program that calculates 23-126+78 and display the result?

SOLUTION | EX. 10.53

(to fill in)

EXERCISE 10.54 · *Q11 — 3 pts* | § 10.3

*What check does the following program for SAP-3 do?

Address 0x800 0x802 0x805 0x806 0x809 0x80B 0x80C 0x80F

Mnemonics MVI B 0xFF LDA 0x2800 RAR JNC 0x080B MVI B 0x00 MOV A B STA 0x2810 HLT*

SOLUTION | EX. 10.54

(to fill in)

EXERCISE 10.55 · *Q12 — 4 pts* | § 10.3

Write a program for SAP-3 that find the absolute value of a given number

SOLUTION | EX. 10.55

(to fill in)

10.4 Exam — 23 March 2023

EXERCISE 10.56 · *Worked exercise 1 — 23/03/23* | § 10.4

Using bit-masking operations, clear the ninth bit (starting from the most significant) of a 32-bit register R_0.

SOLUTION | EX. 10.56

To means to reset to 0. To do this, just perform an *AND* with a mask consisting only of 1s except for a 0 in the position concerned:

$$R_0 \; AND \; 0xFF7FFFFF$$

EXERCISE 10.57 · *Worked exercise 2 — 23/03/23* | § 10.4

Using eight bits for the fractional part, write in fixed point representation the decimal number 34.2349.

SOLUTION | EX. 10.57

The integer part will be given by

$$34 = 32 + 2 = 2^5 + 2^1 = 100010$$

Let's calculate the fractional part:

$$0.2349 \cdot 2 = 0.4698 \; extwhichhasintegerpart \; 0$$

$$0.4698 \cdot 2 = 0.9396 \; extwhichhasintegerpart \; 0$$

$$0.9396 \cdot 2 = 1.8792 \; extwhichhasintegerpart \; 1$$

$$0.8792 \cdot 2 = 1.7584 \; extwhichhasintegerpart \; 1$$

$$0.7584 \cdot 2 = 1.5168 \; extwhichhasintegerpart \; 1$$

$$0.5168 \cdot 2 = 1.0336 \; extwhichhasintegerpart \; 1$$

$$0.0336 \cdot 2 = 0.0672 \; extwhichhasintegerpart \; 0$$

$$0.0672 \cdot 2 = 0.1344 \; extwhichhasintegerpart \; 0$$

It is not clear to me whether it is intentional or not, but with eight bits we only get an approximation of the requested number:

$$34.2349_{10} \simeq 100010.00111100$$

EXERCISE 10.58 · *Worked exercise 3 — 23/03/23* | § 10.4

If the clock frequency of a SAP-1 is 2 MHz, how long does it take to execute a program that multiplies a number by two and shows the result?

SOLUTION | EX. 10.58

To multiply a number by two, just add it to itself.

Time of each timing state:

$$T = \frac{1}{2 \cdot 10^6 \; Hz} = 500 \; ns$$

Total number of timing states:

$$6 \cdot 3 + 4 = 22$$

Total time:

$$T_{tot} = 22 \cdot 500 = 11\,000 \; ns = 11 \; \mu s$$

EXERCISE 10.59 · *Worked exercise 4 — 23/03/23* | § 10.4

Write a program for SAP-2 that swaps the two nibbles of the data contained in the address 0$ 2001$ using the minimum possible number of clock cycles. I can't think of any faster programs than the one that, trivially, rotates (left or right as it may be) all the bits four times. Obviously it is possible to replace *RAR* with *RAL* obtaining the same result.

SOLUTION | EX. 10.59

(to fill in)

EXERCISE 10.60 · *Worked exercise 5 — 23/03/23* | § 10.4

What does the following program for SAP-3 do? First, the extended register *HL* is loaded with the data 0x2501:

$$HL = 0x2501$$

The content of the memory indicated by *HL* is copied into *C* and *A*:

$$A = C = R_{0x2501}$$

The extended register *HL* is incremented, thus pointing to the next memory address; the content of the latter is subtracted from the content in the accumulator, which is the same content as the previous address:

$$A = R_{0x2501} - R_{0x2502}$$

The sign flag is therefore activated as follows:

$$S = \begin{cases} 0 \text{ if } R_{0x2501} \geqslant R_{0x2502} \\ 1 \text{ if } R_{0x2501} < R_{0x2502} \end{cases}$$

The content of R_{0x2501} is copied again into the accumulator:

$$A = R_{0x2501}$$

If the sign flag is null, the content of the accumulator, and therefore of R_{0x2501}, is saved in the address R_{0x2503}; if instead the sign flag is set, the content of the address to which the data pointer points, which is always R_{0x2502}, is copied into *A*, and it is this that is saved in R_{0x2503}.

In summary, the program saves in the address R_{0x2503} the largest content between that contained in the two previous addresses.

SOLUTION | EX. 10.60

(to fill in)

10.4.1 From the exam paper

EXERCISE 10.61 · *Q1 — 2 pts* | § 10.4

*Using algebraic manipulation, prove that □ □ + □ □ + □ □ + □ □ + □□ = □ □ + □ □ + □□**

SOLUTION | EX. 10.61

(to fill in)

EXERCISE 10.62 · *Q2 — 2 pts* | § 10.4

Using 2-bit multiplexers, design a 5-bit multiplexer with as less select lines as possible

SOLUTION | EX. 10.62

(to fill in)

EXERCISE 10.63 · *Q3 — 2 pts* | § 10.4

Find the minterms of the following expression □ □ + □ □ + □ □ □

SOLUTION | EX. 10.63

(to fill in)

EXERCISE 10.64 · *Q4 — 2 pts* | § 10.4

The output □ of a toggle flip-flop inverts from its previous state (toggles) when its input □ is 1, otherwise it remains unchanged. Write the truth table of a toggle flip- flop

SOLUTION | EX. 10.64

(to fill in)

EXERCISE 10.65 · *Q5 — 2 pts* | § 10.4

Make the toggle flip-flop from a J-K flip-flop and from a D flip-flop

SOLUTION | EX. 10.65

(to fill in)

EXERCISE 10.66 · *Q6 — 4 pts* | § 10.4

Design (draw the circuit of) a decimal counter that counts from -4 to 4 and loopback using the 2's complement for the representation of negative numbers

SOLUTION | EX. 10.66

(to fill in)

EXERCISE 10.67 · *Q7 — 4 pts* | § 10.4

Design (draw the Moore diagram and the circuit of) a state machine able to detect the sequence ...000111... on a data line □

SOLUTION | EX. 10.67

(to fill in)

EXERCISE 10.68 · *Q8 — 2 pts* | § 10.4

Using bit-masking operations clear the ninth bit (starting from the MSB) of the 32-bit register □0

SOLUTION | EX. 10.68

(to fill in)

EXERCISE 10.69 · *Q9 — 2 pts* | § 10.4

Using 8 bits for the fractional part, write in fixed point representation the decimal number 34.2349

SOLUTION | EX. 10.69

(to fill in)

EXERCISE 10.70 · *Q10 — 3 pts* | § 10.4

If the clock frequency of SAP-1 is 2 MHz, how long does it take to run a program that multiplies a number by two and display the result?

SOLUTION | EX. 10.70

(to fill in)

EXERCISE 10.71 · *Q11 — 3 pts* | § 10.4

*What does the following program for SAP-3 do?

Address 0x800 0x803 0x804 0x805 0x806 0x807 0x808 0x80B 0x80C 0x80F

Mnemonics LXI H 0x2501 MOV C M MOV A C INX H SUB M MOV A C JP 0x80C MOV A M STA 0x2503 HLT*

SOLUTION | EX. 10.71

(to fill in)

EXERCISE 10.72 · *Q12 — 4 pts* | § 10.4

Write a program for SAP-2 that swaps the two nibbles of data at memory address 0x2001 using as few clock cycles as possible

SOLUTION | EX. 10.72

(to fill in)

10.5 Exam — 23 June 2023

EXERCISE 10.73 · *Q1 — 2 pts* | § 10.5

*Using algebraic manipulation, prove that □ □ + □ □ + □ □ + □ □ =

(□ + □ + □ + □)(□ + □ + □ + □)*

SOLUTION | EX. 10.73

(to fill in)

EXERCISE 10.74 · *Q2 — 2 pts* | § 10.5

Using 2-bit □□□ gates, design a 2-bit □□□□ gate

SOLUTION | EX. 10.74

(to fill in)

EXERCISE 10.75 · *Q3 — 2 pts* | § 10.5

Find the minterms of the following expression □(□ □ + □ □) + □(□ + □ □)

SOLUTION | EX. 10.75

(to fill in)

EXERCISE 10.76 · *Q4 — 2 pts* | § 10.5

Find the truth table of the flip-flop with the following equation □□+1 = [(□ + □)□]□

SOLUTION | EX. 10.76

(to fill in)

EXERCISE 10.77 · *Q5 — 2 pts* | § 10.5

Make the above flip-flop from a J-K flip-flop and from a D flip-flop

SOLUTION | EX. 10.77

(to fill in)

EXERCISE 10.78 · *Q6 — 4 pts* | § 10.5

*Design (draw the circuit of) a 3-bit counter that counts by one when a control line is

set to zero and by two when it is set to one*

SOLUTION | EX. 10.78

(to fill in)

EXERCISE 10.79 · *Q7 — 4 pts* | § 10.5

Design (draw the Moore diagram and the circuit of) a state machine that, given an input sequence of bits, outputs a 1 when exactly two of the last three inputs are 1. For instance, the input sequence 011011100 produces an output sequence of 001111010

SOLUTION | EX. 10.79

(to fill in)

EXERCISE 10.80 · *Q8 — 2 pts* | § 10.5

Using 2's complement notation and 16 bits, write in binary the decimal number -122

SOLUTION | EX. 10.80

(to fill in)

EXERCISE 10.81 · *Q9 — 2 pts* | § 10.5

*Find the decimal value of the sum of two floating point (IEEE 754) numbers:

0xC1B90000 and 0x40100000*

SOLUTION | EX. 10.81

(to fill in)

EXERCISE 10.82 · *Q10 — 3 pts* | § 10.5

If the clock frequency of SAP-1 is 50 MHz, how long does it take to run a program that calculates the expression □ + 3□ and displays the result? □ and □ are data stored in memory locations

SOLUTION | EX. 10.82

(to fill in)

EXERCISE 10.83 · *Q11 — 3 pts* | § 10.5

*What operation does the following program for SAP-3 do? Mnemonics MVI B 0x01 MVI C 0x00 MOV A C ADD B MOV C A INR B CPI 0x0B JNZ 0x0804 MOV A C STA 0x2500 HLT

Address 0x0800 0x0802 0x0804 0x0805 0x0806 0x0807 0x0808 0x080A 0x080D 0x080E 0x0811*

SOLUTION | EX. 10.83

(to fill in)

EXERCISE 10.84 · *Q12 — 4 pts* | § 10.5

Write a program for SAP-2 to floor (round down) a given 8-bit number to the nearest multiple of 8

SOLUTION | EX. 10.84

(to fill in)

10.6 Exam — 21 July 2023

EXERCISE 10.85 · *Q1 — 2 pts* | § 10.6

Using algebraic manipulation, prove that □ □ □ + □ □ □ + □□ + □ □ = □ + □ □

SOLUTION | EX. 10.85

(to fill in)

EXERCISE 10.86 · *Q2 — 2 pts* | § 10.6

How many NAND gate are needed for implementing a NOR gate? Draw the circuit

SOLUTION | EX. 10.86

(to fill in)

EXERCISE 10.87 · *Q3 — 2 pts* | § 10.6

Find the minterms of the following expression □(□ + □) + □(□ + □ + □)

SOLUTION | EX. 10.87

(to fill in)

EXERCISE 10.88 · *Q4 — 2 pts* | § 10.6

The output □ of a flip-flop inverts from its previous state when its input □ is 0, otherwise it remains unchanged. Write the truth table of a toggle flip-flop

SOLUTION | EX. 10.88

(to fill in)

EXERCISE 10.89 · *Q5 — 2 pts* | § 10.6

Make the above flip-flop from a J-K flip-flop and from a D flip-flop

SOLUTION | EX. 10.89

(to fill in)

EXERCISE 10.90 · *Q6 — 4 pts* | § 10.6

Design (draw the circuit of) a decimal counter that counts from -3 to 5 and loopback using the 2's complement for the representation of negative numbers

SOLUTION | EX. 10.90

(to fill in)

EXERCISE 10.91 · *Q7 — 4 pts* | § 10.6

Design (draw the Moore diagram and the circuit of) a state machine able to detect the sequence (1,1) -> (1,1) -> (1,0) of two bits (M,N)

SOLUTION | EX. 10.91

(to fill in)

EXERCISE 10.92 · *Q8 — 2 pts* | § 10.6

Using bit-masking operations invert the fourteenth bit (starting from the MSB) of the 32-bit register □0

SOLUTION | EX. 10.92

(to fill in)

EXERCISE 10.93 · *Q9 — 2 pts* | § 10.6

Using 5 bits for the fractional part, write in fixed point representation the decimal number 77.375

SOLUTION | EX. 10.93

(to fill in)

EXERCISE 10.94 · *Q10 — 3 pts* | § 10.6

*How many instruction cycles and how many clock cycles are need by SAP-1 for adding

three numbers located in its memory?*

SOLUTION | EX. 10.94

(to fill in)

EXERCISE 10.95 · *Q11 — 3 pts* | § 10.6

*What operation does the following program for SAP-3 do? Mnemonics MVI B 0x00 MVI C 0x08 MOV A D RAR JNC 0x080A INR B DCR C JNZ 0x0805 HLT

Address 0x0800 0x0802 0x0804 0x0805 0x0806 0x0809 0x080A 0x080B 0x080E*

SOLUTION | EX. 10.95

(to fill in)

EXERCISE 10.96 · *Q12 — 4 pts* | § 10.6

*Write a program for SAP-3 that erases (writes 0x00 into) memory location 0x1070

through 0x1080 using as few lines of code as possible*

SOLUTION | EX. 10.96

(to fill in)

10.7 Exam — 1 September 2023

EXERCISE 10.97 · *Q1 — 2 pts* | § 10.7

*Given that □ □ = 0 and □ + □ = 1, use algebraic manipulation to prove that

(□ + □) · (□ + □) · (□ + □) = □ □*

SOLUTION | EX. 10.97

(to fill in)

EXERCISE 10.98 · *Q2 — 2 pts* | § 10.7

Using only NAND ports, design a 2-bit multiplexer

SOLUTION | EX. 10.98

(to fill in)

EXERCISE 10.99 · *Q3 — 2 pts* | § 10.7

*Using the graphical representation method prove that □ □ + □ □ = □ + □ □ +

□ □ □*

SOLUTION | EX. 10.99

(to fill in)

EXERCISE 10.100 · *Q4 — 2 pts* | § 10.7

*Find the truth table of the flip-flop with the following equation □□+1 =

(□□ + □□) □ □□*

SOLUTION | EX. 10.100

(to fill in)

EXERCISE 10.101 · *Q5 — 2 pts* | § 10.7

Make the above flip-flop from a J-K flip-flop and from a D flip-flop

SOLUTION | EX. 10.101

(to fill in)

EXERCISE 10.102 · *Q6 — 4 pts* | § 10.7

*Design (draw the circuit of) a decimal counter that counts from 0 to 15 by three and

loopbacks (0, 3, 6, 9, 12, 15, 0, 3...)*

SOLUTION | EX. 10.102

(to fill in)

EXERCISE 10.103 · *Q7 — 4 pts* | § 10.7

*Design (draw the Moore diagram and the circuit of) a state machine able to detect

the transition (0,1) -> (1,0) -> (1,1) of two bits (M,N)*

SOLUTION | EX. 10.103

(to fill in)

EXERCISE 10.104 · *Q8 — 2 pts* | § 10.7

*Using bit-masking operations clear the twelfth bit (starting from the LSB) of the 32-bit

register □0*

SOLUTION | EX. 10.104

(to fill in)

EXERCISE 10.105 · *Q9 — 2 pts* | § 10.7

Using 8 bits for the fractional part, write in fixed point representation the decimal number 14.293

SOLUTION | EX. 10.105

(to fill in)

EXERCISE 10.106 · *Q10 — 3 pts* | § 10.7

If the clock frequency of SAP-1 is 2.5 MHz, how long does it take to run a program that multiplies a number by three and display the result?

SOLUTION | EX. 10.106

(to fill in)

EXERCISE 10.107 · *Q11 — 3 pts* | § 10.7

*What does the following program for SAP-2 do?

Address 0x800 0x803 0x805 0x808 0x80B 0x80D 0x80E 0x80F 0x810 0x811 0x814

Mnemonics LDA 0x2500 ANI 0x0F STA 0x2501 LDA 0x2500 ANI 0xF0 RAR RAR RAR RAR STA 0x2502 HLT*

SOLUTION | EX. 10.107

(to fill in)

EXERCISE 10.108 · *Q12 — 4 pts* | § 10.7

Write a program for SAP-3 that counts the number of one's in a vector of 8 bit

SOLUTION | EX. 10.108

(to fill in)

10.8 Exam — 13 November 2023

EXERCISE 10.109 · *Q1 — 2 pts* | § 10.8

*Using algebraic manipulation prove that

□□ + □□ + □□□□ = □□ + □□□ + □□□*

SOLUTION | EX. 10.109

(to fill in)

EXERCISE 10.110 · *Q2 — 2 pts* | § 10.8

Using only NOR ports, design a 2-bit multiplexer

SOLUTION | EX. 10.110

(to fill in)

EXERCISE 10.111 · *Q3 — 2 pts* | § 10.8

*Find the minterms of the following expression □(□ + □) + □(□ + □ + □)

(Suggestion: use the graphical method)*

SOLUTION | EX. 10.111

(to fill in)

EXERCISE 10.112 · *Q4 — 2 pts* | § 10.8

*Find the truth table of the flip-flop with the following equation

□□+1 = (□ + □□) □ □□*

SOLUTION | EX. 10.112

(to fill in)

EXERCISE 10.113 · *Q5 — 2 pts* | § 10.8

Make the above flip-flop from a J-K flip-flop or from a D flip-flop

SOLUTION | EX. 10.113

(to fill in)

EXERCISE 10.114 · *Q6 — 4 pts* | § 10.8

*Design (draw the circuit of) a decimal counter that starting from 9 counts downwards

by two (9, 7, 5, 3, 1, 9, 7, 5...)*

SOLUTION | EX. 10.114

(to fill in)

EXERCISE 10.115 · *Q7 — 4 pts* | § 10.8

Design (draw the Moore diagram and the circuit of) a state machine able to detect the transition (0,1) -> (1,0) -> (1,1) of two bits (M,N)

SOLUTION | EX. 10.115

(to fill in)

EXERCISE 10.116 · *Q8 — 2 pts* | § 10.8

Using 2's complement notation and 10 bits, write in binary the decimal number -98

SOLUTION | EX. 10.116

(to fill in)

EXERCISE 10.117 · *Q9 — 2 pts* | § 10.8

Using 5 bits for the fractional part, write in fixed point representation the decimal number 22.5628

SOLUTION | EX. 10.117

(to fill in)

EXERCISE 10.118 · *Q10 — 3 pts* | § 10.8

Write a program for SAP-1 that executes and displays the result of the arithmetic operation 201-15-62

SOLUTION | EX. 10.118

(to fill in)

EXERCISE 10.119 · *Q11 — 3 pts* | § 10.8

*What does the following program for SAP-3 do?

Address 0x2000 0x2002 0x2004 0x2007 0x2008 0x200B 0x200C 0x200D 0x2010 0x2011 0x2014

Mnemonics MVI B 0x00 MVI C 0x08 LDA 0x3000 RAR JNC 0x200C INR B DCR C JNZ 0x2007 MOV A B STA 0x3001 HLT*

SOLUTION | EX. 10.119

(to fill in)

EXERCISE 10.120 · *Q12 — 4 pts* | § 10.8

Write a program for SAP-2 that returns the reminder of a division by 8 of a given 8-bit number, using as few clock cycles as possible

SOLUTION | EX. 10.120

(to fill in)

10.9 Exam — 25 January 2024

EXERCISE 10.121 · *Q1 — 2 pts* | § 10.9

*Using algebraic manipulation prove that

□□ + □□ + □□□ = □□ + □□□ + □□ (□□ + □□)*

SOLUTION | EX. 10.121

(to fill in)

EXERCISE 10.122 · *Q2 — 2 pts* | § 10.9

How many NOR gate are needed for implementing a NAND gate? Draw the circuit

SOLUTION | EX. 10.122

(to fill in)

EXERCISE 10.123 · *Q3 — 2 pts* | § 10.9

*Using the graphical representation method, prove that

□□ + □□ = □□ + □□ + □□ + □□*

SOLUTION | EX. 10.123

(to fill in)

EXERCISE 10.124 · *Q4 — 2 pts* | § 10.9

The output □ of an Enable flip-flop registers its input □ when its input □ is 1, otherwise it remains unchanged. Write the truth table of an Enable flip-flop

SOLUTION | EX. 10.124

(to fill in)

EXERCISE 10.125 · *Q5 — 2 pts* | § 10.9

Make the Enable flip-flop from a J-K flip-flop or from a D flip-flop

SOLUTION | EX. 10.125

(to fill in)

EXERCISE 10.126 · *Q6 — 4 pts* | § 10.9

Design (draw the circuit of) a decimal counter that counts from -5 to 2 and loopback using the 2's complement for the representation of negative numbers

SOLUTION | EX. 10.126

(to fill in)

EXERCISE 10.127 · *Q7 — 4 pts* | § 10.9

Design (draw the Moore diagram and the circuit of) a state machine able to detect the sequence ...101110... on a data line □

SOLUTION | EX. 10.127

(to fill in)

EXERCISE 10.128 · *Q8 — 2 pts* | § 10.9

Using bit-masking operations invert the eleventh bit (starting from the MSB) of the 32-bit register □0

SOLUTION | EX. 10.128

(to fill in)

EXERCISE 10.129 · *Q9 — 2 pts* | § 10.9

Sum the two floating point (IEEE 754) numbers: 0x40ca0000 and 0x3f980000

SOLUTION | EX. 10.129

(to fill in)

EXERCISE 10.130 · *Q10 — 3 pts* | § 10.9

*How many instruction cycles and how many clock cycles are needed by SAP-1 for

adding four 8-bit numbers located in its memory?*

SOLUTION | EX. 10.130

(to fill in)

EXERCISE 10.131 · *Q11 — 3 pts* | § 10.9

*What does the following program for SAP-3 do?

Address 0x2000 0x2003 0x2004 0x2005 0x2006 0x2009 0x200A 0x200D 0x200E 0x200F 0x2010 0x2013

Mnemonics LXI H 0x2050 MOV A M INX H SUB M JP 0x200E MOV A M STA 0x2090 HLT DCX H MOV A M STA 0x2090 HLT*

SOLUTION | EX. 10.131

(to fill in)

EXERCISE 10.132 · *Q12 — 4 pts* | § 10.9

*Write a program for SAP-2 that copies the LSB of the data at memory address 0x2001

to its MSB (overwriting it) using as few clock cycles as possible*

SOLUTION | EX. 10.132

(to fill in)

10.10 Exam — 22 February 2024

EXERCISE 10.133 · *Q1 — 2 pts* | § 10.10

*Using algebraic manipulation prove that

□ + □ □ + □ □ = □ + □ + □*

SOLUTION | EX. 10.133

(to fill in)

EXERCISE 10.134 · *Q2 — 2 pts* | § 10.10

Using 2-bit □□□□ gates, design a □□□ gate

SOLUTION | EX. 10.134

(to fill in)

EXERCISE 10.135 · *Q3 — 2 pts* | § 10.10

*Find the maxterms of the following expression

□(□ □ + □ □) + □(□ + □ □)*

SOLUTION | EX. 10.135

(to fill in)

EXERCISE 10.136 · *Q4 — 2 pts* | § 10.10

*Find the truth table of the flip-flop with the following equation

□□+1 = [□□ □ □□]*

SOLUTION | EX. 10.136

(to fill in)

EXERCISE 10.137 · *Q5 — 2 pts* | § 10.10

Make the above flip-flop from a J-K flip-flop or from a D flip-flop

SOLUTION | EX. 10.137

(to fill in)

EXERCISE 10.138 · *Q6 — 4 pts* | § 10.10

Design (draw the circuit of) a 3-bit counter that counts by one upwards when a control line is set to zero and downwards by two when it is set to one

SOLUTION | EX. 10.138

(to fill in)

EXERCISE 10.139 · *Q7 — 4 pts* | § 10.10

Design (draw the Moore diagram and the circuit of) a state machine able to detect the sequence (1,0) → (1,0) → (1,1) of two bits (□, □)

SOLUTION | EX. 10.139

(to fill in)

EXERCISE 10.140 · *Q8 — 2 pts* | § 10.10

Using 2's complement notation and 14 bits, write in binary the decimal number -99

SOLUTION | EX. 10.140

(to fill in)

EXERCISE 10.141 · *Q9 — 2 pts* | § 10.10

Using 4 bits for the fractional part, write in fixed point representation the decimal number 143.846

SOLUTION | EX. 10.141

(to fill in)

EXERCISE 10.142 · *Q10 — 3 pts* | § 10.10

If the clock frequency of SAP-1 is 40MHz, how long does it take to run the program that calculates the sum of two 8-bit numbers and displays the result?

SOLUTION | EX. 10.142

(to fill in)

EXERCISE 10.143 · *Q11 — 3 pts* | § 10.10

*What does the following program for SAP-2 do?

Address 0x800 0x803 0x805 0x808 0x80B 0x80D 0x80E 0x80F 0x810 0x811 0x814

Mnemonics LDA 0x2500 ANI 0x0F STA 0x2501 LDA 0x2500 ANI 0xF0 RAR RAR RAR RAR STA 0x2502 HLT*

SOLUTION | EX. 10.143

(to fill in)

EXERCISE 10.144 · *Q12 — 4 pts* | § 10.10

Write a program for SAP-3 that counts the number of ones in a vector of 8 bit

SOLUTION | EX. 10.144

(to fill in)

10.11 Exam — 20 June 2024

EXERCISE 10.145 · *Q1 — 2 pts* | § 10.11

*Using algebraic manipulation prove that

□ + □ □ + □ □ = □ + □ + □*

SOLUTION | EX. 10.145

(to fill in)

EXERCISE 10.146 · *Q2 — 2 pts* | § 10.11

Implement a 3-input AND gate using only two 2-input AND gate. How many 2-input NAND gate are needed for implementing a 3-input NAND gate?

SOLUTION | EX. 10.146

(to fill in)

EXERCISE 10.147 · *Q3 — 2 pts* | § 10.11

*Find the maxterms of the following expression

□(□ □ + □ □) + □(□ + □ □)*

SOLUTION | EX. 10.147

(to fill in)

EXERCISE 10.148 · *Q4 — 2 pts* | § 10.11

*Find the truth table of the flip-flop with the following equation

□□+1 = □□□ · □□□*

SOLUTION | EX. 10.148

(to fill in)

EXERCISE 10.149 · *Q5 — 2 pts* | § 10.11

Make the above flip-flop from a J-K flip-flop

SOLUTION | EX. 10.149

(to fill in)

EXERCISE 10.150 · *Q6 — 4 pts* | § 10.11

Design (draw the circuit of) a decimal counter that counts downwards by two from 15 to 1 and restart

SOLUTION | EX. 10.150

(to fill in)

EXERCISE 10.151 · *Q7 — 4 pts* | § 10.11

Design (draw the Moore diagram and the circuit of) a state machine able to detect the sequence ...010111... on a data line □

SOLUTION | EX. 10.151

(to fill in)

EXERCISE 10.152 · *Q8 — 2 pts* | § 10.11

Using bit-masking operations set the twelfth bit (starting from the LSB) of the 32-bit register □0

SOLUTION | EX. 10.152

(to fill in)

EXERCISE 10.153 · *Q9 — 2 pts* | § 10.11

Using 8 bits for the fractional part, write in fixed point representation the decimal number 35.623

SOLUTION | EX. 10.153

(to fill in)

EXERCISE 10.154 · *Q10 — 3 pts* | § 10.11

If the clock frequency of SAP-1 is 8MHz, how long does it take to run the program that calculates the sum of three 8-bit numbers and displays the result?

SOLUTION | EX. 10.154

(to fill in)

EXERCISE 10.155 · *Q11 — 3 pts* | § 10.11

*What does the following program for SAP-3 do?

Address 0x800 0x803 0x804 0x805 0x806 0x807 0x808 0x80B 0x80C 0x80F

Mnemonics LXI H 0x2501 MOV C M MOV A C INX H SUB M MOV A C JP 0x80C MOV A M STA 0x2503 HLT*

SOLUTION | EX. 10.155

(to fill in)

EXERCISE 10.156 · *Q12 — 4 pts* | § 10.11

Write a program for SAP-2 that counts the number of ones in a vector of 8 bit

SOLUTION | EX. 10.156

(to fill in)

10.12 Exam — 11 July 2024

EXERCISE 10.157 · *Q1 — 2 pts* | § 10.12

Using algebraic manipulation prove that □□ + □□ + □□ = □□ + □□ + □□

SOLUTION | EX. 10.157

(to fill in)

EXERCISE 10.158 · *Q2 — 2 pts* | § 10.12

How many NOR gate are needed for implementing a XNOR (□□□) gate? Draw the circuit

SOLUTION | EX. 10.158

(to fill in)

EXERCISE 10.159 · *Q3 — 2 pts* | § 10.12

*Using the graphical simplification method, simplify as much as possible the function

□□□□ + □□□□ + □□□□ + □□□□ + □□□□ + □□□□ + □□□□ + □□□□ + □□□□ + □□□□*

SOLUTION | EX. 10.159

(to fill in)

EXERCISE 10.160 · *Q4 — 2 pts* | § 10.12

*Find the truth table of the flip-flop with the following equation

□□+1 = [□ □ (□ □ □)]

□*

SOLUTION | EX. 10.160

(to fill in)

EXERCISE 10.161 · *Q5 — 2 pts* | § 10.12

Make the above flip-flop from a J-K flip-flop

SOLUTION | EX. 10.161

(to fill in)

EXERCISE 10.162 · *Q6 — 4 pts* | § 10.12

Design (draw the circuit of) a counter that counts by two from 0 to 12 and then it stops (i.e. it remains at 12)

SOLUTION | EX. 10.162

(to fill in)

EXERCISE 10.163 · *Q7 — 4 pts* | § 10.12

Design (draw the Moore diagram and the circuit of) a device able to find the sequence ...111000... on a data line

SOLUTION | EX. 10.163

(to fill in)

EXERCISE 10.164 · *Q8 — 2 pts* | § 10.12

Using 2's complement notation and 11 bits, write in binary the decimal number -623

SOLUTION | EX. 10.164

(to fill in)

EXERCISE 10.165 · *Q9 — 2 pts* | § 10.12

Find the decimal value of the floating point (IEEE 754) number 0x411A0000

SOLUTION | EX. 10.165

(to fill in)

EXERCISE 10.166 · *Q10 — 3 pts* | § 10.12

Write a program for SAP-1 that executes and displays the result of the arithmetic operation 220+25+32

SOLUTION | EX. 10.166

(to fill in)

EXERCISE 10.167 · *Q11 — 3 pts* | § 10.12

What operation does the following program for SAP-2 do? Suggest an alternative program for performing the same operation Address 0x800 0x803 0x804 0x805 0x808 Mnemonics LDA 0x2100 CMA INR A STA 0x2101 HLT

SOLUTION | EX. 10.167

(to fill in)

EXERCISE 10.168 · *Q12 — 4 pts* | § 10.12

Write a program for SAP-3 that checks if a signed number loaded into the accumulator is positive or negative using as few clock cycles as possible

SOLUTION | EX. 10.168

(to fill in)

10.13 Exam — 29 August 2024

EXERCISE 10.169 · *Q1 — 2 pts* | § 10.13

Using algebraic manipulation, prove that □□ + □□ + □□ + □□ = (□ + □ + □ + □)(□ + □ + □ + □)

SOLUTION | EX. 10.169

(to fill in)

EXERCISE 10.170 · *Q2 — 2 pts* | § 10.13

Using 2-bit input multiplexers, design a 4-bit input AND port using as less select lines as possible

SOLUTION | EX. 10.170

(to fill in)

EXERCISE 10.171 · *Q3 — 2 pts* | § 10.13

Find the maxterms of the following expression □□ + □□ + □□□

SOLUTION | EX. 10.171

(to fill in)

EXERCISE 10.172 · *Q4 — 2 pts* | § 10.13

*Find the truth table of the flip-flop with the following equation

□□+1 = [□(□ □ □)]□*

SOLUTION | EX. 10.172

(to fill in)

EXERCISE 10.173 · *Q5 — 2 pts* | § 10.13

Make the above flip-flop from a D flip-flop and from a J-K flip flop

SOLUTION | EX. 10.173

(to fill in)

EXERCISE 10.174 · *Q6 — 4 pts* | § 10.13

Design (draw the circuit of) a decimal counter that counts downwards from 4 to -4 and loopback using the 2's complement for the representation of negative numbers

SOLUTION | EX. 10.174

(to fill in)

EXERCISE 10.175 · *Q7 — 4 pts* | § 10.13

Design (draw the Moore diagram and the circuit of) a state machine able to detect the sequence ...111001... on a data line □

SOLUTION | EX. 10.175

(to fill in)

EXERCISE 10.176 · *Q8 — 2 pts* | § 10.13

Using bit-masking operations clear the seventh bit (starting from the MSB) of the 32-bit register □0

SOLUTION | EX. 10.176

(to fill in)

EXERCISE 10.177 · *Q9 — 2 pts* | § 10.13

Using 8 bits for the fractional part, write in fixed point representation the decimal number 23.1633

SOLUTION | EX. 10.177

(to fill in)

EXERCISE 10.178 · *Q10 — 3 pts* | § 10.13

If the clock frequency of SAP-1 is 2 MHz, how long does it take to run a program that multiplies a number by three and displays the result?

SOLUTION | EX. 10.178

(to fill in)

EXERCISE 10.179 · *Q11 — 3 pts* | § 10.13

*What does the following program for SAP-3 do?

Address 0x800 0x803 0x804 0x805 0x806 0x808 0x809 0X80A 0X80C 0x80D 0x80F 0x812

Mnemonics LXI H 0x107F INX H MOV B M MOV A L ADI 0x20 MOV L A MOV M B SUI 0x20 MOV L A CPI 0x8F JNZ 0x803 HLT*

SOLUTION | EX. 10.179

(to fill in)

EXERCISE 10.180 · *Q12 — 4 pts* | § 10.13

Write a program for SAP-2 that adds the two nibbles of data at memory address 0x2001 using as few clock cycles as possible

SOLUTION | EX. 10.180

(to fill in)

PhyAMR

ASCII Table

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	|
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

FIG. 11.1 image

PhyAMR

Times Table of 16

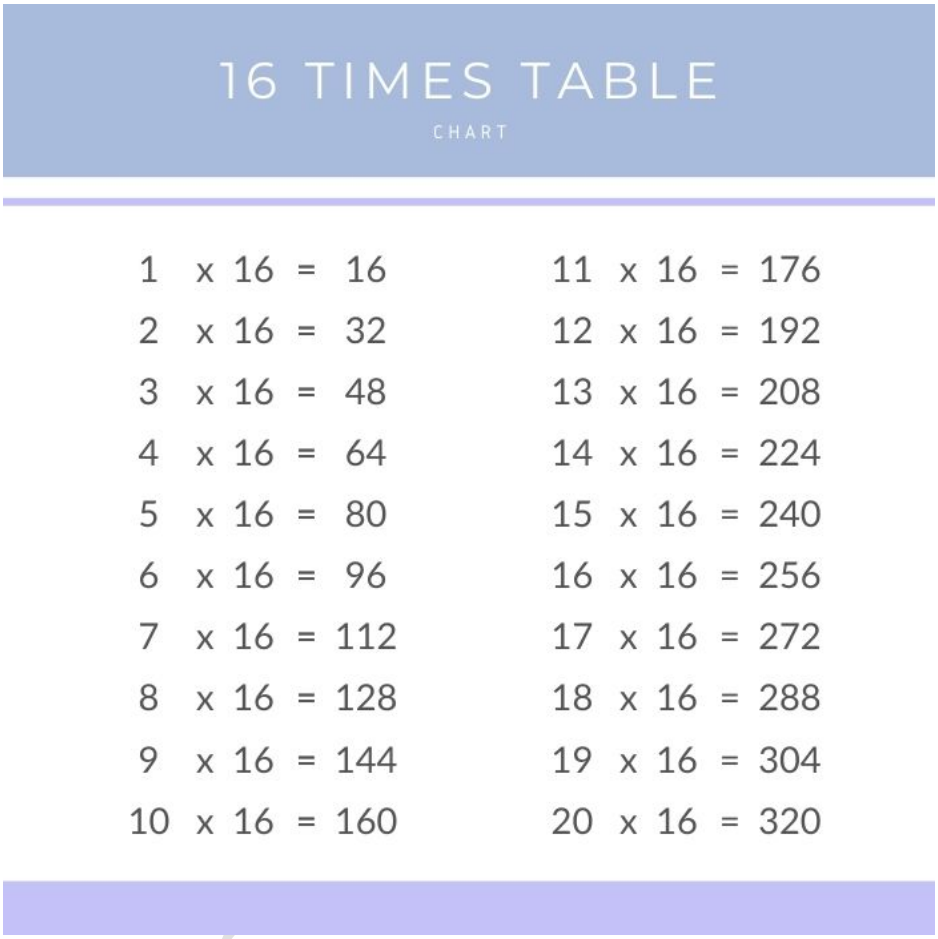


FIG. 12.1 image

PhyAMR

Throughput calculation in RAID 0

Let n be the number of volumes used to store a quantity of data D involved in an I/O operation, and let's assume for simplicity that they are equally distributed across the n units, so that each of them contains exactly $\frac{D}{n}$ data.

The average throughput τ_m will be given by the ratio between the total memory D occupied by the data and the sum of the times with which the individual disks perform a read/write operation:

$$\tau_m = \frac{D}{\sum_{i=1}^{n} t_i}$$

The time t_i is given by the ratio between the amount of data on a disk, which is $\frac{D}{n}$, and the throughput of the relative disk:

$$t_i = \frac{D}{n\tau_i}$$

From which follows

$$\tau_m = \frac{\cancel{D}}{\frac{\cancel{D}}{n}\sum_{i=1}^{n}\frac{1}{\tau_i}} = \frac{n}{\frac{1}{\tau_1}+\frac{1}{\tau_2}+\dots+\frac{1}{\tau_n}} = \frac{n\tau_1\tau_2\dots\tau_n}{\tau_2\tau_3\dots\tau_n+\tau_1\tau_3\dots\tau_n+\dots+\tau_1\tau_2\dots\tau_{n-1}}$$

Which expressed in a compact way will be

$$\tau_m = \frac{n\prod_{i=1}^{n}\tau_i}{\sum_{i=1}^{n}\left(\prod_{j=0\ j\neq i}^{n}\tau_j\right)} \implies \begin{aligned} n=2 &\rightarrow \tau_m = \frac{n\tau_1\tau_2}{\tau_1+\tau_2} \\ n=3 &\rightarrow \tau_m = \frac{n\tau_1\tau_2\tau_3}{\tau_1\tau_2+\tau_1\tau_3+\tau_2\tau_3} \\ &\dots \end{aligned}$$

PhyAMR

PhyAMR

Cheatsheet

PhyAMR

PhyAMR

Management and Analysis of Physics Datasets — Module A: Cheatsheet

Number Systems

Bases: dec, bin (b), oct (o), hex (h, 0x).
dec → bin (double-dabble): repeat //2, remainder is the next bit (LSB first).
bin → dec: $\sum b_i\, 2^i$.
bin → hex: group 4 bits from the LSB.
Powers: $2^{10} = 1024$, $2^{16} = 65536$;
1 kB $= 10^3$ B, 1 KiB $= 2^{10}$ B.
1 nibble = 4 bits, 1 byte = 8 bits.
Range, unsigned n bits: $[0,\ 2^n - 1]$.

2's Complement

Negative number in n bits:
1. write $|x|$ in binary on n bits;
2. invert every bit (1's complement);
3. add 1.

Range: $[-2^{n-1},\ 2^{n-1} - 1]$.
MSB acts as sign: $0{=}+$, $1{=}-$.
Sign extension: copy the MSB to widen.
Negation: $-x = \bar{x} + 1$.
Subtraction: $A - B = A + \overline{B} + 1$.
Ex. -765 on 11 bits: $765 = 01011111101_2 \to 10100000010 + 1 = 10100000011$.

Fixed Point

Integer part in binary · fractional part with weights $2^{-1}, 2^{-2}, \ldots$
Fractional algorithm: multiply $\times 2$, the integer part of the result is the next bit (MSB first).
Ex. $0.625 \to 1.25 \to 0.5 \to 1.0 \Rightarrow .101$.
With f fractional bits the resolution is 2^{-f}. Round to the nearest representable value.

Floating Point (IEEE 754)

IEEE 754 single (32 bit):
1 sign — 8 exponent (bias 127) — 23 mantissa.
$x = (-1)^S\,(1.m)\,2^{E-127}$.
Decode hex:
1. write 32 bits;
2. read S, E, mantissa;
3. $E_{\text{real}} = E - 127$;
4. recompose $\pm 1.m \cdot 2^{E_{\text{real}}}$.

Sum: align exponents (right-shift the smaller mantissa by ΔE, restore the hidden 1), add, renormalise.
Ex. 0x41160000 $= 0\,10000010\,0010110\ldots = +1.0010110_2 \cdot 2^3 = 9.375$.

Boolean Algebra

P1 $0, 1$ are the only values; $A \neq 0 \Leftrightarrow A = 1$.
P2 (id.) $A + 0 = A$, $A \cdot 1 = A$.
P3 (compl.) $A + \overline{A} = 1$, $A \cdot \overline{A} = 0$.
P4 (distr.) $A(B+C) = AB + AC$; $A + BC = (A+B)(A+C)$.
T1 (idemp.) $A + A = A$, $AA = A$.
T2 (abs.) $A + 1 = 1$, $A \cdot 0 = 0$.
T3 (cov.) $A + AB = A$, $A(A+B) = A$.
T4 $A + \overline{A}B = A + B$, $A(\overline{A} + B) = AB$.
T5 (consensus) $AB + \overline{A}C + BC = AB + \overline{A}C$.

De Morgan & XOR

De Morgan:

$$\overline{A+B} = \overline{A} \cdot \overline{B} \qquad \overline{A \cdot B} = \overline{A} + \overline{B}$$

Duality: swap $+ \leftrightarrow \cdot$ and $0 \leftrightarrow 1$; the result is still a theorem.
XOR: $A \oplus B = \overline{A}B + A\overline{B} = \overline{A \oplus B}$.
$A \oplus 0 = A$, $A \oplus 1 = \overline{A}$, $A \oplus A = 0$, $A \oplus \overline{A} = 1$.
Associative + commutative; $A \oplus B \oplus C$ is the 3-input parity.

Logic Gates

NAND and NOR are universal.

NOT	1 NAND or 1 NOR (tied inputs)
AND	NAND + NAND-as-NOT (2)
OR	NOR + NOR-as-NOT (2)
NAND	1 NAND
NOR	1 NOR
XOR	4 NAND or 4 NOR
XNOR	5 NAND or 5 NOR

Mux $2^n \to 1$: n select lines, output $= \sum_i S_i\, D_i$ with S_i minterm of selects.
Decoder = inverse of multiplexer.
Parity: cascade of XOR for even, XNOR for odd.

Minterms, Maxterms, K-maps

Minterm m_i: product where every variable appears (complemented if 0 in row i). Sum of all minterms with $f = 1$ gives the SOP form.
Maxterm M_i: sum where every variable appears (complemented if 1 in row i). Product of all maxterms with $f = 0$ gives the POS form.
$f = \sum_i m_i = \prod_j M_j$.
Karnaugh map: cells follow Gray code; group 1s in rectangles of 2^k (wrap edges + corners). The largest group eliminates the most variables.

Combinatorial Circuits

Half-adder $A + B$: $S = A \oplus B$, $C = AB$.
Full-adder $A + B + C_i$: $S = A \oplus B \oplus C_i$, $C_o = AB + (A \oplus B)C_i$.
Ripple-carry adder: chain full-adders; for sub use $A + \overline{B} + 1$ with the first $C_i = 1$.
Comparator: $A = B \Leftrightarrow \overline{A \oplus B}$ on each bit, AND them.
Multiplier: 2-bit columns of ANDs + adders.
Encoder $2^n \to n$, **Decoder** $n \to 2^n$ (one-hot output).

Flip-Flops

Truth tables (Q^{n+1}):

SR	$Q\overline{R} + S$, forbid $SR{=}1$
D	D
JK	$J\overline{Q} + \overline{K}Q$
T	$T \oplus Q$

Conversion (excitation):
$D = Q^{n+1}$ (just write the target equation).
$J = Q^{n+1}|_{Q=0}$, $K = \overline{Q^{n+1}}|_{Q=1}$.
Recipe for "flip-flop from equation E": build the truth table of $Q^{n+1} = E$, then read J, K (or D) row by row.

Counters

Use n flip-flops for 2^n states (or to count up to a custom max).
Up by k: state transition $S^{n+1} = S + k$ on each clock; if $S = \max$ go to start.
Down counter: $S^{n+1} = S - k$, wrap at 0.
Bidirectional (control C): $S^{n+1} = (S+k)\overline{C} + (S-k)C$.
Counts and stops at max: include $S = \max$ as an absorbing state.
Steps: (1) list states; (2) write next-state truth table; (3) extract J, K (or D) using the conversion above; (4) draw the circuit.

Moore Sequence Detector

Moore machine (output depends only on state):
1. identify states (one per prefix of the target sequence + an extra "all bits read" state with output 1);
2. draw transitions for each input symbol;
3. assign binary codes ($\lceil \log_2 N \rceil$ flip-flops);
4. write the next-state truth table;
5. derive J, K (or D) for each state bit;
6. output $Z = 1$ only in the accept state.

Always include the "fall-back" transition from each state for the wrong-symbol case. For sequences on bit pairs (M, N) each transition is labelled by a 2-bit input.

Bit-Masking

Let b be the bit position; mask $M = 1 \ll b$.

Set $R \leftarrow R \,\text{OR}\, M$
Clear $R \leftarrow R \,\text{AND}\, \overline{M}$
Invert $R \leftarrow R \,\text{XOR}\, M$
Check $(R \,\text{AND}\, M) \neq 0$

Numbering: "k-th bit from the LSB" means $b = k - 1$; "from the MSB" of an n-bit register means $b = n - k$.

Combine masks (OR) to act on multiple bits simultaneously.

SAP-1

Registers: PC, MAR, IR, A, B, OUT.
Word/instr: 8 bits, 4-bit opcode + 4-bit address.
Memory: 16 bytes (0x0 to 0xF).
Cycle: fetch (3 T-states) + execute (3 T-states) = 6 T per instruction.

LDA $\langle a \rangle$	$A \leftarrow M[a]$
ADD $\langle a \rangle$	$A \leftarrow A + M[a]$
SUB $\langle a \rangle$	$A \leftarrow A - M[a]$
OUT	display A
HLT	stop

Timing: $T = N_{\text{instr}} \cdot 6/f_{\text{clk}}$.
Ex. $f = 2$ MHz, multiply-by-two = LDA + ADD + OUT + HLT = 4 instr $\Rightarrow 24 \cdot 0.5\,\mu s = 12\,\mu s$.

SAP-2 Instructions

16-bit address bus; 0x0000–0x07FF ROM, rest RAM. Tri-state registers (A, B, C, TMP). Flags: sign S, zero Z.

Op-codes (8-bit) + immediate / address bytes (1–2):

LDA addr	$A \leftarrow M$[addr]
STA addr	M[addr] $\leftarrow A$
MVI X,i	$X \leftarrow i$ ($X \in \{A, B, C\}$)
MOV X,Y	$X \leftarrow Y$
ADD/SUB X	$A \leftarrow A \pm X$ (updates flags)
ADI/SUI i	$A \leftarrow A \pm i$
INR/DCR X	$X \leftarrow X \pm 1$
ANA/ORA/XRA X	bitwise on A
ANI/ORI/XRI i	immediate version
CMA	$A \leftarrow \overline{A}$
RAR/RAL	rotate A right/left (1 bit)
JMP/JZ/JNZ/JM/JP addr	jump
CALL/RET addr	subroutine
OUT p / HLT	I/O / stop

SAP-3 Additions

SAP-2 instruction set, plus:

LXI H,ww	H:$L \leftarrow$ 16-bit word
MOV X,M / MOV M,X	memory via $H : L$
INX H / DCX H	16-bit increment of $H : L$
CMP X / CPI i	compare ($A - X$, set flags)
SUB M / ADD M	operand from $M[H : L]$

Tips for "program X" questions:

- trace registers/memory line by line;
- watch CALL/RET stack and the $H : L$ pointer;
- identify loops (DCR C + JNZ): typical pattern is a counter for multiplication, division, or bit-by-bit walk via RAR.

SAP-2/3 Recipes

Arithmetic via LDA/ADD/SUB: load, add or subtract addresses, OUT, HLT. Negative subtotals stay valid in 2's complement.

Multiplication by k: loop $k - 1$ adds, or CALL a subroutine that adds the multiplicand to itself; loop counter in C, decrement and JNZ.
Division by 2^k: k right-rotations (RAR) after clearing the carry.
Modulo 2^k: ANI $2^k - 1$.
Count ones: init counter $B = 0$, $C = 8$; loop: RAR, JNC skip, INR B, DCR C, JNZ; finally MOV A,B; STA.
Absolute value: JP done; CMA; INR A; done: ...
Sign test: RAR, then JC for negative.
Swap nibbles: RAR/RAL four times (or ANI/RAR + ANI + ORA).
Erase memory range: loop with LXI/MVI/MOV M,A/INX H/CPI.

Exam Recipes

Algebraic proof: expand with P4, kill terms with T2/T3, factor; alternatively reduce both sides to a canonical SOP and compare.

K-map simplification: draw the map, group, write the SOP / POS, sanity-check with the original equation on the corner rows.

Flip-flop design from $Q^{n+1} = E$:
1. truth table of E;
2. $J = Q^{n+1}|_{Q=0}$, $K = \overline{Q^{n+1}}|_{Q=1}$;
3. draw the JK with those gates feeding J, K.

Counter design: target sequence → next-state TT → J, K per bit → minimise with K-map.
Moore machine: prefix-states → TT → JK + output decoder.

Identities Toolbox

Quick algebra moves:

- $X + \overline{X}Y = X + Y$
- $XY + \overline{X}Z + YZ = XY + \overline{X}Z$ (consensus)
- $X(\overline{X} + Y) = XY$
- $X + XY = X$
- $\overline{X \oplus Y} = XY + \overline{X}\,\overline{Y}$ (XNOR)
- $X \oplus Y \oplus Z =$ odd-parity of 3 inputs

XOR / XNOR identities:
$A \oplus B = \overline{A \odot B}$, $A \oplus \overline{B} = \overline{A} \oplus B = A \odot B$,
$A \oplus (B \oplus C) = (A \oplus B) \oplus C$.

Tip: when forced to use only NAND or NOR, rewrite via De Morgan; usually ≤ 5 gates suffice for XOR/XNOR.